

A Practitioner’s Guide to Correlation, Dependence, and Feature Importance

Classical correlations, information-theoretic dependence, geometric/kernel measures, and feature-importance metrics

Nikolas Markou
nikolasmarkou@gmail.com

Abstract

This guide is an integrated, self-contained reference covering the four large families of metrics that quantify the relationship between variables in modern data analysis: classical correlation coefficients (Pearson, Spearman, Kendall, point-biserial, ϕ , Cramér’s V , tetrachoric/polychoric), information-theoretic dependence (mutual information, MIC, NMI, transfer entropy, CMI), modern geometric and kernel-based independence criteria (distance correlation, Brownian covariance, kernel-CCA, HSIC, Chatterjee’s ξ), and the dominant families of feature-importance metrics used in machine learning (SHAP, permutation importance, LIME, PDP/ICE, plus tree-specific impurity-based importances). Each metric is documented with its definition, derivation, asymptotic properties, code recipe, canonical pitfalls, and guidance on when to use — and when not to. A master comparison table classifies every measure along five properties (range, symmetry, captures non-linearity, requires distributional assumptions, computational cost), a prerequisites primer brings the reader up to speed on the supporting probability and statistics, and three appendices catalogue reference implementations and provide a fully worked numerical walkthrough.

Contents

1	Introduction	8
2	How to Read This Guide	9
3	Notation and Conventions	10
4	Prerequisites Primer	11
4.1	P1. Random variables, expectations, and moments	11
4.2	P2. Entropy, cross-entropy, KL divergence	11
4.3	P3. Characteristic functions	11
4.4	P4. Reproducing kernel Hilbert spaces (RKHS)	12
4.5	P5. U-statistics and V-statistics	12
4.6	P6. Bootstrap, permutation, jackknife	12
4.7	P7. Copulas	13
4.8	P8. Granger causality and DAGs	13
4.9	P9. Shapley values from cooperative game theory	13
5	The Five Properties That Distinguish Dependence Measures	14
6	Master Comparison Table	15
7	Part I — Classical and Rank Correlation Coefficients	16
7.1	1. Pearson's Product-Moment Correlation r	16
7.1.1	1.1 Definition	16
7.1.2	1.2 Derivation and geometric interpretation	17
7.1.3	1.3 Inference	17
7.1.4	1.4 Properties	17
7.1.5	1.5 The canonical counterexample	17
7.1.6	1.6 Pitfalls	17
7.1.7	1.7 Code	18
7.1.8	1.8 Worked tiny example	18
7.1.9	1.9 When to use / when not	18
7.1.10	1.10 Cross-references	18
7.2	2. Spearman's Rank Correlation ρ	18
7.2.1	2.1 Definition	18
7.2.2	2.2 Derivation of the shortcut formula	19
7.2.3	2.3 Intuition	19
7.2.4	2.4 Relationship to Pearson	19
7.2.5	2.5 Inference	19
7.2.6	2.6 Tie handling	19
7.2.7	2.7 Code	19
7.2.8	2.8 Worked tiny example	20
7.2.9	2.9 Pitfalls	20
7.2.10	2.10 When to use / when not	20
7.3	3. Kendall's τ	20
7.3.1	3.1 Definition	20
7.3.2	3.2 Probabilistic interpretation	20
7.3.3	3.3 Computation	20
7.3.4	3.4 Relationship to Spearman and Pearson	21
7.3.5	3.5 Inference	21
7.3.6	3.6 Code	21
7.3.7	3.7 Worked tiny example	21
7.3.8	3.8 Pitfalls and trade-offs	21

7.3.9	3.9 When to use / when not	22
7.4	4. Point-Biserial Correlation r_{pb}	22
7.4.1	4.1 Definition and equivalence to Pearson	22
7.4.2	4.2 Derivation	22
7.4.3	4.3 Relation to the two-sample t -test	22
7.4.4	4.4 Point-biserial versus biserial	22
7.4.5	4.5 The role of group balance, and a common false bound	23
7.4.6	4.6 Code	23
7.4.7	4.7 When to use / when not	23
7.5	5. The Phi Coefficient ϕ	23
7.5.1	5.1 Definition	23
7.5.2	5.2 Relation to χ^2	23
7.5.3	5.3 Bounds and marginal-dependence pathology	24
7.5.4	5.4 Versus the odds ratio	24
7.5.5	5.5 Code	24
7.5.6	5.6 When to use	24
7.6	6. Cramér's V	24
7.6.1	6.1 Definition	24
7.6.2	6.2 Derivation of the normalisation	24
7.6.3	6.3 Properties	24
7.6.4	6.4 Bias correction (Bergsma 2013)	25
7.6.5	6.5 Code	25
7.6.6	6.6 Pitfalls	25
7.6.7	6.7 When to use	25
7.7	7. Tetrachoric and Polychoric Correlation	25
7.7.1	7.1 The latent-normal model	25
7.7.2	7.2 Why bother	26
7.7.3	7.3 Estimation	26
7.7.4	7.4 Standard errors	26
7.7.5	7.5 Assumptions and pitfalls	26
7.7.6	7.6 Code	26
7.7.7	7.7 When to use	27
8	Part II — Information-Theoretic Dependence	30
8.1	8. Mutual Information (MI)	30
8.1.1	8.1 Definitions	30
8.1.2	8.2 Derivation of equivalences	30
8.1.3	8.3 Foundational properties	31
8.1.4	8.4 Estimation — the hard part	31
8.1.5	8.5 The canonical "MI sees what Pearson misses" example	32
8.1.6	8.6 Pitfalls	32
8.1.7	8.7 When to use / when not	32
8.1.8	8.8 Cross-references	32
8.2	9. Maximal Information Coefficient (MIC)	32
8.2.1	9.1 Definition	32
8.2.2	9.2 Intuition	32
8.2.3	9.3 The "equitability" claim and the controversy	33
8.2.4	9.4 Properties and performance	33
8.2.5	9.5 Code	33
8.2.6	9.6 Pitfalls	33
8.2.7	9.7 When to use	33
8.3	10. Normalized Mutual Information (NMI)	33
8.3.1	10.1 Family of definitions	33
8.3.2	10.2 Why normalise	34

8.3.3	10.3 Cluster evaluation — the dominant use case	34
8.3.4	10.4 Code	34
8.3.5	10.5 Pitfalls	34
8.3.6	10.6 When to use	35
8.4	11. Transfer Entropy	35
8.4.1	11.1 Definition	35
8.4.2	11.2 Intuition	35
8.4.3	11.3 Relation to Granger causality	35
8.4.4	11.4 Properties	35
8.4.5	11.5 Estimation	35
8.4.6	11.6 Code	36
8.4.7	11.7 Pitfalls	36
8.4.8	11.8 When to use	36
8.5	12. Conditional Mutual Information (CMI)	36
8.5.1	12.1 Definition	36
8.5.2	12.2 Intuition	36
8.5.3	12.3 Key identities	36
8.5.4	12.4 Estimation	37
8.5.5	12.5 Causal discovery — why it matters	37
8.5.6	12.6 Code	37
8.5.7	12.7 Pitfalls	37
8.5.8	12.8 When to use	37
9	Part III — Geometric, Kernel, and Modern Independence Measures	38
9.1	13. Distance Correlation (dCor)	38
9.1.1	13.1 Definition (sample)	38
9.1.2	13.2 Population definition (characteristic-function form)	38
9.1.3	13.3 Derivation of the equivalence between sample and population forms	39
9.1.4	13.4 Bias-corrected estimator	39
9.1.5	13.5 Properties	39
9.1.6	13.6 Asymptotic null distribution	39
9.1.7	13.7 Fast algorithms	39
9.1.8	13.8 Code	40
9.1.9	13.9 Pitfalls	40
9.1.10	13.10 When to use	40
9.2	14. Brownian Covariance	40
9.2.1	14.1 Definition	40
9.2.2	14.2 Equivalence to dCov	40
9.2.3	14.3 Why this equivalence matters	40
9.2.4	14.4 Practical implication	41
9.2.5	14.5 Generalisations	41
9.3	15. Kernel Canonical Correlation Analysis (KCCA)	41
9.3.1	15.1 From CCA to KCCA	41
9.3.2	15.2 Sample computation	41
9.3.3	15.3 Properties	41
9.3.4	15.4 Cost	41
9.3.5	15.5 Code	42
9.3.6	15.6 Deep CCA	42
9.3.7	15.7 When to use	42
9.4	16. Hilbert–Schmidt Independence Criterion (HSIC)	42
9.4.1	16.1 Definition (population)	42
9.4.2	16.2 Sample estimator	42
9.4.3	16.3 Why it characterises independence	42
9.4.4	16.4 Relationship to dCor	43

9.4.5	16.5 Asymptotic null distribution	43
9.4.6	16.6 Variants	43
9.4.7	16.7 Code	43
9.4.8	16.8 Kernel bandwidth selection	43
9.4.9	16.9 Pitfalls	44
9.4.10	16.10 When to use	44
9.5	17. Chatterjee's Rank Correlation ξ	44
9.5.1	17.1 Definition	44
9.5.2	17.2 Intuition	45
9.5.3	17.3 Theoretical headlines	45
9.5.4	17.4 Why "asymmetric" is a feature	45
9.5.5	17.5 Properties	45
9.5.6	17.6 Variants and extensions	45
9.5.7	17.7 Code	45
9.5.8	17.8 Pitfalls	46
9.5.9	17.9 When to use	46
10	Part IV — Model-Agnostic Feature Importance	47
10.1	18. SHAP (SHapley Additive exPlanations)	47
10.1.1	18.1 Setup	47
10.1.2	18.2 Formal definition	48
10.1.3	18.3 The value function: interventional vs observational	48
10.1.4	18.4 Estimators	48
10.1.5	18.5 Global aggregation	49
10.1.6	18.6 Code	49
10.1.7	18.7 Pitfalls	49
10.1.8	18.8 When to use	49
10.2	19. Permutation Feature Importance (PFI)	50
10.2.1	19.1 Algorithm	50
10.2.2	19.2 Intuition	50
10.2.3	19.3 Properties	50
10.2.4	19.4 The correlated-features trap	50
10.2.5	19.5 The extrapolation problem	50
10.2.6	19.6 Code	50
10.2.7	19.7 Pitfalls	51
10.2.8	19.8 When to use	51
10.3	20. LIME (Local Interpretable Model-agnostic Explanations)	51
10.3.1	20.1 Setup	51
10.3.2	20.2 Algorithm	51
10.3.3	20.3 Intuition	52
10.3.4	20.4 Code	52
10.3.5	20.5 Properties	52
10.3.6	20.6 Pitfalls	52
10.3.7	20.7 When to use	52
10.4	21. Partial Dependence (PDP) and Individual Conditional Expectation (ICE)	53
10.4.1	21.1 Partial dependence	53
10.4.2	21.2 Individual Conditional Expectation (ICE)	53
10.4.3	21.3 Accumulated Local Effects (ALE)	54
10.4.4	21.4 Code	54
10.4.5	21.5 Properties	54
10.4.6	21.6 Pitfalls	54
10.4.7	21.7 When to use	54

11 Part V — Tree-Specific and Embedded Metrics	55
11.1 22. Gini Importance / Mean Decrease in Impurity (MDI)	55
11.1.1 22.1 Definition	55
11.1.2 22.2 Intuition	55
11.1.3 22.3 Why MDI is biased — three reasons	55
11.1.4 22.4 Empirical demonstration of cardinality bias	55
11.1.5 22.5 Unbiased alternatives	56
11.1.6 22.6 When to use	56
11.2 23. Mean Decrease in Accuracy (MDA)	56
11.2.1 23.1 Algorithm	56
11.2.2 23.2 Relationship to PFI	56
11.2.3 23.3 Properties	56
11.2.4 23.4 Code	56
11.2.5 23.5 When to use	57
11.3 24. Gain, Split, and Cover Importance in Boosted Trees	57
11.3.1 24.1 Definitions	57
11.3.2 24.2 Differences from MDI	57
11.3.3 24.3 TreeSHAP as the modern replacement	57
11.3.4 24.4 Code	58
11.3.5 24.5 When to use	58
11.4 25. LASSO and Ridge Coefficients as Importance	58
11.4.1 25.1 Definitions	58
11.4.2 25.2 The geometry of L_1 vs L_2	58
11.4.3 25.3 Closed-form Ridge solution	58
11.4.4 25.4 Critical preprocessing	59
11.4.5 25.5 Stability under correlated features	59
11.4.6 25.6 Post-selection inference	59
11.4.7 25.7 Code	59
11.4.8 25.8 Other uses	59
11.4.9 25.9 Pitfalls	60
11.4.10 25.10 When to use	60
12 Cross-Cutting Themes	61
12.1 Choosing a Measure: A Decision Tree	61
12.2 Significance, Permutation Tests, and Confidence Intervals	61
12.3 Multiple Testing and the Garden of Forking Paths	62
12.4 Correlation, Causation, and Confounders	62
12.5 Pitfalls Common to All Metrics	63
13 Common Confusions and How to Resolve Them	64
13.1 "Pearson vs covariance"	64
13.2 "Spearman vs Pearson — which to default to?"	64
13.3 "MI vs correlation — interpretive equivalence?"	64
13.4 "dCor vs HSIC"	64
13.5 "SHAP vs PFI"	64
13.6 "MDI vs PFI vs SHAP for tree ensembles"	64
13.7 "LIME vs SHAP"	65
13.8 "Granger causality vs transfer entropy"	65
13.9 "Partial correlation vs CMI"	65
13.10 "Correlation between binary variables — should I use ϕ or the odds ratio?"	65
13.11 "Adjusted Rand Index vs Adjusted Mutual Information for clustering comparison"	65
13.12 "Why does my training-set PFI show high importance for noise features?"	65
13.13 "Why don't my SHAP values sum to the prediction?"	65

14 Glossary	67
15 Appendix A — Reference Implementations and Libraries	70
15.1 Python	70
15.2 R (selected canonical references)	70
16 Appendix B — A Complete Numerical Walkthrough with Code	71
16.1 B.1 Generate a synthetic dataset	71
16.2 B.2 Classical correlations	71
16.3 B.3 Mutual information (KSG)	71
16.4 B.4 Distance correlation and HSIC	72
16.5 B.5 Chatterjee's ξ	72
16.6 B.6 Permutation feature importance	72
16.7 B.7 SHAP (TreeSHAP)	72
16.8 B.8 LASSO with cross-validated λ	73
16.9 B.9 Putting it all together	73
17 Appendix C — Annotated Bibliography	74
17.1 Classical and rank correlation	74
17.2 Information theory and entropy-based	74
17.3 Foundations of dependence	74
17.4 Geometric, kernel, and modern independence	75
17.5 Tetrachoric / polychoric	75
17.6 Model-agnostic feature importance	75
17.7 Tree and linear feature importance	76
17.8 Further modern non-parametric measures and tests	76
17.9 Causation, multiplicity, foundations	77

1 Introduction

Asking "*how strongly are two variables related?*" sounds like a single question. It is not. The answer depends on **what you mean by related** (linear? monotonic? functional? statistical?), **what kind of variables you have** (continuous, ordinal, binary, multivariate, time-indexed?), **what you intend to do with the answer** (rank features? test a hypothesis? interpret a black-box model? estimate a causal effect?), and **what assumptions you are willing to make** about the joint distribution.

Over the last 130 years the statistical literature has produced a sprawling menagerie of correlation coefficients, dependence measures, and importance scores. They are not interchangeable. Pearson's r can be zero for a deterministic relationship; mutual information cannot distinguish positive from negative association; SHAP gives you per-prediction credit but tells you nothing about the data-generating process; Gini importance is biased toward high-cardinality features; and a permutation feature importance computed on training data is closer to a measure of memorisation than of generalisation. Each measure encodes a different theory of what "association" *means*, and the wrong choice can flip the qualitative conclusion of an analysis.

A guiding distinction will resurface throughout. **Correlation coefficients** are summary statistics of the joint distribution of *raw variables*. **Feature importance metrics** are summary statistics of *a model fitted to those variables*. A feature can be enormously important to a particular model without being statistically associated with the target after controlling for the rest, and vice versa. Confusing the two is one of the most common mistakes in applied machine learning, and it has cost people money, reputations, and the occasional clinical trial.

The structure of the guide mirrors the historical and mathematical arc of the field:

- **Part I** covers the classical and rank-based coefficients (Pearson, Spearman, Kendall, point-biserial, ϕ , Cramér's V , tetrachoric/polychoric). These are the workhorses of nineteenth- and early-twentieth-century statistics and remain the most-used measures by an enormous margin.
- **Part II** introduces information-theoretic dependence (mutual information, MIC, NMI, transfer entropy, conditional MI). Shannon's framework gave statisticians a *fully general* notion of dependence, at the cost of new estimation difficulties.
- **Part III** covers the modern geometric, kernel, and rank-functional measures (distance correlation, Brownian covariance, KCCA, HSIC, Chatterjee's ξ). These resolve many of MI's estimation pains while preserving the "detects any dependence" property.
- **Part IV** turns to model-agnostic feature importance (SHAP, permutation importance, LIME, PDP/ICE). The questions change: we are no longer summarising the joint distribution of raw variables, but the behaviour of a fitted model.
- **Part V** covers tree-specific and embedded metrics (MDI/Gini, MDA, gain/split/cover in boosting, LASSO/Ridge coefficients). These are cheap, ubiquitous, and frequently misinterpreted.

Each metric section follows a uniform template — formal definition, derivation, intuition, properties, asymptotic theory, estimation algorithm, runnable code snippet, worked numerical example, pitfalls, when-to-use / when-not-to-use, cross-references — so you can navigate by metric without reading top-to-bottom. The cross-cutting themes at the end consolidate ideas (significance testing, multiplicity, confounding) that apply to every measure in the document.

A note on style: this is a *practitioner's* guide. It is mathematically rigorous where rigor is illuminating but stops short of measure-theoretic foundations. References to deeper material are flagged inline and collected in Appendix C.

2 How to Read This Guide

Three reading modes are supported:

1. **Linear.** Read top to bottom. This works because the parts are ordered roughly by mathematical sophistication and by the chronological development of the ideas. You will end up with a coherent map of the entire landscape.
2. **Per-metric.** Use the TOC and the master comparison table to find the metric you need; each metric section is self-contained except for cross-references that are explicitly hyperlinked.
3. **By question.** The decision tree under [Choosing a Measure](#) and the [Common Confusions](#) section route you from a real-world question ("my data are ordinal, how do I correlate them?") to the right metric.

The [Prerequisites Primer](#) is intentionally placed before the technical content. If a term in a later section is unfamiliar, the primer almost certainly defines it; the glossary at the end is a one-line fallback.

3 Notation and Conventions

Symbol	Meaning
X, Y, Z	Random variables (capital letters); X may be a vector $\mathbf{X} \in \mathbb{R}^p$.
x_i, y_i	Realised values of X, Y in a sample of size n , $i = 1, \dots, n$.
$\bar{x}, \bar{y}$	Sample means $\bar{x} = \frac{1}{n} \sum_i x_i$.
s_x, s_y	Sample standard deviations (with Bessel's correction $n - 1$ in the denominator unless noted).
F_X, p_X	Cumulative distribution function (CDF) and probability density / mass function (PDF/PMF) of X .
$H(X)$	Shannon entropy of X (differential entropy if X is continuous).
$I(X; Y)$	Mutual information between X and Y .
$D_{\text{KL}}(p \parallel q)$	Kullback–Leibler divergence of p relative to q ($= \mathbb{E}_p[\log p/q]$).
R_i, Q_i	Ranks of x_i and y_i (smallest = rank 1; ties handled by average rank unless noted).
$\mathbb{I}[\cdot]$	Indicator function.
$\ \cdot\ $	Euclidean norm on $\mathbb{R}^d$ (or RKHS norm $\ \cdot\ _{\mathcal{H}}$ where specified). Single bars $ \cdot $ denote absolute value or set cardinality.
$\mathcal{H}, k$	A reproducing kernel Hilbert space and its kernel function.
$f, \hat{f}$	A model (function) and its empirical fit.
$\mathcal{L}$	A loss function.
$\mathbb{E}, \text{Var}, \text{Cov}$	Expectation, variance, covariance operators.
Φ, φ	Standard normal CDF and characteristic function respectively (context-dependent).
$\perp$	Statistical independence: $X \perp Y$ means $p_{XY} = p_X p_Y$.
$\binom{n}{k}$	Binomial coefficient $n!/(k!(n-k)!)$.
$\rightarrow_p, \rightarrow_d$	Convergence in probability, convergence in distribution.

All logarithms in information-theoretic sections are natural unless we explicitly write $\log_2$ (in which case the unit is bits rather than nats). Code snippets assume Python 3.11+, NumPy, SciPy, and scikit-learn unless stated otherwise.

4 Prerequisites Primer

This section is for readers who want a refresher before encountering the dependence measures. Skim it once or skip and return as needed.

4.1 P1. Random variables, expectations, and moments

A random variable X is a measurable function from a probability space to $\mathbb{R}$ (or $\mathbb{R}^d$). It is characterised by its **distribution**, summarised either by a CDF $F_X(x) = P(X \leq x)$ or, where it exists, a density $p_X(x) = F'_X(x)$.

The **expectation** $\mathbb{E}[X] = \int x dF_X(x)$ is the population mean; the **variance** $\text{Var}(X) = \mathbb{E}[(X - \mathbb{E}[X])^2]$ measures spread. **Covariance** between two variables is

$$\text{Cov}(X, Y) = \mathbb{E}[(X - \mathbb{E}[X])(Y - \mathbb{E}[Y])] = \mathbb{E}[XY] - \mathbb{E}[X]\mathbb{E}[Y].$$

Covariance is unbounded; **correlation** is the scale-free version, $\rho_{XY} = \text{Cov}(X, Y)/(\sigma_X \sigma_Y)$.

Moments of X are $m_k = \mathbb{E}[X^k]$ (raw) or $\mu_k = \mathbb{E}[(X - \mathbb{E}[X])^k]$ (central). The third central moment relates to skewness, the fourth to kurtosis. A distribution is *characterised* by its moments if its moment-generating function $M_X(t) = \mathbb{E}[e^{tX}]$ exists in a neighbourhood of zero — but distributions with all finite moments need not be uniquely determined by them (the log-normal is a classic counterexample). The characteristic function below sidesteps this issue.

4.2 P2. Entropy, cross-entropy, KL divergence

Shannon entropy of a discrete random variable with PMF p on alphabet $\mathcal{X}$:

$$H(X) = - \sum_{x \in \mathcal{X}} p(x) \log p(x).$$

The unit is *nats* with $\log = \ln$, *bits* with $\log_2$. $H(X) \geq 0$, with $H(X) = 0$ iff X is deterministic, and $H(X) \leq \log |\mathcal{X}|$ with equality at uniform distribution. Intuitively, $H(X)$ is the average number of nats needed to optimally encode a draw from X .

For continuous X with density p the **differential entropy** is $h(X) = - \int p(x) \log p(x) dx$. Differential entropy can be *negative* (e.g. a uniform on $[0, 0.1]$) and is not invariant under change of variables — these quirks matter when interpreting MI estimates.

Joint and conditional entropy: $H(X, Y) = - \sum p(x, y) \log p(x, y)$ and $H(Y | X) = H(X, Y) - H(X)$. The **chain rule** is $H(X, Y) = H(X) + H(Y | X)$.

Kullback–Leibler divergence (relative entropy) between two distributions p, q with $p \ll q$ (i.e., p absolutely continuous w.r.t. q ; equivalently $\text{supp}(p) \subseteq \text{supp}(q)$):

$$D_{\text{KL}}(p \parallel q) = \sum_x p(x) \log \frac{p(x)}{q(x)} \quad (\text{discrete}), \quad D_{\text{KL}}(p \parallel q) = \int p(x) \log \frac{p(x)}{q(x)} dx \quad (\text{continuous}).$$

Asymmetric ($D_{\text{KL}}(p \parallel q) \neq D_{\text{KL}}(q \parallel p)$), non-negative, zero iff $p = q$ almost everywhere. KL is *not* a metric (no triangle inequality, no symmetry) but is the canonical "distance from p to q " in information theory.

Mutual information is the KL divergence between the joint and the product of marginals:

$$I(X; Y) = D_{\text{KL}}(p_{XY} \parallel p_X p_Y) = H(X) + H(Y) - H(X, Y).$$

The decomposition $I = H(X) + H(Y) - H(X, Y)$ is exact for discrete variables. For continuous variables each differential-entropy term can be infinite while I itself remains finite and well-defined — the KL form is the more fundamental quantity.

4.3 P3. Characteristic functions

The **characteristic function** of X is $\varphi_X(t) = \mathbb{E}[e^{itX}]$ for $t \in \mathbb{R}$. Three facts to remember:

1. φ_X *always* exists and uniquely determines the distribution.

2. Independence is equivalent to factorisation: $X \perp Y \iff \varphi_{X,Y}(t,s) = \varphi_X(t) \varphi_Y(s)$ for all t, s .
3. Distance correlation (Part III) is built directly on this characterisation: it measures the L^2 distance between φ_{XY} and $\varphi_X \varphi_Y$ in a specific weighted sense. This is the entire reason dCor is an *if and only if* test of independence while Pearson is not.

4.4 P4. Reproducing kernel Hilbert spaces (RKHS)

A **kernel** is a symmetric positive-definite function $k : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$ — that is, for any finite collection $x_1, \dots, x_n \in \mathcal{X}$, the Gram matrix $K_{ij} = k(x_i, x_j)$ is positive semi-definite. Examples: linear $k(x, y) = \langle x, y \rangle$; polynomial $k(x, y) = (\langle x, y \rangle + c)^d$; Gaussian RBF $k(x, y) = \exp(-\|x - y\|^2 / 2\sigma^2)$; Laplace $k(x, y) = \exp(-\|x - y\| / \sigma)$.

To each kernel corresponds a unique Hilbert space $\mathcal{H}$ of functions $\mathcal{X} \rightarrow \mathbb{R}$ — the **RKHS** — with the *reproducing property*: $f(x) = \langle f, k(\cdot, x) \rangle_{\mathcal{H}}$. The map $\phi : x \mapsto k(\cdot, x)$ is the **feature map**; it embeds $\mathcal{X}$ into $\mathcal{H}$, and inner products in $\mathcal{H}$ can be computed via the kernel (the *kernel trick*): $\langle \phi(x), \phi(y) \rangle_{\mathcal{H}} = k(x, y)$.

A kernel is **characteristic** if the mean embedding $\mu_P = \mathbb{E}_{X \sim P}[\phi(X)]$ is injective in P , i.e. $\mu_P = \mu_Q \Rightarrow P = Q$. The Gaussian RBF is characteristic on $\mathbb{R}^d$. This property is what allows HSIC with an RBF kernel to be a true independence test.

4.5 P5. U-statistics and V-statistics

Many estimators in this guide are written as averages over k -tuples of observations rather than over single observations. A **U-statistic** of order k with symmetric kernel $h : \mathcal{X}^k \rightarrow \mathbb{R}$ is

$$U_n = \binom{n}{k}^{-1} \sum_{i_1 < i_2 < \dots < i_k} h(x_{i_1}, \dots, x_{i_k}).$$

V-statistics allow repeated indices: $V_n = n^{-k} \sum_{i_1, \dots, i_k} h(x_{i_1}, \dots, x_{i_k})$. U-statistics are unbiased; V-statistics are biased but often simpler. The bias-corrected distance covariance estimator (Part III, §13) is a U-statistic; the original 2007 estimator is a V-statistic.

Hoeffding's central limit theorem: under mild conditions and *non-degeneracy* ($\zeta_1 > 0$), $\sqrt{n}(U_n - \theta) \rightarrow_d \mathcal{N}(0, k^2 \zeta_1)$ where ζ_1 is the variance of the first conditional projection. When the kernel is *degenerate* ($\zeta_1 = 0$) the limit is instead a weighted sum of χ_1^2 variables at rate n rather than $\sqrt{n}$. This degenerate regime is precisely what arises under the null $H_0 : X \perp Y$ for dCor and HSIC — so the actual engine for those null distributions (Part III) is the *degenerate* U-statistic limit, not the classical Hoeffding CLT above.

4.6 P6. Bootstrap, permutation, jackknife

Three resampling tools:

- **Bootstrap**: sample n observations *with replacement* from the data, recompute the statistic, repeat B times. The empirical distribution of the bootstrap statistics approximates the sampling distribution. Used for **confidence intervals** (percentile, BCa, basic).
- **Permutation**: shuffle one variable's values across rows, recompute the statistic. Under $H_0 : X \perp Y$ the joint sample is *exchangeable*, so all permutations of the labels are equally likely and the empirical distribution of permuted statistics is the exact null distribution. Exchangeability — not just marginal correctness — is the load-bearing assumption; it fails for dependent observations (time series, repeated measures), where block permutation or stationary bootstrap is required instead. Used for **hypothesis tests**.
- **Jackknife**: leave-one-out replicates; recompute the statistic on each $n - 1$ subset. Older and less powerful than the bootstrap; mainly used for **bias estimation**.

For time series, replace plain bootstrap/permutation with **block bootstrap** to preserve autocorrelation.

4.7 P7. Copulas

Sklar's theorem (1959): any multivariate CDF F_{XY} can be written as $F_{XY}(x, y) = C(F_X(x), F_Y(y))$ where C is a **copula** — a CDF on $[0, 1]^2$ with uniform marginals. The copula captures the *dependence structure*, separated from the marginals.

Spearman's ρ and Kendall's τ are functionals of the copula alone, which is why they are invariant under monotonic marginal transformations. Pearson's r depends on both the copula and the marginals. Polychoric correlation assumes a Gaussian copula.

4.8 P8. Granger causality and DAGs

Granger causality (Granger 1969): X Granger-causes Y if past values of X improve the prediction of Y beyond what is achievable from past values of Y alone. In the linear-Gaussian setting this is a comparison of nested VAR models. The information-theoretic generalisation is **transfer entropy** (§11).

Directed acyclic graphs (DAGs) in the Pearl-style causal framework represent causal relationships among variables as a graph in which an edge $X \rightarrow Y$ encodes a direct causal effect. The **d-separation** criterion translates graph structure into conditional independence claims about the data. Algorithms like PC and FCI use conditional-independence tests (often based on partial correlation, KCIT, or CMI) to learn the DAG skeleton from data.

A pointed caution: every metric in this guide measures *statistical* dependence in observed data. None of them measure interventional causal effect on their own. Combining them with the DAG framework and assumptions like *causal faithfulness* and *no unobserved confounders* yields causal claims; without those assumptions, you have only association.

4.9 P9. Shapley values from cooperative game theory

A **cooperative game** is a pair (N, v) where $N = \{1, \dots, p\}$ is the player set and $v : 2^N \rightarrow \mathbb{R}$ is the value function with $v(\emptyset) = 0$. The **Shapley value** of player i is

$$\phi_i = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|! (p - |S| - 1)!}{p!} [v(S \cup \{i\}) - v(S)].$$

This is the *unique* payoff vector satisfying four axioms:

1. **Efficiency:** $\sum_i \phi_i = v(N)$.
2. **Symmetry:** if $v(S \cup \{i\}) = v(S \cup \{j\})$ for all S not containing i, j , then $\phi_i = \phi_j$.
3. **Dummy / null player:** if $v(S \cup \{i\}) = v(S)$ for all S , then $\phi_i = 0$.
4. **Additivity:** for two games v_1, v_2 on the same players, $\phi_i(v_1 + v_2) = \phi_i(v_1) + \phi_i(v_2)$.

In SHAP (§18) the "players" are features and $v(S)$ is the model's expected output when only features in S are known.

5 The Five Properties That Distinguish Dependence Measures

A short checklist that distinguishes the dependence measures at a glance. For a candidate measure $D(X, Y)$:

1. **Range.** Bounded ($[-1, 1]$ or $[0, 1]$) or unbounded ($[0, \infty)$)? Bounded measures are easier to compare across datasets; unbounded ones often have a more natural information-theoretic interpretation.
2. **Sign.** Does D distinguish positive from negative association, or only the strength? Pearson, Spearman, Kendall do; MI, dCor, HSIC, MIC do not.
3. **Independence characterisation.** Is $D(X, Y) = 0 \iff X \perp Y$? Pearson is famously *not* (it only equals zero under linear non-association); dCor, HSIC with a characteristic kernel, MI, and Chatterjee's ξ *are*.
4. **Invariance.** Under what transformations is D unchanged? Pearson under affine transforms; Spearman/Kendall/ ξ under strictly monotonic transforms of X and Y separately; MI under any componentwise *measurable bijection* of X and Y (diffeomorphisms in the continuous case); dCor under separate translation, orthogonal rotation, and global scaling per component (Székely–Rizzo 2007); HSIC's invariance is entirely *kernel-driven* — translation-invariant kernels give translation invariance, rotation-invariant kernels (e.g. Gaussian RBF) give orthogonal invariance, and so on.
5. **Symmetry.** Is $D(X, Y) = D(Y, X)$? All classical measures are symmetric. Chatterjee's ξ is *not* — $\xi(X, Y) \neq \xi(Y, X)$ in general — which is what makes it usable as a *functional dependence* detector.

Add three further axes the original five often elide: **computational cost** (linear, quadratic, cubic in n), **estimator bias under small samples**, and **convergence rate** (parametric $O(n^{-1/2})$ versus nonparametric $O(n^{-\alpha})$ for some $\alpha < 1/2$). These eight properties explain most of the differences you will see in practice.

6 Master Comparison Table

The single most-requested artefact when people compare these measures. Cells use shorthand: $\checkmark$ = yes, $\times$ = no, $\sim$ = with caveats. "Indep" = "characterises independence", i.e. $D = 0 \iff X \perp Y$. "Time" is asymptotic time complexity in the sample size n ; "Memory" is dominant memory footprint. "Sign" = produces signed values.

#	Measure	Variable types	Range	Sign	Indep	Invariance
1	Pearson r	cont $\times$ cont	$[-1, 1]$	$\checkmark$	$\times$ (linear only)	Affine
2	Spearman ρ	ord/cont $\times$ ord/cont	$[-1, 1]$	$\checkmark$	$\times$ (monotone only)	Monotone
3	Kendall τ	ord/cont $\times$ ord/cont	$[-1, 1]$	$\checkmark$	$\times$ (monotone only)	Monotone
4	Point-biserial r_{pb}	cont $\times$ binary	$[-1, 1]$	$\checkmark$	$\sim$ ($\equiv$ Pearson)	Affine in Y
5	ϕ	binary $\times$ binary	$[-1, 1]$	$\checkmark$	$\checkmark$ on 2×2	Coding
6	Cramér's V	cat $\times$ cat	$[0, 1]$	$\times$	$\checkmark$ on tables	Coding
7	Polychoric	ord $\times$ ord	$[-1, 1]$	$\checkmark$	$\times$ (latent normal)	Threshold
8	Mutual Information	any $\times$ any	$[0, \infty)$	$\times$	$\checkmark$	Invertible
9	MIC	cont $\times$ cont	$[0, 1]$	$\times$	$\times$ (consistency debated)	Monotone
10	NMI	any $\times$ any (typ. cat $\times$ cat)	$[0, 1]$	$\times$	$\checkmark$	Bijjective
11	Transfer Entropy	time-series	$[0, \infty)$	directed	$\times$ (cond. on Y 's past)	Bijjective
12	CMI	any $\times$ any $\times$ any	$[0, \infty)$	$\times$	$\checkmark$ (conditional)	Invertible
13	dCor	vec $\times$ vec	$[0, 1]$	$\times$	$\checkmark$	Orthogonal
14	Brownian Cov	vec $\times$ vec	$[0, \infty)$	$\times$	$\checkmark$	Orthogonal
15	KCCA	vec $\times$ vec	$[0, 1]$	$\times$	$\sim$ (kernel + reg.)	Bijjective
16	HSIC	vec $\times$ vec	$[0, \infty)$	$\times$	$\checkmark$ (characteristic k)	Orthogonal
17	Chatterjee's ξ	cont $\times$ cont (asym)	$[-1/2, 1]$	$\sim$	$\checkmark$ + functional	Monotone
18	SHAP	model output	per-feature $\mathbb{R}$	$\checkmark$	n/a	model-speci
19	PFI	model loss	$\mathbb{R}$	$\sim$	n/a	model-speci
20	LIME	local linear	per-feature $\mathbb{R}$	$\checkmark$	n/a	model-speci
21	PDP / ICE	marginal effect	$\mathbb{R}$ curve	n/a	n/a	model-speci
22	MDI / Gini	tree-based	$[0, \infty)$	$\times$	n/a	tree-specific
23	MDA	tree-based	$\mathbb{R}$	$\times$	n/a	tree-specific
24	Gain / Split / Cover	boosting	$[0, \infty)$	$\times$	n/a	tree-specific
25	LASSO / Ridge β	linear model	$\mathbb{R}$	$\checkmark$	n/a	scaling

Read this table sideways: pick the column you care about and skim. When you need a measure for non-linear vector–vector dependence with a closed-form p -value, the table tells you HSIC with a gamma approximation. When you need a categorical–categorical effect size, it tells you Cramér's V . The detailed sections explain *why* each row is what it is.

Caveats on the Range column. Several entries describe the *population* or *V-statistic* range. The bias-corrected U-statistic dCor and the unbiased U-statistic HSIC can be *negative* in finite samples (their expectations are zero under independence, so roughly half of null draws are negative). Chatterjee's ξ is in $[0, 1]$ at the population level; the finite-sample range is asymptotically $[-1/2 + o(1), 1]$. For Transfer Entropy and CMI the listed $O(n)$ / $O(n \log n)$ time complexities assume KSG-style neighbour search with a kd-tree; brute-force pairwise distance computation costs $O(n^2)$. PFI's "Sign $\sim$ " reflects that the score-change is itself signed but does not encode direction of effect on individual predictions.

7 Part I — Classical and Rank Correlation Coefficients

This first family is the oldest, the most widely understood, and the one most often abused. All seven measures in this part summarise the joint distribution by a single number in $[-1, 1]$ or $[0, 1]$ and rely on strong implicit assumptions about *what kind* of association is interesting. The historical arc runs from Pearson's 1895 product-moment coefficient through Spearman (1904), Kendall (1938), and Cramér (1946) — a half-century in which descriptive statistics matured into a mathematical discipline.

7.1 1. Pearson's Product–Moment Correlation r

Anscombe's Quartet: same r , very different data

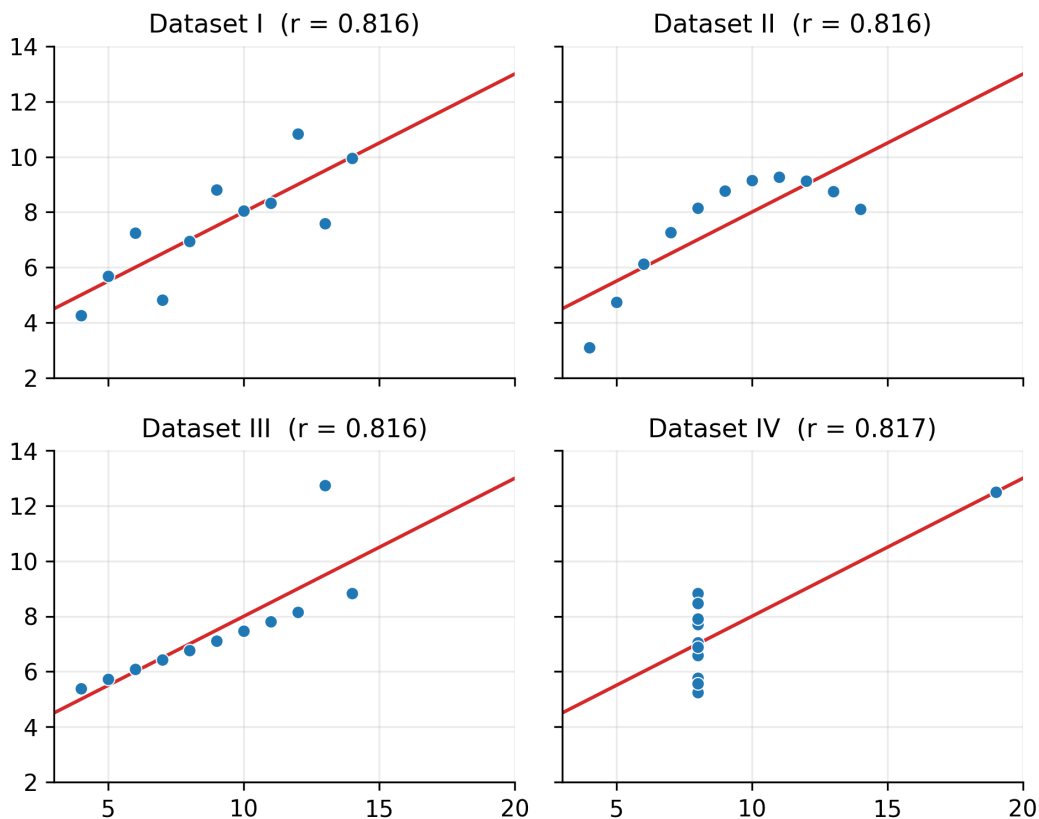


Figure 1: Anscombe's quartet: four datasets that share the same Pearson correlation ($r \approx 0.816$), means, variances, and regression line, but have radically different shapes. A single summary statistic is never a substitute for looking at the data.

7.1.1 1.1 Definition

For random variables X, Y with finite second moments,

$$\rho_{XY} = \frac{\text{Cov}(X, Y)}{\sqrt{\text{Var}(X) \text{Var}(Y)}} = \frac{\mathbb{E}[(X - \mu_X)(Y - \mu_Y)]}{\sigma_X \sigma_Y}.$$

The sample analogue:

$$r = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2} \sqrt{\sum_{i=1}^n (y_i - \bar{y})^2}}.$$

7.1.2 1.2 Derivation and geometric interpretation

Centre the data: $\tilde{x}_i = x_i - \bar{x}$, $\tilde{y}_i = y_i - \bar{y}$. Treat $\tilde{\mathbf{x}}, \tilde{\mathbf{y}}$ as vectors in $\mathbb{R}^n$. Then

$$r = \frac{\langle \tilde{\mathbf{x}}, \tilde{\mathbf{y}} \rangle}{\|\tilde{\mathbf{x}}\| \|\tilde{\mathbf{y}}\|} = \cos \theta,$$

where θ is the angle between the two centred vectors. From this:

- $r = +1$ when $\tilde{\mathbf{y}} = c\tilde{\mathbf{x}}$ for some $c > 0$ (perfect positive linear relation).
- $r = -1$ when $\tilde{\mathbf{y}} = c\tilde{\mathbf{x}}$ for some $c < 0$ (perfect negative).
- $r = 0$ when the centred vectors are orthogonal.

The Cauchy–Schwarz inequality immediately gives $|r| \leq 1$. Equivalently, r is the slope of the **standardised** OLS regression of y on x : if $z_x = (x - \bar{x})/s_x$ and similarly z_y , the OLS estimate of β in $z_y = \beta z_x + \varepsilon$ is exactly r .

7.1.3 1.3 Inference

Under bivariate normality, the studentised statistic

$$t = r \sqrt{\frac{n-2}{1-r^2}}$$

is t -distributed with $n-2$ degrees of freedom under $H_0 : \rho = 0$. **Fisher's z -transformation:**

$$z = \frac{1}{2} \ln \left(\frac{1+r}{1-r} \right) = \operatorname{arctanh}(r)$$

yields an asymptotically normal statistic with mean $\operatorname{arctanh}(\rho)$ and variance $\frac{1}{n-3}$, used to construct CIs and to compare two correlations from independent samples. The variance-stabilising property follows from a Taylor expansion: $\operatorname{Var}(r) \approx (1-\rho^2)^2/n$, and the derivative $z'(r) = 1/(1-r^2)$ exactly cancels the factor, giving $\operatorname{Var}(z) \approx 1/n$; Fisher's $1/(n-3)$ is the small-sample correction (Fisher 1921).

7.1.4 1.4 Properties

- **Symmetry:** $r(X, Y) = r(Y, X)$.
- **Affine invariance:** $r(aX + b, cY + d) = \operatorname{sign}(ac) \cdot r(X, Y)$ for $a, c \neq 0$.
- **Not monotonically invariant:** $r(X, Y) \neq r(\log X, Y)$ in general.
- **Bound on quadrant probability:** $r = 0$ does *not* imply independence.

7.1.5 1.5 The canonical counterexample

Let $X \sim \mathcal{U}(-1, 1)$ and $Y = X^2$. Then $\mathbb{E}[X] = 0$, $\mathbb{E}[XY] = \mathbb{E}[X^3] = 0$, so $\operatorname{Cov}(X, Y) = 0$ and $r = 0$ — yet Y is *deterministic* in X . Pearson's r is blind to non-linear (especially symmetric) dependence. This single example motivates the entirety of Parts II and III.

7.1.6 1.6 Pitfalls

- **Outliers dominate:** squared deviations enter both numerator and denominator; a single high-leverage point can swing r from near 0 to near 1 or vice versa.
- **Non-linearity is invisible:** always plot the data.
- **Heteroscedasticity** does not bias r as a *point* estimate but invalidates the t -test.
- **Range restriction** attenuates r : subsetting on X mechanically pulls r toward zero (the "restricted range" problem familiar in psychometrics).
- **Anscombe's quartet and the Datasaurus dozen** construct datasets with identical r but radically different scatter plots — a permanent reminder that one number is never sufficient.

7.1.7 1.7 Code

```
import numpy as np
from scipy import stats

rng = np.random.default_rng(0)
n = 500
x = rng.normal(size=n)
y = 0.7 * x + 0.3 * rng.normal(size=n)

r, p = stats.pearsonr(x, y)
print(f"r = {r:.3f}, p = {p:.3g}")

# Fisher z confidence interval
z = np.arctanh(r)
se = 1.0 / np.sqrt(n - 3)
lo, hi = np.tanh(z - 1.96 * se), np.tanh(z + 1.96 * se)
print(f"95% CI: [{lo:.3f}, {hi:.3f}]" )
```

7.1.8 1.8 Worked tiny example

Five points: (1, 2), (2, 4), (3, 5), (4, 4), (5, 6).

- $\bar{x} = 3, \bar{y} = 4.2$.
- $\sum (x_i - \bar{x})(y_i - \bar{y}) = (-2)(-2.2) + (-1)(-0.2) + 0 \cdot 0.8 + 1 \cdot (-0.2) + 2 \cdot 1.8 = 4.4 + 0.2 + 0 - 0.2 + 3.6 = 8$.
- $\sum (x_i - \bar{x})^2 = 4 + 1 + 0 + 1 + 4 = 10$.
- $\sum (y_i - \bar{y})^2 = 4.84 + 0.04 + 0.64 + 0.04 + 3.24 = 8.8$.
- $r = 8 / \sqrt{10 \cdot 8.8} = 8 / \sqrt{88} \approx 0.853$.

7.1.9 1.9 When to use / when not

Use when data are continuous, the relationship is approximately linear, outliers are absent, and you want a quick interpretable scalar. **Avoid** for ordinal, ranked, heavy-tailed, or visibly non-linear data; when you care about *any* dependence rather than specifically linear dependence.

7.1.10 1.10 Cross-references

For monotonic but non-linear relationships use [Spearman](#) or [Kendall](#). For any-shape dependence use [dCor](#), [HSIC](#), or [MI](#).

7.2 2. Spearman's Rank Correlation ρ

7.2.1 2.1 Definition

Replace observations by their ranks and apply Pearson. Let $R_i = \text{rank}(x_i)$ and $Q_i = \text{rank}(y_i)$, both in $\{1, \dots, n\}$ (ties broken by averaging). Then

$$\rho_S = \text{Pearson}(R, Q).$$

With *no ties*, the convenient simplification (derivable from the fact that $\sum R_i = n(n+1)/2$ and $\sum R_i^2 = n(n+1)(2n+1)/6$) is

$$\rho_S = 1 - \frac{6 \sum_{i=1}^n d_i^2}{n(n^2 - 1)}, \quad d_i = R_i - Q_i.$$

7.2.2 2.2 Derivation of the shortcut formula

When ranks are a permutation of $\{1, \dots, n\}$, the mean rank is $\bar{R} = (n+1)/2$ and $\sum_i (R_i - \bar{R})^2 = n(n^2-1)/12$. Substituting into Pearson's formula:

$$\rho_S = \frac{\sum_i (R_i - \bar{R})(Q_i - \bar{R})}{n(n^2-1)/12}.$$

Expanding $\sum_i (R_i - Q_i)^2 = \sum_i (R_i - \bar{R})^2 + \sum_i (Q_i - \bar{R})^2 - 2 \sum_i (R_i - \bar{R})(Q_i - \bar{R})$ and solving:

$$\sum_i (R_i - \bar{R})(Q_i - \bar{R}) = \frac{n(n^2-1)}{12} - \frac{1}{2} \sum_i d_i^2.$$

Divide by $n(n^2-1)/12$ and simplify to obtain the famous form $1 - 6 \sum d_i^2 / (n(n^2-1))$.

7.2.3 2.3 Intuition

ρ_S measures the strength of the **monotonic** relationship: any strictly increasing transformation $X \mapsto g(X)$ leaves ρ_S unchanged because g preserves ranks. It is therefore *scale-free* and *outlier-robust* — replacing the largest observation by an arbitrarily larger number changes only its rank, not the calculation.

7.2.4 2.4 Relationship to Pearson

For jointly normal (X, Y) with population correlation ρ ,

$$\rho_S = \frac{6}{\pi} \arcsin\left(\frac{\rho}{2}\right) \approx 0.95 \rho \text{ for moderate } |\rho|.$$

For monotonic but non-linear relations (e.g. $Y = \tanh(X) + \text{noise}$), Spearman exceeds Pearson — sometimes substantially. This gap is a useful diagnostic: a large $\rho_S - r$ signals non-linear monotonicity.

7.2.5 2.5 Inference

For $n \gtrsim 10$,

$$t_S = \rho_S \sqrt{\frac{n-2}{1-\rho_S^2}} \stackrel{H_0}{\approx} t_{n-2}.$$

This form is borrowed from Pearson and is only asymptotic for Spearman; under H_0 , $\text{Var}(\rho_S) = 1/(n-1)$ so the simpler exact null statistic is $\sqrt{n-1} \rho_S \rightarrow \mathcal{N}(0, 1)$. SciPy's `spearmanr` uses the t -approximation; the difference matters only for small n . Exact tables exist for small n (Olds 1938); permutation tests are exact and the default for heavy ties.

7.2.6 2.6 Tie handling

With ties, the simple shortcut formula is biased. SciPy's `spearmanr` applies the tie-corrected formula automatically. When more than ~25% of values are tied, consider Kendall's τ_b instead — it handles ties more gracefully.

7.2.7 2.7 Code

```
from scipy import stats
import numpy as np

rng = np.random.default_rng(0)
x = rng.normal(size=200)
y = np.tanh(2 * x) + 0.1 * rng.normal(size=200)

r_pearson, _ = stats.pearsonr(x, y)
```

```
rho_spearman, _ = stats.spearmanr(x, y)
print(f"Pearson r = {r_pearson:.3f}")
print(f"Spearman rho = {rho_spearman:.3f}")
# Spearman is materially larger because the relationship is monotonic-NL
```

7.2.8 2.8 Worked tiny example

Same five points (1, 2), (2, 4), (3, 5), (4, 4), (5, 6).

Ranks of x : 1, 2, 3, 4, 5. Ranks of y : 1, 2.5, 4, 2.5, 5 (the two 4's tied at midrank 2.5). Without tie correction: $d = (0, -0.5, -1, 1.5, 0)$, $\sum d^2 = 0 + 0.25 + 1 + 2.25 + 0 = 3.5$. $\rho_S \approx 1 - 6(3.5)/(5 \cdot 24) = 1 - 21/120 = 0.825$. The tie-corrected value (which SciPy reports) is slightly different because of the midrank at $y = 4$; for tied data, prefer `spearmanr` over the shortcut formula.

7.2.9 2.9 Pitfalls

- **Ties** require correction; use `spearmanr` not the shortcut.
- **Discrete or coarse variables** lose resolution under ranking.
- **Monotonicity is not all you want**: U-shaped or sinusoidal relationships have $\rho_S \approx 0$.

7.2.10 2.10 When to use / when not

Use for ordinal data; continuous data with outliers or visibly non-linear-but-monotonic shape; any robust-correlation slot in EDA. **Avoid** when the relationship is non-monotonic — switch to `dCor` or [Chatterjee's \$\xi\$](#) .

7.3 3. Kendall's τ

7.3.1 3.1 Definition

A pair of observations $(x_i, y_i), (x_j, y_j)$ is **concordant** if $(x_i - x_j)(y_i - y_j) > 0$, **discordant** if negative, **tied** otherwise. Out of $\binom{n}{2}$ pairs, let C be the number of concordant and D the number of discordant:

$$\tau_a = \frac{C - D}{\binom{n}{2}}.$$

The tie-corrected τ_b :

$$\tau_b = \frac{C - D}{\sqrt{(C + D + T_x)(C + D + T_y)}},$$

where T_x, T_y count pairs tied on X or Y but not both. τ_c is a further variant for non-square contingency tables.

7.3.2 3.2 Probabilistic interpretation

Define $\tau = P\{(X_1 - X_2)(Y_1 - Y_2) > 0\} - P\{(X_1 - X_2)(Y_1 - Y_2) < 0\}$ for independent copies $(X_1, Y_1), (X_2, Y_2)$. The sample τ_a is an unbiased U-statistic for this functional with kernel $h((x_i, y_i), (x_j, y_j)) = \text{sgn}(x_i - x_j)\text{sgn}(y_i - y_j)$.

Equivalently, $\tau \in [-1, 1]$ where the probability that a randomly chosen pair is concordant equals $(1 + \tau)/2$. This *probabilistic* interpretation is the cleanest in the entire correlation literature.

7.3.3 3.3 Computation

Naively $O(n^2)$, but Knight's (1966) merge-sort-based algorithm achieves $O(n \log n)$ and is the implementation in `scipy.stats.kendalltau`. Idea: sort by x ; then D = number of inversions in the y -sequence, which merge sort counts as a by-product.

7.3.4 3.4 Relationship to Spearman and Pearson

Under bivariate normality:

$$\tau = \frac{2}{\pi} \arcsin(\rho), \quad \rho_S = \frac{6}{\pi} \arcsin\left(\frac{\rho}{2}\right).$$

Empirically $\rho_S/\tau \approx 1.5$ across most of the range, and τ is roughly $\frac{2}{3}\rho_S$ in the *small*- ρ regime (the ratio $\rho_S/\tau \rightarrow 3/2$ as $\rho \rightarrow 0$); for moderate to large ρ the ratio drops toward ~ 1.25 (e.g. $\rho = 0.9$ gives $\tau \approx 0.71$, $\rho_S \approx 0.89$).

7.3.5 3.5 Inference

Under H_0 of independence, $\mathbb{E}[\tau] = 0$ and $\text{Var}(\tau) = 2(2n+5)/(9n(n-1))$; standardising, $\sqrt{(9n(n-1))/(2(2n+5))}$. $\tau \rightarrow_d \mathcal{N}(0, 1)$. Exact null distributions are tabulated for $n \lesssim 50$ and used by `scipy.stats.kendalltau(method="exact")` when n is small.

7.3.6 3.6 Code

```
from scipy import stats
import numpy as np

rng = np.random.default_rng(0)
x = rng.normal(size=100)
y = x + rng.normal(size=100) * 0.5

tau, p = stats.kendalltau(x, y)
print(f"Kendall tau_b = {tau:.3f}, p = {p:.3g}")
```

7.3.7 3.7 Worked tiny example

Points (1, 2), (2, 4), (3, 5), (4, 4), (5, 6). The $\binom{5}{2} = 10$ pairs and their concordance:

pair	Δx	Δy	concordant?
(1,2)-(2,4)	+	+	C
(1,2)-(3,5)	+	+	C
(1,2)-(4,4)	+	+	C
(1,2)-(5,6)	+	+	C
(2,4)-(3,5)	+	+	C
(2,4)-(4,4)	+	0	tied on y
(2,4)-(5,6)	+	+	C
(3,5)-(4,4)	+	-	D
(3,5)-(5,6)	+	+	C
(4,4)-(5,6)	+	+	C

$$C = 8, D = 1, \text{ one } y\text{-tie. } \tau_b = (8 - 1)/\sqrt{(8 + 1 + 0)(8 + 1 + 1)} = 7/\sqrt{90} \approx 0.738.$$

7.3.8 3.8 Pitfalls and trade-offs

- **More robust to outliers than Spearman:** Spearman still uses squared rank differences; Kendall counts signs.
- **Lower power than Spearman for smooth alternatives** in some regimes; more reliable on weak or noisy signals.
- **Tie correction is nontrivial** — `scipy.stats.kendalltau` returns τ_b by default; report which variant.

7.3.9 3.9 When to use / when not

Use for ordinal data with ties, small samples (exact distributions available), and any time you want a probabilistic interpretation ("randomly chosen pair concordant with probability $(1 + \tau)/2$ "). **Avoid** for very large n where the $O(n \log n)$ cost beats the $O(n)$ Pearson by a constant factor that no longer matters.

7.4 4. Point-Biserial Correlation r_{pb}

7.4.1 4.1 Definition and equivalence to Pearson

A degenerate case of Pearson when one variable is dichotomous (coded 0/1). Let Y be continuous and $X \in \{0, 1\}$ with group sizes n_1, n_0 , group means $\bar{y}_1, \bar{y}_0$, and pooled SD s_y :

$$r_{pb} = \frac{\bar{y}_1 - \bar{y}_0}{s_y} \sqrt{\frac{n_1 n_0}{n^2}}.$$

This is **identical** to applying Pearson's formula directly to the 0/1 coded variable.

7.4.2 4.2 Derivation

With $X \in \{0, 1\}$ coded, $\bar{x} = n_1/n$, $\sum_i (x_i - \bar{x})^2 = n_1(n_0/n)^2 + n_0(n_1/n)^2 = n_1 n_0 / n$. Using the population-SD convention (denominator n , as in `scipy.stats.pointbiserialr`), $s_x = \sqrt{n_1 n_0} / n$ and similarly for s_y . The numerator $\sum_i (x_i - \bar{x})(y_i - \bar{y}) = (n_1 n_0 / n)(\bar{y}_1 - \bar{y}_0)$. Substituting into $r = \text{Cov}(X, Y) / (s_x s_y)$ gives the formula above. (Switching to the sample-SD convention with denominator $n - 1$ replaces $\sqrt{n_1 n_0 / n^2}$ by $\sqrt{n_1 n_0 / (n(n - 1))}$; the identity to `scipy` is preserved by using the same convention for s_y .)

7.4.3 4.3 Relation to the two-sample t -test

r_{pb} is monotonically equivalent to the independent-samples t :

$$t = r_{pb} \sqrt{\frac{n - 2}{1 - r_{pb}^2}}, \quad \text{df} = n - 2.$$

Thus the p -value of r_{pb} from `pointbiserialr` is identical (up to floating-point precision) to that of `ttest_ind` on the two groups.

7.4.4 4.4 Point-biserial versus biserial

- **Point-biserial** assumes X is *intrinsically* binary (sex, treatment/control).
- **Biserial** (r_b) assumes X is a coarse-grained version of a latent continuous normal — for example a pass/fail derived from an underlying score. It rescales:

$$r_b = r_{pb} \cdot \frac{\sqrt{pq}}{\varphi(z_p)},$$

where $p = n_1/n$, $q = 1 - p$, $z_p = \Phi^{-1}(p)$, and φ is the standard normal PDF. r_b is always $\geq |r_{pb}|$ in magnitude.

Use r_{pb} when the binary variable is genuinely binary; use r_b only if you have substantive reason to believe a latent continuous variable was thresholded.

7.4.5 4.5 The role of group balance, and a common false bound

As a Pearson correlation, $|r_{pb}| \leq 1$ universally (Cauchy-Schwarz). It is sometimes claimed that $|r_{pb}| \leq \sqrt{n_1 n_0 / n^2}$; this is *wrong*. The factor $\sqrt{n_1 n_0 / n^2}$ appearing in the formula is bounded by 1/2 and tends to zero as groups become imbalanced, but it bounds only the multiplier, not r_{pb} itself: the ANOVA identity $s_y^2 \geq (n_1 n_0 / n^2)(\bar{y}_1 - \bar{y}_0)^2$ precisely cancels the factor and yields $r_{pb}^2 \leq 1$. Zero within-group variance gives $r_{pb} = \pm 1$ at any split.

What *does* happen with extreme imbalance is a loss of **statistical power**: tiny groups make the standard error of $\bar{y}_1 - \bar{y}_0$ explode, so the same true effect produces much smaller t -statistics and wider confidence intervals on r_{pb} . For this reason it is standard to also report a sample-size-aware effect size such as Cohen's d alongside r_{pb} .

7.4.6 4.6 Code

```
from scipy import stats
import numpy as np

rng = np.random.default_rng(0)
group = rng.integers(0, 2, size=200)
score = group * 0.8 + rng.normal(size=200)

r_pb, p = stats.pointbiserialr(group, score)
print(f"r_pb = {r_pb:.3f}, p = {p:.3g}")
```

7.4.7 4.7 When to use / when not

Use when correlating a continuous variable with a true binary one (sex, churn, conversion) and you want a correlation-style effect size alongside a t -test. **Avoid** when the binary variable is a coarse-grained continuous one (use biserial), or when groups are extremely imbalanced (interpret the cap on magnitude).

7.5 5. The Phi Coefficient ϕ

7.5.1 5.1 Definition

For a 2×2 contingency table

	Y = 0	Y = 1
X = 0	a	b
X = 1	c	d

$$\phi = \frac{ad - bc}{\sqrt{(a+b)(c+d)(a+c)(b+d)}}.$$

This is exactly Pearson's r applied to the 0/1 codings.

7.5.2 5.2 Relation to χ^2

The Pearson chi-squared statistic for the 2×2 table is

$$\chi^2 = \frac{n(ad - bc)^2}{(a+b)(c+d)(a+c)(b+d)},$$

hence $\phi^2 = \chi^2 / n$. This links the *test* (chi-squared) and the *effect size* (phi) into one calculation.

7.5.3 5.3 Bounds and marginal-dependence pathology

$\phi \in [-1, 1]$ in principle, but the maximum attainable depends on the marginals. If $a + b \neq a + c$ (i.e., row and column marginals are unequal), $|\phi|_{\max} < 1$. The **maximum** is

$$\phi_{\max} = \sqrt{\frac{p_{1+}q_{+1}}{q_{1+}p_{+1}}} \quad (\text{assuming } p_{1+} \leq p_{+1}; \text{ swap rows otherwise})$$

where p_{i+}, p_{+j} are marginal probabilities — useful to know when comparing ϕ across tables with different marginals.

7.5.4 5.4 Versus the odds ratio

For 2×2 tables many epidemiologists prefer the odds ratio $OR = (ad)/(bc)$, which is invariant to whether the data come from prospective, retrospective, or cross-sectional sampling and is the natural parameter in logistic regression. ϕ is symmetric and bounded but lacks OR's sampling invariance.

A useful rule: report **both** for 2×2 analyses — ϕ as effect size, OR for design-invariant inference.

7.5.5 5.5 Code

```
import numpy as np
from scipy.stats import chi2_contingency

# Confusion-style table
table = np.array([[40, 10],
                  [ 5, 45]])
chi2, p, dof, expected = chi2_contingency(table, correction=False)
n = table.sum()
phi = np.sqrt(chi2 / n) * np.sign(table[0, 0] * table[1, 1] - table[0, 1] * table[1, 0])
print(f"phi = {phi:.3f}, chi2 = {chi2:.2f}, p = {p:.3g}")
```

7.5.6 5.6 When to use

A/B test responses, two-class agreement, any binary-binary association. For sparse cells (any expected count < 5), prefer Fisher's exact test for the p -value.

7.6 6. Cramér's V

7.6.1 6.1 Definition

For an $r \times c$ contingency table with chi-squared statistic χ^2 and total n ,

$$V = \sqrt{\frac{\chi^2/n}{\min(r-1, c-1)}}.$$

7.6.2 6.2 Derivation of the normalisation

For any $r \times c$ table, the maximum value of χ^2 is $n \cdot \min(r-1, c-1)$, attained when each row is concentrated in a single column (or vice versa). Dividing χ^2/n by this maximum normalises V to $[0, 1]$.

7.6.3 6.3 Properties

- Always non-negative; no sign because categorical variables lack a natural ordering.
- Reduces to $|\phi|$ for 2×2 tables.
- Invariant under relabelling of categories (the partition into cells is what matters).

7.6.4 6.4 Bias correction (Bergsma 2013)

For small n or sparse tables, V is upward biased. The bias-corrected:

$$\tilde{V} = \sqrt{\max\left(0, \frac{\chi^2/n - (r-1)(c-1)/(n-1)}{\min(\tilde{r}-1, \tilde{c}-1)}\right)}, \quad \tilde{k} = k - \frac{(k-1)^2}{n-1} \text{ for } k \in \{r, c\}.$$

Strongly recommended when n is small or any cell count is below ~ 10 .

7.6.5 6.5 Code

```
import numpy as np
from scipy.stats import chi2_contingency

def cramers_v(table, bias_correction=True):
    table = np.asarray(table)
    chi2, _, _, _ = chi2_contingency(table, correction=False)
    n = table.sum()
    r, c = table.shape
    phi2 = chi2 / n
    if bias_correction:
        phi2 = max(0, phi2 - (r - 1) * (c - 1) / (n - 1))
        r_tilde = r - (r - 1) ** 2 / (n - 1)
        c_tilde = c - (c - 1) ** 2 / (n - 1)
        return np.sqrt(phi2 / min(r_tilde - 1, c_tilde - 1))
    return np.sqrt(phi2 / min(r - 1, c - 1))

table = np.array([[50, 20, 10],
                  [10, 40, 50]])
print(f"V = {cramers_v(table, bias_correction=False):.3f}")
print(f"V (bias-corrected) = {cramers_v(table, bias_correction=True):.3f}")
```

7.6.6 6.6 Pitfalls

- **High-cardinality categoricals** inflate χ^2 and therefore V — bin or regularise (e.g. group low-frequency levels into "Other").
- **Sparse cells** invalidate the χ^2 approximation for the *test* (use Monte Carlo or exact); the point estimate V remains meaningful.
- V does not tell you *where* the association lives; pair with a mosaic plot or standardised residuals.

7.6.7 6.7 When to use

Categorical–categorical EDA, feature screening with mixed types, mosaic plot annotations. The natural Pearson analogue for fully categorical data.

7.7 7. Tetrachoric and Polychoric Correlation

7.7.1 7.1 The latent-normal model

Assume the *observed* categorical variables $X \in \{1, \dots, r\}$, $Y \in \{1, \dots, c\}$ are coarse-grained slices of underlying jointly normal continuous $(X^*, Y^*) \sim \mathcal{N}_2(\mathbf{0}, \Sigma)$, $\Sigma = \begin{pmatrix} 1 & \rho \\ \rho & 1 \end{pmatrix}$, with thresholds $\tau_1^X < \dots < \tau_{r-1}^X$ and $\tau_1^Y < \dots < \tau_{c-1}^Y$ such that $X = k \iff \tau_{k-1}^X < X^* \leq \tau_k^X$.

- **Tetrachoric** = polychoric with $r = c = 2$.
- **Polychoric** = the general ordinal case.

The parameter of interest is ρ , the correlation between the latent continuous variables.

7.7.2 7.2 Why bother

Pearson's r on the ordinal codes is *attenuated*: categorising continuous variables throws away information and biases the correlation toward zero. Dichotomising two truly normal variables with $\rho = 0.7$ at the medians yields $\phi \approx 0.49$. Tetrachoric/polychoric reverse-engineer the latent correlation, recovering ρ close to the true value *if* the normality assumption holds.

7.7.3 7.3 Estimation

Two methods in standard use:

- **Two-step**: estimate thresholds from marginal frequencies via inverse normal CDF; then maximise the bivariate-normal likelihood over ρ holding thresholds fixed.
- **Full MLE**: jointly estimate thresholds and ρ by maximising

$$\ell(\rho, \tau) = \sum_{k,l} n_{kl} \log P(X = k, Y = l \mid \rho, \tau).$$

The cell probabilities are

$$P(X = k, Y = l) = \int_{\tau_{k-1}^X}^{\tau_k^X} \int_{\tau_{l-1}^Y}^{\tau_l^Y} \phi_2(u, v; \rho) du dv,$$

requiring numerical integration of the bivariate normal CDF.

7.7.4 7.4 Standard errors

Hessian-based (Wald) SEs are standard but unreliable for sparse tables; bootstrap CIs are recommended when n is small or cells are sparse.

7.7.5 7.5 Assumptions and pitfalls

- **Normality of the latent variables** is load-bearing. Violations produce material bias. Sensitivity analyses (Foldnes & Grønneberg 2021) or copula-based generalisations exist but are rarely used in practice.
- **Empty or near-empty cells** make the MLE unstable; pool categories.
- **Large polychoric correlation matrices** must be positive-semi-definite; pairwise polychorics may not be — apply nearest-PSD smoothing before factor analysis.

7.7.6 7.6 Code

```
# Requires factor_analyzer
from factor_analyzer.utils import polychoric_correlation
import numpy as np

# Two ordinal variables on 5-point Likert scale
rng = np.random.default_rng(0)
n = 500
latent = rng.multivariate_normal([0, 0], [[1, 0.6], [0.6, 1]], size=n)
thresholds = np.array([-1.5, -0.5, 0.5, 1.5])
def discretise(z):
    return np.searchsorted(thresholds, z)
x_ord = discretise(latent[:, 0])
y_ord = discretise(latent[:, 1])

r_pearson = np.corrcoef(x_ord, y_ord)[0, 1]
rho_poly = polychoric_correlation(x_ord, y_ord)
print(f"Pearson r on codes: {r_pearson:.3f}")
print(f"Polychoric rho      : {rho_poly:.3f} (true 0.60)")
```

7.7.7 7.7 When to use

Psychometrics, education research, item-response theory, structural equation modelling on Likert-scale data — any time you have ordinal indicators of a continuous construct and need correlations on the construct scale. The standard input to factor analysis of Likert items is the polychoric matrix, not the Pearson matrix.

Pearson $r = 0$ does not imply independence

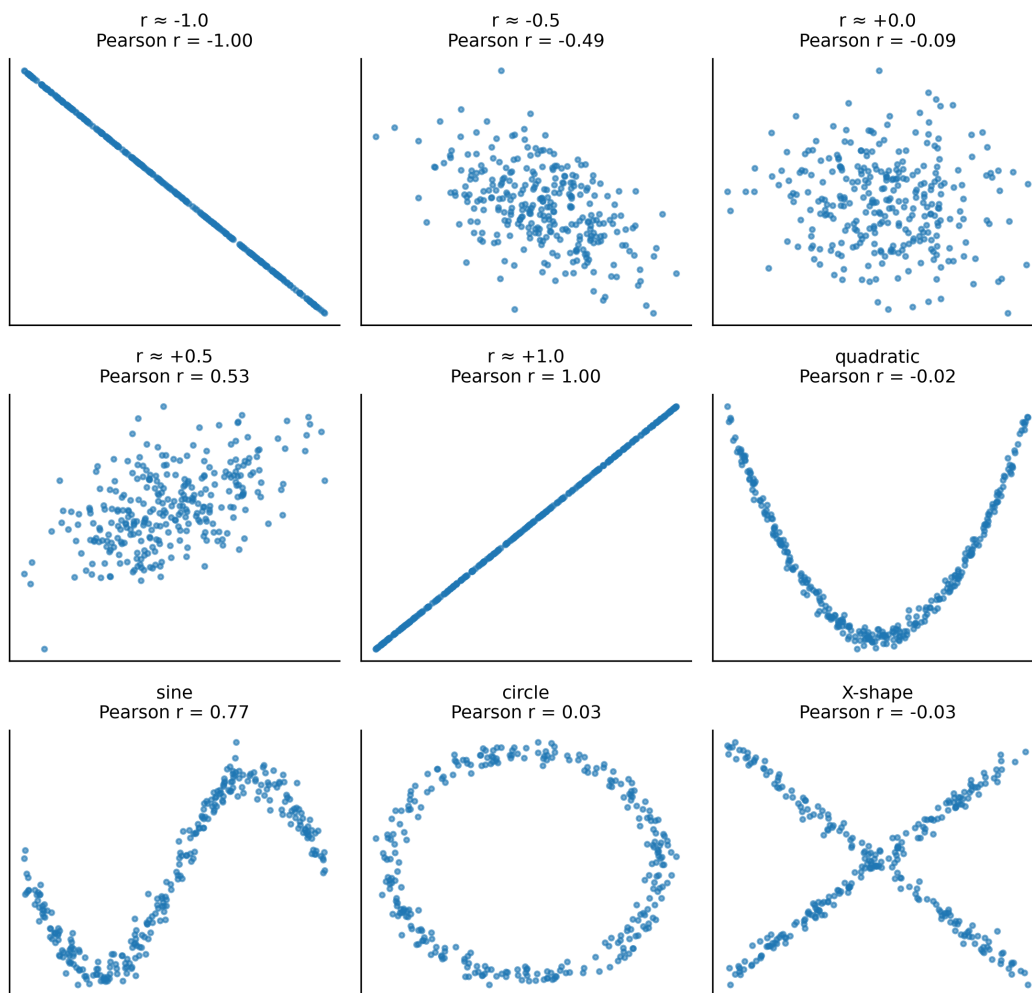


Figure 2: Pearson r across nine joint distributions. The top row varies the underlying linear coefficient; the bottom row shows distinctly non-linear (but strongly dependent) patterns whose Pearson r is near zero. $r = 0$ does *not* imply independence.

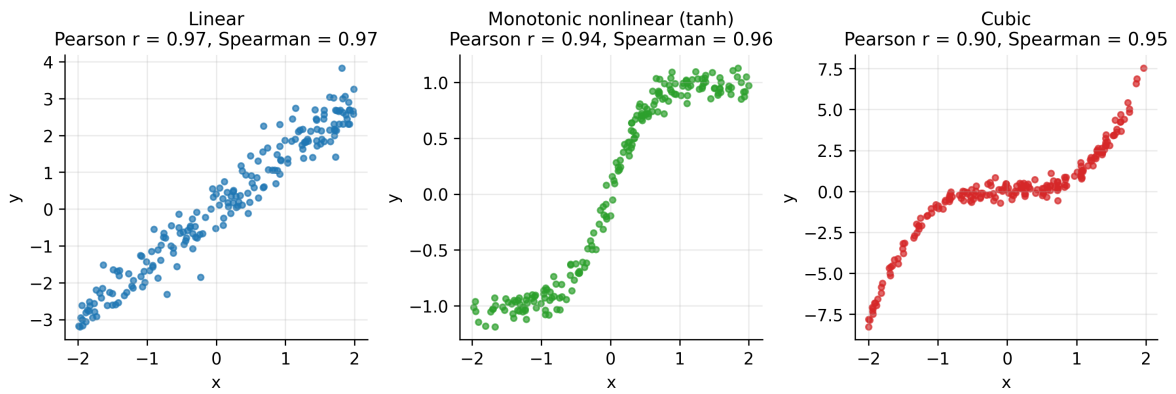


Figure 3: Pearson vs. Spearman on three relationships. Left: linear data, both high. Middle: a monotonic squashing $y = \tanh(2x)$ where Spearman stays at 1 while Pearson drops. Right: $y = x^3$, where Spearman is ≈ 1 but Pearson is markedly lower. Spearman captures monotonicity; Pearson captures linearity.

8 Part II — Information-Theoretic Dependence

The classical coefficients in Part I ask: *how close is the joint distribution to one with a specific functional form (linear, monotone, latent-normal)?* Information theory asks a strictly more general question: *how much does knowing X reduce uncertainty about Y ?* The answer makes **no assumption** about functional form. The price you pay is in estimation difficulty — even defining a mutual-information estimator for two continuous variables is nontrivial — and in interpretation: MI tells you whether and how strongly X and Y are dependent, but never *what direction* (positive or negative association) or *what shape*.

8.1 8. Mutual Information (MI)

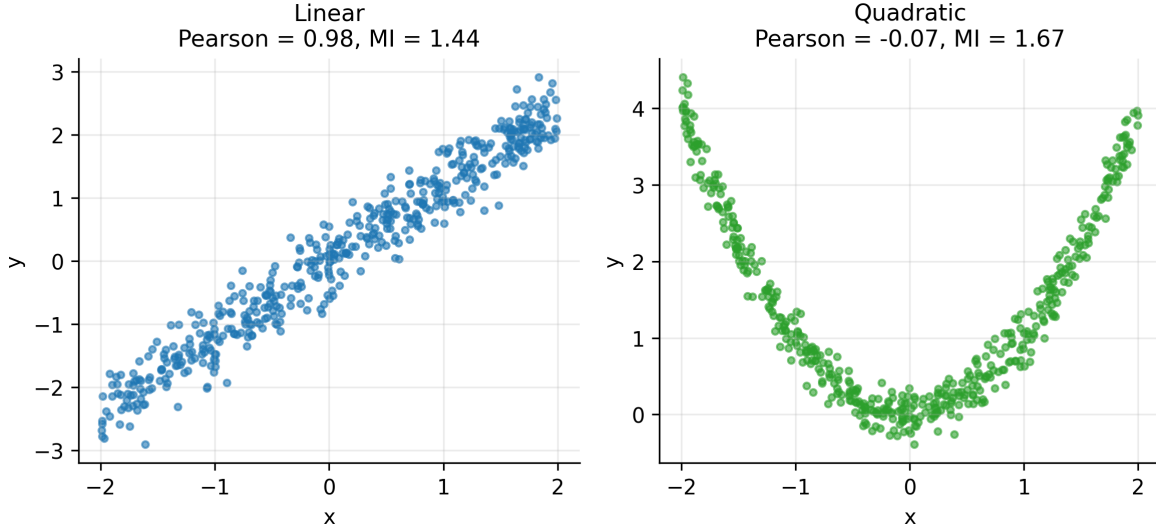


Figure 4: Mutual information vs. Pearson on a linear relationship (both informative) and a quadratic relationship (Pearson collapses to ≈ 0 , MI remains high). MI characterises any form of statistical dependence, not just linear association.

8.1.1 8.1 Definitions

For discrete X, Y with joint PMF p_{XY} :

$$I(X; Y) = \sum_{x, y} p_{XY}(x, y) \log \frac{p_{XY}(x, y)}{p_X(x)p_Y(y)}.$$

For continuous X, Y with joint density:

$$I(X; Y) = \iint p_{XY}(x, y) \log \frac{p_{XY}(x, y)}{p_X(x)p_Y(y)} dx dy.$$

Equivalent formulations:

$$I(X; Y) = H(X) + H(Y) - H(X, Y) = H(Y) - H(Y | X) = H(X) - H(X | Y) = D_{\text{KL}}(p_{XY} \parallel p_X p_Y).$$

8.1.2 8.2 Derivation of equivalences

$H(X, Y) = -\sum p(x, y) \log p(x, y) = -\sum p(x, y) [\log p(y|x) + \log p(x)] = H(Y | X) + H(X)$. Adding and subtracting $H(Y)$: $H(X) + H(Y) - H(X, Y) = H(Y) - H(Y | X)$. The KL formulation falls out by writing $\log p(x, y) - \log p(x)p(y) = \log p(y|x)/p(y)$.

8.1.3 8.3 Foundational properties

- **Non-negativity:** $I(X; Y) \geq 0$, with equality iff $X \perp Y$ (Gibbs inequality).
- **Symmetry:** $I(X; Y) = I(Y; X)$.
- **Bounded by entropy:** $I(X; Y) \leq \min(H(X), H(Y))$ for discrete variables.
- **Invariance under invertible transformations:** $I(g(X); h(Y)) = I(X; Y)$ for any bijective g, h . A much larger invariance group than Pearson or Spearman.
- **Chain rule:** $I(X; Y, Z) = I(X; Y) + I(X; Z | Y)$.
- **Data processing inequality:** if $X \rightarrow Y \rightarrow Z$ is a Markov chain, $I(X; Z) \leq I(X; Y)$.

8.1.4 8.4 Estimation — the hard part

Discrete plug-in For discrete X, Y with empirical joint $\hat{p}_{xy} = n_{xy}/n$:

$$\hat{I}_{\text{plug}} = \sum_{x,y} \hat{p}_{xy} \log \frac{\hat{p}_{xy}}{\hat{p}_x \hat{p}_y}.$$

Bias: $\mathbb{E}[\hat{I}_{\text{plug}}] - I \approx \frac{(K_{XY} - K_X - K_Y + 1)}{2n}$ where K 's are the cardinalities of the supports (Miller–Madow correction, 1955); note the sign — the plug-in estimator is biased *upward*, so naive plug-in MI *over*-estimates dependence and Miller–Madow subtracts the estimated bias. For small n and large alphabets the bias dominates; for $n \gg K_{XY}$ it is negligible. Improved estimators (Chao–Shen, Grassberger, NSB, James–Stein) reduce bias further at the cost of complexity.

Continuous: binning Discretise each variable into b bins of equal width (or equal probability), apply the discrete plug-in. Bias is highly sensitive to bin width; rules of thumb include Sturges' ($b = \lceil \log_2 n + 1 \rceil$), Freedman–Diaconis ($b = (\max - \min)/(2 \cdot \text{IQR} \cdot n^{-1/3})$), or simply $b \approx n^{1/3}$. Adaptive binning (e.g., MIC's grid search, §9) improves on this.

Continuous: k -NN (Kraskov–Stögbauer–Grassberger, KSG 2004) The modern default. Algorithm:

1. For each point (x_i, y_i) , find the k -th nearest neighbour in the joint space using Chebyshev (max-norm) distance: $\varepsilon_i = \max(|x_i - x_{i,(k)}|, |y_i - y_{i,(k)}|)$.
2. Let $n_x(i) = \#\{j \neq i : |x_j - x_i| < \varepsilon_i\}$ and $n_y(i)$ analogously.
3. KSG-1 estimator:

$$\hat{I}_{\text{KSG}} = \psi(k) - \frac{1}{n} \sum_{i=1}^n [\psi(n_x(i) + 1) + \psi(n_y(i) + 1)] + \psi(n),$$

where ψ is the digamma function.

Typical $k \in \{3, 5, 7\}$ — smaller $k \rightarrow$ lower bias, higher variance. KSG handles continuous variables on their native scale (no binning), is asymptotically unbiased, and is the implementation behind `sklearn.feature_selection.mutual_info_regression`.

Variational lower bounds (modern, for high-dim) For high-dimensional X, Y (text embeddings, image representations), histogram and kNN methods break down. Variational estimators train a neural network to maximise a lower bound on MI:

- **MINE** (Belghazi et al. 2018): Donsker–Varadhan bound, $I(X; Y) \geq \mathbb{E}_{p_{XY}}[T] - \log \mathbb{E}_{p_{XPY}}[e^T]$.
- **InfoNCE** (Oord et al. 2018): $I(X; Y) \geq \log K - L_{\text{NCE}}$, where K is batch size. Because the estimator itself cannot exceed $\log K$, InfoNCE is *incapable* of detecting $I > \log K$ — a hard ceiling that saturates at modest values of I . Detecting $I = b$ nats faithfully therefore requires batch size at least e^b , which is exponential in the quantity being estimated (McAllester & Stratos 2020 establish this as a fundamental limit of any distribution-free high-confidence lower bound on MI).
- **NWJ** (Nguyen–Wainwright–Jordan 2010): $I(X; Y) \geq \mathbb{E}[T] - e^{-1} \mathbb{E}[e^T]$.

McAllester & Stratos (2020) and Song & Ermon (2020) show all variational MI bounds have either large bias, large variance, or both — use them for representation learning, not for absolute MI estimates.

8.1.5 8.5 The canonical "MI sees what Pearson misses" example

```
import numpy as np
from sklearn.feature_selection import mutual_info_regression
from scipy.stats import pearsonr

rng = np.random.default_rng(0)
n = 2000
x = rng.uniform(-1, 1, size=n)
y = x ** 2 + 0.05 * rng.normal(size=n) # quadratic

r, _ = pearsonr(x, y)
mi = mutual_info_regression(x.reshape(-1, 1), y, n_neighbors=3, random_state=0)[0]
print(f"Pearson r: {r:.4f}    MI (KSG): {mi:.4f}")
# Pearson ~ 0; MI (KSG, k=3) typically lands in ~1.0-1.4 nats here.
```

8.1.6 8.6 Pitfalls

- **No sign:** MI cannot tell you positive vs negative association.
- **Hard to compare across datasets** without a reference — see NMI (§10).
- **Curse of dimensionality:** KSG works well up to ~5–8 dimensions per side; beyond that variance explodes.
- **Sample-size hungry** for high MI: estimator variance scales with I itself; reliable estimation of $I = 5$ nats requires substantially more data than $I = 0.5$ nats.
- **No closed-form null distribution:** significance is almost always assessed via permutation.

8.1.7 8.7 When to use / when not

Use when you want a *fully general* measure of dependence, have enough data ($n \gtrsim 1000$ for continuous), and the dimensionality is moderate. Feature selection where non-linearity is expected; information-bottleneck analysis; communication theory; representation learning (with caveats).

Avoid for small n with continuous variables and complex marginal shapes — estimator variance can swamp the signal. Use dCor or HSIC instead.

8.1.8 8.8 Cross-references

For a normalised version see [NMI](#). For the conditional version see [CMI](#). For a directional time-series version see [Transfer Entropy](#). For modern alternatives with cleaner estimation theory see [dCor](#) and [HSIC](#).

8.2 9. Maximal Information Coefficient (MIC)

8.2.1 9.1 Definition

Reshef et al. (Science 2011). For continuous X, Y and grid sizes a, b (number of bins on each axis), let $I^*(X, Y; a, b)$ be the maximum empirical mutual information over all $a \times b$ partitions of the data. Then

$$\text{MIC}(X, Y) = \max_{ab \leq B(n)} \frac{I^*(X, Y; a, b)}{\log \min(a, b)},$$

with $B(n) = n^\alpha$ ($\alpha = 0.6$ by default).

8.2.2 9.2 Intuition

For each "grid resolution," MIC searches for the *most informative* binning, normalises by the maximum-possible MI on a uniform $\min(a, b)$ -bin distribution, and reports the supremum over grid sizes. The normalisation ensures $\text{MIC} \in [0, 1]$. The grid-size cap $B(n)$ prevents the search from overfitting at high resolution.

8.2.3 9.3 The "equitability" claim and the controversy

Reshef et al. claimed MIC has an **equitability** property: equally noisy relationships of different functional types (linear, sinusoidal, parabolic, exponential) should receive equal MIC scores. This made MIC immediately appealing for exploratory screening of "any kind of relationship."

Kinney & Atwal (PNAS 2014) proved that **no nontrivial dependence measure can be equitable in the original formalisation**: any measure invariant under monotonic transforms and depending only on the dependence structure (the copula) must violate equitability for some pair of noise models. Subsequent work (Reshef et al. 2016) reformulated equitability and introduced **MICe** (a much faster approximation), but the controversy has dampened initial enthusiasm.

8.2.4 9.4 Properties and performance

- Range $[0, 1]$, symmetric, no sign.
- MIC = 1 for noise-free functional relationships *given sufficient* n .
- **Lower statistical power than dCor and HSIC** on many smooth alternatives (Simon & Tibshirani 2014 simulation benchmarks).
- Computationally expensive: original MIC is $O(n^{2.4})$; MICe is $O(n^{1.2})$.

8.2.5 9.5 Code

```
# pip install minepy
from minepy import MINE
import numpy as np

rng = np.random.default_rng(0)
n = 1000
x = rng.uniform(0, 1, n)
y = np.sin(20 * x) + 0.2 * rng.normal(size=n)

mine = MINE(alpha=0.6, c=15) # alpha: B(n)=n^alpha cap; c: clump factor for grid search
mine.compute_score(x, y)
print(f"MIC = {mine.mic():.3f}")
```

8.2.6 9.6 Pitfalls

- The "any kind of relationship" claim is **empirical**, not theoretical.
- For most modern applications, **dCor or HSIC are simpler, faster, and more powerful**.
- The grid-size cap $\alpha = 0.6$ is a hyperparameter; results are sensitive to it for small n .

8.2.7 9.7 When to use

When you genuinely want a $[0, 1]$ scalar that returns 1 for any noise-free functional relationship and you do not care about hypothesis tests. Honest assessment: in 2026, prefer dCor or HSIC unless you have a specific reason to want MIC's equitability heuristic.

8.3 10. Normalized Mutual Information (NMI)

8.3.1 10.1 Family of definitions

NMI is not a single quantity — it is at least four variants that rescale MI into $[0, 1]$. With H denoting Shannon entropy:

Variant	Formula	Notes
Arithmetic mean	$2I(X; Y)/(H(X) + H(Y))$	sklearn default
Geometric mean	$I(X; Y)/\sqrt{H(X)H(Y)}$	"Strehl-Ghosh" form
Min	$I(X; Y)/\min(H(X), H(Y))$	optimistic — closest to 1
Max	$I(X; Y)/\max(H(X), H(Y))$	pessimistic — most conservative

All four agree iff $H(X) = H(Y)$.

8.3.2 10.2 Why normalise

For raw MI, $I(X; Y) = 2$ nats means different things on a 4-symbol alphabet (where max is $\log 4 \approx 1.39$) versus a 100-symbol alphabet (max $\log 100 \approx 4.6$). NMI factors this out, giving a "fraction of shared entropy" interpretation.

8.3.3 10.3 Cluster evaluation — the dominant use case

NMI is the de facto standard for comparing two partitions U, V of the same data. Treat cluster labels as random variables; compute NMI between them; report.

Two important variants in this context:

- **Adjusted Mutual Information (AMI)**: subtracts the *expected* MI under random labellings (analogous to the Adjusted Rand Index) before normalising. Concretely,

$$\text{AMI}(U, V) = \frac{I(U; V) - \mathbb{E}[I(U; V)]}{\text{avg}(H(U), H(V)) - \mathbb{E}[I(U; V)]},$$

where the expectation is over a hypergeometric model preserving the cluster-size marginals. NMI rewards finer partitions (more clusters $\rightarrow$ higher entropy $\rightarrow$ easier to be informative); AMI corrects this. Vinh et al. (2010) is the canonical reference.

- **Adjusted Rand Index (ARI)**: pair-counting analogue, sometimes preferred for partition comparison; AMI for information-theoretic, ARI for pair-counting.

8.3.4 10.4 Code

```
from sklearn.metrics import (
    normalized_mutual_info_score,
    adjusted_mutual_info_score,
    adjusted_rand_score,
)

labels_true = [0, 0, 1, 1, 2, 2, 3, 3]
labels_pred = [0, 0, 1, 1, 1, 1, 2, 2]
print(f"NMI (am): {normalized_mutual_info_score(labels_true, labels_pred,
    average_method='arithmetic'):.3f}")
print(f"NMI (gm): {normalized_mutual_info_score(labels_true, labels_pred,
    average_method='geometric'):.3f}")
print(f"AMI: {adjusted_mutual_info_score(labels_true, labels_pred):.3f}")
print(f"ARI: {adjusted_rand_score(labels_true, labels_pred):.3f}")
```

8.3.5 10.5 Pitfalls

- **Different normalisations give different rankings** — always report which one.
- **NMI is biased upward toward finer partitions** when comparing clusterings of different cardinalities; **prefer AMI** for empirical clustering comparison.
- **NMI for continuous variables** inherits all of MI's estimation difficulties; it is overwhelmingly used in the discrete (clustering) setting.

8.3.6 10.6 When to use

Clustering evaluation; comparing categorical-label distributions; any setting where a normalised "fraction of shared information" is the interpretable summary.

8.4 11. Transfer Entropy

8.4.1 11.1 Definition

Schreiber (PRL 2000). For two time series X_t, Y_t and history lengths k (for Y) and l (for X),

$$\text{TE}_{X \rightarrow Y}(k, l) = \sum p(y_{t+1}, y_t^{(k)}, x_t^{(l)}) \log \frac{p(y_{t+1} | y_t^{(k)}, x_t^{(l)})}{p(y_{t+1} | y_t^{(k)})},$$

where $y_t^{(k)} = (y_t, y_{t-1}, \dots, y_{t-k+1})$.

Equivalently, $\text{TE}_{X \rightarrow Y} = I(Y_{t+1}; X_t^{(l)} | Y_t^{(k)})$: the conditional mutual information between Y 's next value and X 's past, given Y 's own past.

8.4.2 11.2 Intuition

By *conditioning out the past of Y* , TE isolates the *additional* information about Y_{t+1} that comes from X 's past — over and above what Y 's own dynamics already explain. This makes it directional in a way ordinary MI is not.

8.4.3 11.3 Relation to Granger causality

For jointly Gaussian, linear, stationary (X_t, Y_t) processes, transfer entropy is monotonically related to the Granger causality F -statistic. Barnett, Barrett & Seth (2009) showed:

$$\text{TE}_{X \rightarrow Y} = \frac{1}{2} \log \frac{\text{Var}(Y_{t+1} | Y_t^{(k)})}{\text{Var}(Y_{t+1} | Y_t^{(k)}, X_t^{(l)})} = \frac{1}{2} \text{GC}_{X \rightarrow Y},$$

where $\text{GC}_{X \rightarrow Y} := \log \frac{\text{Var}(Y_{t+1} | Y_t^{(k)})}{\text{Var}(Y_{t+1} | Y_t^{(k)}, X_t^{(l)})}$ is the (population) Granger-causality statistic; the sample F-statistic used to *test* $\text{GC} = 0$ is a separate object. So TE is the *nonlinear, non-parametric generalisation* of Granger causality.

8.4.4 11.4 Properties

- $\text{TE}_{X \rightarrow Y} \geq 0$, with equality iff X 's past provides no additional information about Y_{t+1} beyond Y 's own past.
- **Directional**: $\text{TE}_{X \rightarrow Y} \neq \text{TE}_{Y \rightarrow X}$ in general; the difference is sometimes called the *net* transfer entropy.
- Can be normalised: $\text{nTE}_{X \rightarrow Y} = \text{TE}_{X \rightarrow Y} / H(Y_{t+1} | Y_t^{(k)}) \in [0, 1]$.

8.4.5 11.5 Estimation

- **Discrete or symbolised series**: bin or symbolise (rank short windows), apply discrete CMI estimator. Symbolic TE (Staniek & Lehnertz 2008) is robust to choice of bin width.
- **Continuous**: KSG-style estimator extended to conditional MI (Frenzel & Pompe 2007).
- **Effective TE**: subtract the bias estimated from shuffled surrogates. The reporting standard.

8.4.6 11.6 Code

```
# pip install PyInform
from pyinform.transferentropy import transfer_entropy
import numpy as np

rng = np.random.default_rng(0)
n = 5000
x = rng.integers(0, 2, size=n)
y = np.zeros(n, dtype=int)
for t in range(1, n):
    y[t] = x[t - 1] ^ rng.integers(0, 2) # noisy XOR of x's past
te_x_to_y = transfer_entropy(x, y, k=1)
te_y_to_x = transfer_entropy(y, x, k=1)
print(f"TE X->Y: {te_x_to_y:.4f}    TE Y->X: {te_y_to_x:.4f}")
# X->Y should be much larger
```

8.4.7 11.7 Pitfalls

- **Choice of history lengths** (k, l) is consequential. Common: select by minimising AIC of the conditional density estimate, or use the Ragwitz criterion. Cross-validate where possible.
- **Stationarity and ergodicity** are both required. Stationarity alone is not enough: TE estimators replace ensemble averages by time averages along a single trajectory, which is only valid under ergodicity. Non-ergodic stationary processes (e.g. regime-switching systems that never mix) produce TE estimates that depend on which trajectory was sampled. Non-stationary series should be detrended or differenced before estimation.
- **TE is not causality** in the interventional sense — confounders can produce spurious TE. The terminology "directional information flow" is more accurate.
- **Sample size:** reliable continuous-TE estimation typically requires $n \gtrsim 10^4$.

8.4.8 11.8 When to use

Neuroscience (information flow between brain regions in EEG/fMRI); finance (lead-lag between assets); climate (teleconnections); any time-series setting where direction matters and you cannot assume linearity.

8.5 12. Conditional Mutual Information (CMI)

8.5.1 12.1 Definition

$$I(X; Y | Z) = \mathbb{E}_Z[I(X; Y | Z = z)] = \iiint p(x, y, z) \log \frac{p(x, y | z)}{p(x | z) p(y | z)} dx dy dz.$$

Equivalent: $I(X; Y | Z) = H(X | Z) + H(Y | Z) - H(X, Y | Z)$.

8.5.2 12.2 Intuition

CMI measures the dependence between X and Y that *remains* after controlling for Z . It is the information-theoretic analogue of **partial correlation**: where partial correlation removes a linear effect of Z , CMI removes the *full* statistical effect of Z , regardless of functional form.

8.5.3 12.3 Key identities

- **Independence characterisation:** $I(X; Y | Z) = 0 \iff X \perp Y | Z$.
- **Chain rule:** $I(X; Y, Z) = I(X; Z) + I(X; Y | Z)$.
- **Non-monotonicity in Z :** $I(X; Y | Z)$ can be *larger* or *smaller* than $I(X; Y)$ — conditioning on a collider (the "v-structure" $X \rightarrow Z \leftarrow Y$; the resulting bias when sampling on Z is sometimes also called Berkson's paradox) introduces dependence between independent parents. Example: X, Y independent Bernoulli(1/2), $Z = X \oplus Y$; then $I(X; Y) = 0$ but $I(X; Y | Z) > 0$.

8.5.4 12.4 Estimation

- **kNN (Frenzel–Pompe 2007)**: extends KSG to the conditional case. With k the chosen neighbour order, Chebyshev distances in (X, Y, Z) space, and $n_{xz}(i), n_{yz}(i), n_z(i)$ the counts of points within the marginal slabs at point i ,

$$\hat{I}(X; Y | Z) = \psi(k) - \frac{1}{n} \sum_i [\psi(n_{xz}(i) + 1) + \psi(n_{yz}(i) + 1) - \psi(n_z(i) + 1)].$$

- **Copula-based**: estimate the conditional copula and integrate.
- **Variational**: extends MINE/InfoNCE to conditional. High variance in practice.
- **KCIT (Zhang et al. 2011)**: kernel-based CI test that produces a CMI-like statistic.

Estimator variance grows quickly with $\dim(Z)$. For $\dim(Z) > 3$, expect difficulty.

8.5.5 12.5 Causal discovery — why it matters

CI tests based on CMI (or its kernel proxies) are the workhorse of **constraint-based causal discovery** (PC, FCI algorithms): learn the skeleton of the underlying DAG by removing edges (X, Y) for which $\exists S$ with $X \perp Y | S$. Faithfulness assumption: *every* conditional independence in the distribution is entailed by d-separation in the DAG — i.e., no two paths' effects cancel by coincidence (the Markov assumption supplies the converse direction).

8.5.6 12.6 Code

```
# Install: pip install git+https://github.com/gregversteeg/NPEET.git (not on PyPI)
import numpy as np
from npeet import entropy_estimators as ee

rng = np.random.default_rng(0)
n = 5000
z = rng.normal(size=(n, 1))
x = z + 0.5 * rng.normal(size=(n, 1))
y = z + 0.5 * rng.normal(size=(n, 1))
# X and Y are dependent through Z; should be independent given Z

mi_xy = ee.mi(x, y, k=5)
cmi_xy_given_z = ee.cmi(x, y, z, k=5)
print(f"I(X; Y)           = {mi_xy:.4f} (expect > 0)")
print(f"I(X; Y | Z)        = {cmi_xy_given_z:.4f} (expect ~ 0)")
```

8.5.7 12.7 Pitfalls

- **Conditioning is expensive**: estimator variance is brutal when Z is high-dimensional.
- **Faithfulness violations**: causal-discovery algorithms based on CMI assume that all and only the CI relations in the data reflect the causal graph. Generic faithfulness holds almost everywhere in parameter space but can fail at parameter "knife-edges" (e.g., cancellation in linear systems).
- **Significance testing**: no closed-form null; permutation must respect the conditioning, e.g. local permutation (Runge 2018).

8.5.8 12.8 When to use

Causal discovery; partial-correlation-like analysis with non-linear relationships; Markov-blanket feature selection (X_i is irrelevant given the rest iff $I(Y; X_i | X_{-i}) = 0$); confounder adjustment.

9 Part III — Geometric, Kernel, and Modern Independence Measures

This part collects measures that share three philosophical commitments: (1) they detect *any* form of dependence, not just linear or monotonic; (2) they admit clean theoretical characterisations of independence; (3) they exploit rich representations of the data (distance matrices, RKHS embeddings, rank structures) rather than histograms of values. They are the modern statistician's answer to the limitations of Pearson and the estimation difficulties of MI.

9.1 13. Distance Correlation (dCor)

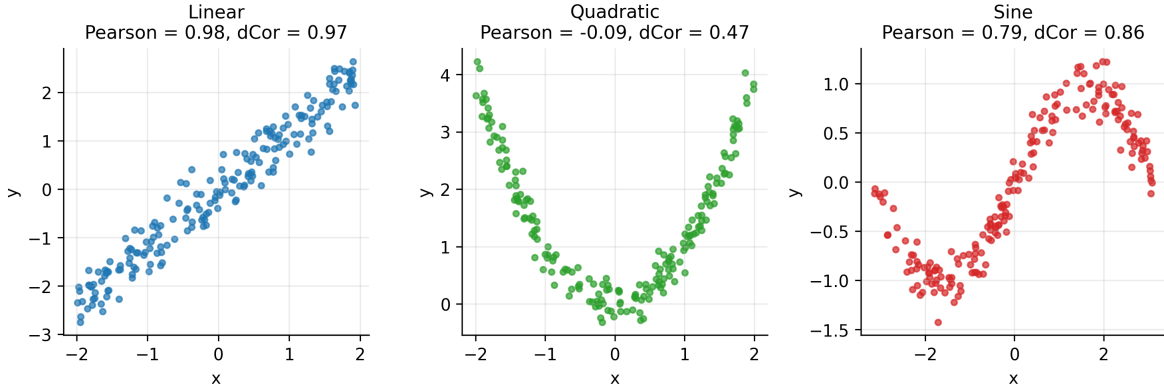


Figure 5: Distance correlation (dCor) vs. Pearson on three relationships. dCor remains positive in the quadratic and sine settings where Pearson drops to ≈ 0 . dCor is zero *iff* the variables are independent, making it a true test of dependence.

9.1.1 13.1 Definition (sample)

Székely, Rizzo, Bakirov (Annals of Statistics 2007). Given samples (x_i, y_i) for $i = 1, \dots, n$ where $x_i \in \mathbb{R}^p$, $y_i \in \mathbb{R}^q$:

1. **Pairwise distance matrices:** $a_{ij} = \|x_i - x_j\|$, $b_{ij} = \|y_i - y_j\|$.
2. **Double-centring:** $A_{ij} = a_{ij} - \bar{a}_{i.} - \bar{a}_{.j} + \bar{a}_{..}$ where $\bar{a}_{i.} = \frac{1}{n} \sum_j a_{ij}$, $\bar{a}_{.j} = \frac{1}{n} \sum_i a_{ij}$, $\bar{a}_{..} = \frac{1}{n^2} \sum_{i,j} a_{ij}$. Similarly B_{ij} .
3. **Distance covariance:** $\text{dCov}^2(X, Y) = \frac{1}{n^2} \sum_{i,j} A_{ij} B_{ij}$.
4. **Distance correlation:** $\text{dCor}(X, Y) = \text{dCov}(X, Y) / \sqrt{\text{dCov}(X, X) \text{dCov}(Y, Y)}$.

9.1.2 13.2 Population definition (characteristic-function form)

$$\text{dCov}^2(X, Y) = \int_{\mathbb{R}^{p+q}} |\varphi_{XY}(t, s) - \varphi_X(t) \varphi_Y(s)|^2 w(t, s) dt ds,$$

where $w(t, s) = (c_p c_q \|t\|^{p+1} \|s\|^{q+1})^{-1}$ for specific constants $c_d = \pi^{(d+1)/2} / \Gamma((d+1)/2)$.

This is the L2 distance between the joint characteristic function and the product of marginals — a "spectral" measure of dependence. **It equals zero iff $X \perp Y$** (which is exactly the characterisation of independence by characteristic functions, P3). This is the theoretical headline.

9.1.3 13.3 Derivation of the equivalence between sample and population forms

Astonishingly, the elaborate weighted-integral expression has the simple distance-matrix computation as its sample analogue. The key identity (Székely & Rizzo 2007, Thm 7) is

$$\int \frac{1 - \cos(\langle t, x - y \rangle)}{c_p \|t\|^{p+1}} dt = \|x - y\|,$$

which lets you convert the integral over characteristic functions into one over pairwise distances. The double-centring step removes the marginal expectations and is what makes the empirical dCov^2 a U-statistic-like quantity.

9.1.4 13.4 Bias-corrected estimator

The V-statistic above has positive bias under independence. Székely & Rizzo (2014) introduced the **U-statistic dCov**:

$$U_n = \frac{1}{n(n-3)} \sum_{i \neq j} \tilde{A}_{ij} \tilde{B}_{ij},$$

where the *U-centred* distance matrix $\tilde{A}_{ij}$ is

$$\tilde{A}_{ij} = a_{ij} - \frac{1}{n-2} \sum_k a_{ik} - \frac{1}{n-2} \sum_k a_{kj} + \frac{1}{(n-1)(n-2)} \sum_{k,l} a_{kl}$$

(denominators $n-2$, $n-2$, $(n-1)(n-2)$), and similarly for $\tilde{B}_{ij}$. U_n has expectation zero under independence and is the recommended statistic for hypothesis testing. Note that because U_n is centred at zero, the bias-corrected sample dCov can be *negative* for finite samples even though the population dCov is non-negative.

9.1.5 13.5 Properties

- $\text{dCor} \in [0, 1]$ (vs $\text{dCov} \in [0, \infty)$).
- Symmetric, no sign.
- Invariant under translations, orthogonal transformations, and global scaling of X and Y separately.
- For univariate, jointly normal (X, Y) with Pearson correlation ρ : $\text{dCor}^2 = (\rho \arcsin \rho + \sqrt{1 - \rho^2} - \rho \arcsin(\rho/2) - \sqrt{4 - \rho^2 + 1}) / (1 + \pi/3 - \sqrt{3})$. Numerically $\text{dCor} \approx |\rho|$ but slightly smaller. Equality at $\rho = 0$ and $\rho = \pm 1$.
- Generalises to arbitrary metric spaces with negative-definite metrics (Lyons 2013).

9.1.6 13.6 Asymptotic null distribution

Under $H_0 : X \perp Y$,

$$n \cdot \text{dCov}^2 \rightarrow_d \sum_{j=1}^{\infty} \lambda_j Z_j^2$$

where Z_j are iid standard normal and λ_j are eigenvalues of a specific integral operator. This is a mixture of χ^2 ; closed-form p -values are hard, so **permutation tests** are the practical standard. A gamma approximation is often used.

9.1.7 13.7 Fast algorithms

- **Generic case** ($p, q > 1$): $O(n^2)$ time and memory. Limits practical use to $n \lesssim 10^5$.
- **Univariate case** ($p = q = 1$): Huo & Székely (2016) achieved $O(n \log n)$ via clever sorting + cumulative sums. Implemented in `dcor.distance_covariance(x, y, method="mergesort")`.

9.1.8 13.8 Code

```
# pip install dcor
import numpy as np
import dcor

rng = np.random.default_rng(0)
n = 500
x = rng.uniform(-1, 1, size=n)
y_lin = 0.7 * x + 0.3 * rng.normal(size=n)
y_quad = x ** 2 + 0.05 * rng.normal(size=n)
y_indep = rng.normal(size=n)

for name, y in [("linear", y_lin), ("quadratic", y_quad), ("independent", y_indep)]:
    dc = dcor.distance_correlation(x, y)
    test = dcor.independence.distance_covariance_test(x, y, num_resamples=999, random_state=0)
    print(f"{name:12s} dCor = {dc:.3f} p = {test.pvalue:.3g}")
```

For the quadratic case, dCor will be substantially positive while Pearson's r is ~ 0 .

9.1.9 13.9 Pitfalls

- $O(n^2)$ **memory** makes dCor uncomfortable for $n \gtrsim 10^5$ unless you use the fast univariate algorithm.
- **Magnitude is hard to interpret in absolute terms** — most useful as a ranking or as a test statistic. Do not transfer a Pearson rule-of-thumb ("0.3 weak, 0.7 strong") directly.
- **Bias under independence** in the V-statistic form: use the bias-corrected estimator for tests.

9.1.10 13.10 When to use

Any general-purpose dependence detection in low-to-moderate sample sizes where you want a single bounded scalar with a clean independence characterisation. Particularly valuable for **multivariate** X and Y — compute dCor between a vector of features and a vector of outcomes directly.

9.2 14. Brownian Covariance

9.2.1 14.1 Definition

Let $U(t), V(s)$ be independent standard Brownian motions on $\mathbb{R}^p, \mathbb{R}^q$, independent of X, Y . The **Brownian covariance** is

$$\text{BCov}^2(X, Y) = \text{Cov}^2(U(X), V(Y)),$$

where the outer covariance is taken jointly over (X, Y, U, V) .

9.2.2 14.2 Equivalence to dCov

Theorem (Székely & Rizzo, Annals of Applied Statistics 2009): $\text{BCov}^2(X, Y) = \text{dCov}^2(X, Y)$.

9.2.3 14.3 Why this equivalence matters

Brownian covariance places dCov inside the **Gaussian process covariance** framework: it is the covariance of two Brownian motions evaluated at the data points. This is one of several routes to seeing dCor as a special case of HSIC (§16) with a particular reproducing kernel — namely the **Brownian motion covariance kernel**

$$k(x, x') = \frac{1}{2}(\|x\| + \|x'\| - \|x - x'\|).$$

The equivalence explains why dCor and HSIC frequently agree empirically — they are two parametrisations of the same broader family of "energy-distance / kernel-mean-embedding" independence measures. In particular, **distance covariance** $\equiv$ **HSIC with the Brownian motion kernel**.

9.2.4 14.4 Practical implication

You almost never compute Brownian covariance directly. You compute dCov, which is equal to it, using the distance-matrix algorithm. The Brownian covariance perspective is theoretical glue, valuable for understanding *why* dCor works and how it relates to kernel methods.

9.2.5 14.5 Generalisations

Sejdinovic et al. (2013) unified distance-based and kernel-based independence statistics: for any **negative-definite metric** d , the corresponding " d -distance covariance" equals an HSIC with kernel $k(x, x') = d(x, x_0) + d(x_0, x') - d(x, x')$ for any choice of origin x_0 . This shows that the difference between dCor and HSIC is, fundamentally, just a choice of kernel.

9.3 15. Kernel Canonical Correlation Analysis (KCCA)

9.3.1 15.1 From CCA to KCCA

Classical **CCA** (Hotelling 1936): given two random vectors $X \in \mathbb{R}^p$, $Y \in \mathbb{R}^q$, find linear combinations $\alpha^\top X$ and $\beta^\top Y$ with maximum Pearson correlation:

$$\rho_{\text{CCA}} = \sup_{\alpha, \beta} \frac{\alpha^\top \Sigma_{XY} \beta}{\sqrt{\alpha^\top \Sigma_{XX} \alpha \cdot \beta^\top \Sigma_{YY} \beta}}.$$

Solved by a generalised eigenvalue problem.

KCCA replaces linear combinations with arbitrary functions in two RKHSs:

$$\rho_{\text{KCCA}} = \sup_{f \in \mathcal{H}_X, g \in \mathcal{H}_Y} \frac{\text{Cov}(f(X), g(Y))}{\sqrt{\text{Var}(f(X)) \text{Var}(g(Y))}}.$$

9.3.2 15.2 Sample computation

With kernel matrices K_X, K_Y (each $n \times n$) and centring $\tilde{K} = HKH$ where $H = I - \frac{1}{n}\mathbf{1}\mathbf{1}^\top$, KCCA reduces to the generalised eigenvalue problem

$$\begin{pmatrix} 0 & \tilde{K}_X \tilde{K}_Y \\ \tilde{K}_Y \tilde{K}_X & 0 \end{pmatrix} \begin{pmatrix} \alpha \\ \beta \end{pmatrix} = \lambda \begin{pmatrix} (\tilde{K}_X + \varepsilon I)^2 & 0 \\ 0 & (\tilde{K}_Y + \varepsilon I)^2 \end{pmatrix} \begin{pmatrix} \alpha \\ \beta \end{pmatrix}.$$

The regularisation $\varepsilon > 0$ is **critical** — without it, for any universal kernel the empirical KCCA always equals 1 (you can always find f, g such that $f(X) = g(Y)$ exactly on the data, since $\phi_X(X)$ and $\phi_Y(Y)$ span n -dimensional subspaces in the RKHSs).

9.3.3 15.3 Properties

- Returns multiple canonical components (eigenvalues), useful for **multi-view representation learning**.
- The leading canonical correlation provides a dependence measure but is *not* an independence characterisation in the sharp HSIC sense without careful regularisation tuning.
- Underlies **kernel ICA** (Bach & Jordan 2002): minimising the leading canonical correlation under whitened sources separates independent components.

9.3.4 15.4 Cost

$O(n^3)$ for the generalised eigenproblem on $n \times n$ kernel matrices, prohibitive beyond $n \approx 10^4$ without Nyström or **random Fourier feature** approximations.

9.3.5 15.5 Code

```
# pip install mlearn
from mvlearn.embed import KCCA
import numpy as np

rng = np.random.default_rng(0)
n, d = 200, 10
shared = rng.normal(size=(n, 3))
X = shared @ rng.normal(size=(3, d)) + 0.5 * rng.normal(size=(n, d))
Y = np.tanh(shared @ rng.normal(size=(3, d))) + 0.5 * rng.normal(size=(n, d))

kcca = KCCA(n_components=2, regs=[1e-2, 1e-2], kernel="rbf")
embeddings = kcca.fit_transform([X, Y])
# Correlation between the two leading components
import numpy as np
print(f"KCCA component 1 corr: {np.corrcoef(embeddings[0][:, 0], embeddings[1][:, 0])[0, 1]:.3f}")
```

9.3.6 15.6 Deep CCA

Andrew et al. (2013) replaced the kernel feature map with a neural network, learning f, g jointly by gradient descent to maximise canonical correlation in the network outputs. DCCA is now the standard tool for non-linear multi-view representation learning at scale.

9.3.7 15.7 When to use

Multi-view representation learning (audio + video, image + text), finding shared non-linear structure between two feature sets, multimodal fusion. For dependence *testing*, prefer HSIC.

9.4 16. Hilbert–Schmidt Independence Criterion (HSIC)

9.4.1 16.1 Definition (population)

Gretton, Bousquet, Smola, Schölkopf (ALT 2005). For characteristic kernels k_X, k_Y with feature maps ϕ_X, ϕ_Y into RKHSs $\mathcal{H}_X, \mathcal{H}_Y$:

$$\text{HSIC}(X, Y) = \|\mathbb{E}_{XY}[\phi_X(X) \otimes \phi_Y(Y)] - \mathbb{E}_X[\phi_X(X)] \otimes \mathbb{E}_Y[\phi_Y(Y)]\|_{\text{HS}}^2.$$

This is the squared Hilbert–Schmidt norm of the **cross-covariance operator** between the two RKHSs.

9.4.2 16.2 Sample estimator

With kernel matrices K, L (each $n \times n$) and centring matrix $H = I - \frac{1}{n}\mathbf{1}\mathbf{1}^\top$:

$$\widehat{\text{HSIC}}_V = \frac{1}{(n-1)^2} \text{tr}(KHLH) = \frac{1}{(n-1)^2} \text{tr}(\tilde{K}\tilde{L}),$$

where $\tilde{K} = HKH$ is the centred kernel matrix. This is the biased V -statistic estimator with bias $O(1/n)$; the *unbiased U -statistic* version (Song, Smola, Gretton et al. 2012) — which can take negative values under independence — is preferred for hypothesis testing.

9.4.3 16.3 Why it characterises independence

The mean embedding $\mu_P = \mathbb{E}[\phi(X)]$ is, for characteristic kernels, an *injective* map from probability distributions into the RKHS. HSIC measures the RKHS distance between the embedding of the joint distribution and the embedding of the product of marginals. With characteristic kernels (the Gaussian RBF is characteristic on $\mathbb{R}^d$), this distance is zero iff the distributions agree, i.e., iff $X \perp Y$.

9.4.4 16.4 Relationship to dCor

With the Brownian-motion covariance kernel, HSIC equals dCov (Sejdinovic et al. 2013, see §14). With a Gaussian kernel and judicious bandwidth, HSIC behaves very similarly to dCor in practice. Both are members of a broader family of "energy-distance / kernel-mean-embedding" independence measures.

9.4.5 16.5 Asymptotic null distribution

Under H_0 of independence the U-statistic estimator satisfies

$$n \cdot \widehat{\text{HSIC}}_U \rightarrow_d \sum_j \lambda_j (Z_j^2 - 1),$$

a centred infinite sum of weighted χ_1^2 ; the biased V-statistic above has the un-centred limit $\sum_j \lambda_j Z_j^2$ (i.e. shifted by the trace of the integral operator). **Gamma approximation:** $n \cdot \widehat{\text{HSIC}}_V \sim \text{Gamma}(\alpha, \beta)$ with the moment match $\alpha = (\mathbb{E}\widehat{\text{HSIC}})^2 / \text{Var}(\widehat{\text{HSIC}})$, $\beta = \text{Var}(\widehat{\text{HSIC}}) / \mathbb{E}\widehat{\text{HSIC}}$ (Gretton, Fukumizu, Teo, Song, Schölkopf, Smola 2008). Fast and reasonably accurate for $n \gtrsim 200$. **Permutation tests** are the most common practical approach.

9.4.6 16.6 Variants

- **HSIC Lasso** (Yamada et al. 2014): non-linear feature selection by sparse regression on per-feature HSIC scores. The optimisation $\min_{\beta} \|\bar{L} - \sum_i \beta_i \bar{K}_i\|_F^2 + \lambda \|\beta\|_1$ selects features with high HSIC to the target while avoiding redundancy.
- **Centered Kernel Alignment (CKA)** (Cortes et al. 2012, popularised by Kornblith et al. 2019): the cosine version of HSIC,

$$\text{CKA}(X, Y) = \frac{\text{HSIC}(X, Y)}{\sqrt{\text{HSIC}(X, X) \cdot \text{HSIC}(Y, Y)}}.$$

CKA has become the dominant tool for comparing **neural network representations** across layers, training runs, and architectures.

- **Conditional HSIC / HSCIC** (Park & Muandet 2020): extends to conditional independence testing for causal discovery.
- **Nyström / random-feature approximations** (Zhang et al. 2018) bring HSIC to large n at $O(nm^2)$ cost for $m \ll n$ landmarks.

9.4.7 16.7 Code

```
# pip install hyppo
import numpy as np
from hyppo.independence import Hsic, Dcorr

rng = np.random.default_rng(0)
n = 300
x = rng.uniform(-1, 1, size=(n, 1))
y = (x.ravel() ** 2 + 0.1 * rng.normal(size=n)).reshape(-1, 1)

hsic_stat, hsic_p = Hsic().test(x, y, reps=999, workers=-1)
dcor_stat, dcor_p = Dcorr().test(x, y, reps=999, workers=-1)
print(f"HSIC = {hsic_stat:.4f}  p = {hsic_p:.3g}")
print(f"dCor = {dcor_stat:.4f}  p = {dcor_p:.3g}")
```

9.4.8 16.8 Kernel bandwidth selection

The **median heuristic** is the default: set σ to the median Euclidean distance between sample pairs. Reasonable for most uses; can be tuned by cross-validating the HSIC test power on permuted data when stakes are high.

9.4.9 16.9 Pitfalls

- **Kernel bandwidth choice** is consequential; results can vary materially across reasonable bandwidths.
- **Power depends on whether the kernel is characteristic** on the marginal supports. For bounded data, RBF and Laplace are safe; polynomial kernels are not characteristic in general.
- $O(n^2)$ **memory** in the standard estimator; use Nyström for $n > 10^5$.
- **CKA's interpretability for neural representations** has been challenged (Davari et al. 2023) — it can be biased toward dimensions with large eigenvalues.

9.4.10 16.10 When to use

General-purpose independence testing; non-linear feature selection (HSIC Lasso); comparing learned representations (CKA); structured-output learning; anywhere you would reach for dCor.

9.5 17. Chatterjee's Rank Correlation ξ

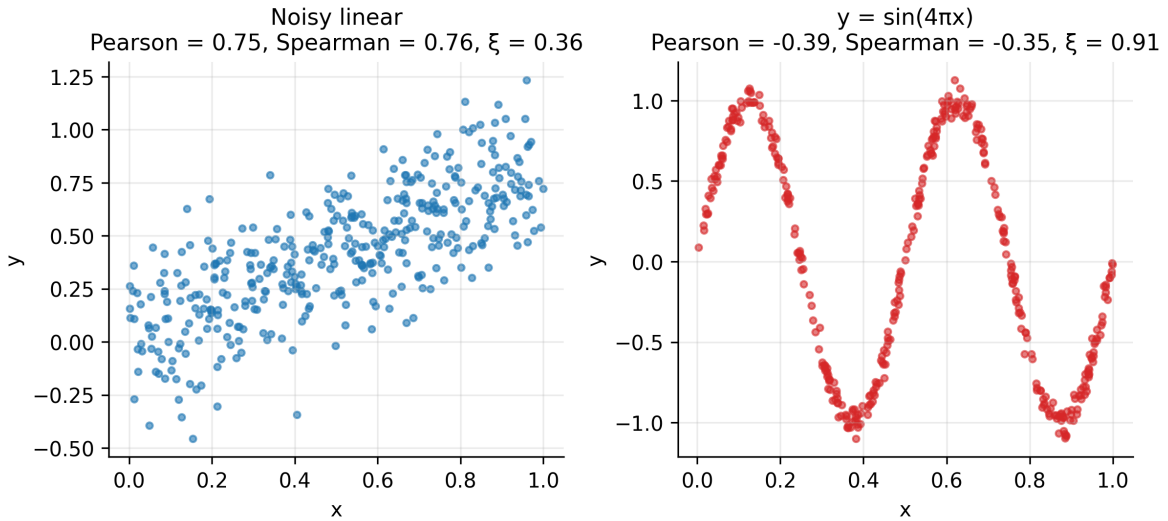


Figure 6: Chatterjee's ξ on two cases. Left: a noisy linear trend; Pearson, Spearman, and ξ all positive. Right: $y = \sin(4\pi x)$, where Pearson and Spearman both vanish but ξ approaches its upper bound, signaling that Y is a (deterministic) function of X .

9.5.1 17.1 Definition

Chatterjee (JASA 2020). Reorder the data so that $x_{(1)} \leq x_{(2)} \leq \dots \leq x_{(n)}$, with corresponding $y_{(1)}, \dots, y_{(n)}$. Let r_i denote the rank of $y_{(i)}$ in $\{1, \dots, n\}$. Then (continuous, no ties):

$$\xi_n(X, Y) = 1 - \frac{3 \sum_{i=1}^{n-1} |r_{i+1} - r_i|}{n^2 - 1}.$$

With ties on Y , the corrected form is

$$\xi_n = 1 - \frac{n \sum_{i=1}^{n-1} |r_{i+1} - r_i|}{2 \sum_{i=1}^n \ell_i (n - \ell_i)},$$

where ℓ_i is the number of j with $y_{(j)} \geq y_{(i)}$.

9.5.2 17.2 Intuition

Sort observations by X . If Y is a (possibly non-monotonic) **function** of X , then neighbouring Y -ranks r_{i+1}, r_i are close to each other and the sum of jumps $\sum |r_{i+1} - r_i|$ is small. If Y is independent of X , neighbouring Y -ranks are roughly uniform on $\{1, \dots, n\}$ and the sum of jumps is large (of order $n^2/3$).

9.5.3 17.3 Theoretical headlines

- **Population version** (continuous, no ties):

$$\xi(X, Y) = \frac{\int \text{Var}\{\mathbb{E}[\mathbb{1}\{Y \geq t\} \mid X]\} d\mu(t)}{\int \text{Var}\{\mathbb{1}\{Y \geq t\}\} d\mu(t)} \in [0, 1].$$

- $\xi(X, Y) = 0$ iff $X \perp Y$.
- $\xi(X, Y) = 1$ iff Y is a *measurable function* of X .
- ξ is **asymmetric**: $\xi(X, Y) \neq \xi(Y, X)$ in general. It measures the extent to which Y is a function of X , not the other way around. **Different from any classical correlation.**
- **Closed-form asymptotic null**: under independence with continuous distributions,

$$\sqrt{n} \xi_n \rightarrow_d \mathcal{N}(0, 2/5).$$

This enables closed-form p -values without permutations — unique among the modern "any-shape" measures.

9.5.4 17.4 Why "asymmetric" is a feature

Suppose $Y = f(X)$ for some many-to-one function f (e.g. $Y = X^2$). Knowing X determines Y — so $\xi(X, Y) \rightarrow 1$. But knowing Y leaves X undetermined (each Y corresponds to two X 's) — so $\xi(Y, X) < 1$. This asymmetry is precisely what makes ξ a *functional dependence detector*, suitable as a screening tool in causal discovery.

9.5.5 17.5 Properties

- Range: population $\xi \in [0, 1]$; the asymptotic finite-sample range is $[-1/2 + o(1), 1]$ (Chatterjee 2020 / 2021). Negative values reflect sampling noise around zero under independence and are not interpreted as anti-correlation.
- Invariant under strictly increasing transformations of X or Y separately.
- Lower power than dCor/HSIC for some smooth alternatives (linear) but uniquely capable of detecting **functional non-monotonic** relationships and providing a closed-form null.

9.5.6 17.6 Variants and extensions

- **Conditional Chatterjee** (Azadkia & Chatterjee, Annals of Statistics 2021): $\xi(Y, Z \mid X)$, enabling variable selection in regression settings.
- **Multi-variable** (Chatterjee 2022; Lin & Han 2023): generalisation to $\xi(\mathbf{X}, Y)$ with $\mathbf{X} \in \mathbb{R}^p$.
- **Lin–Han modification**: improved finite-sample properties relative to the original conservative estimator.

9.5.7 17.7 Code

```
# pip install xicor
from xicor.xicor import Xi
import numpy as np

rng = np.random.default_rng(0)
n = 1000
x = rng.uniform(0, 1, n)
y_func = np.sin(10 * x) + 0.05 * rng.normal(size=n)
```

```

y_indep = rng.normal(size=n)

print("Y = sin(10X) + noise:")
xi = Xi(x, y_func); print(f" xi(X,Y) = {xi.correlation:.3f} p = {xi.pval_asymptotic(ties=False):.3g}")
xi_rev = Xi(y_func, x); print(f" xi(Y,X) = {xi_rev.correlation:.3f} (asymmetric!)")

print("\nIndependent:")
xi = Xi(x, y_indep); print(f" xi(X,Y) = {xi.correlation:.3f} p = {xi.pval_asymptotic(ties=False):.3g}")

```

9.5.8 17.8 Pitfalls

- **Asymmetric** — be deliberate about which variable plays which role. Convention: $\xi(X, Y)$ measures "Y is a function of X".
- **Slower convergence** than parametric correlations; needs reasonably large n to detect weak dependence.
- **Original estimator is conservative**; the Lin–Han modification has better finite-sample power.
- **Asymptotic null assumes continuity** with no ties; for heavy ties use the tied-data correction or permutation.

9.5.9 17.9 When to use

Functional-dependence detection (especially non-monotonic); screening for unusual relationships when you cannot afford permutation tests; replacement for Spearman / Kendall when you suspect non-monotonic structure; causal-discovery-style screening for "is Y a function of X ?" hypotheses.

10 Part IV — Model-Agnostic Feature Importance

The measures up to here summarise the *joint distribution* of variables. Starting now, the question changes: we no longer ask "is X associated with Y ?" but "**how much does this trained model rely on X to make its predictions?**" — a related but importantly different question. A feature with high data-distribution dependence on the target may be ignored by the model (perhaps because the model uses a stronger proxy); a feature with no marginal association may matter to the model through interactions; a feature that adds no out-of-sample value may still acquire high in-sample importance under any of these methods. Read this part with the model-vs-data distinction always in mind.

10.1 18. SHAP (SHapley Additive exPlanations)

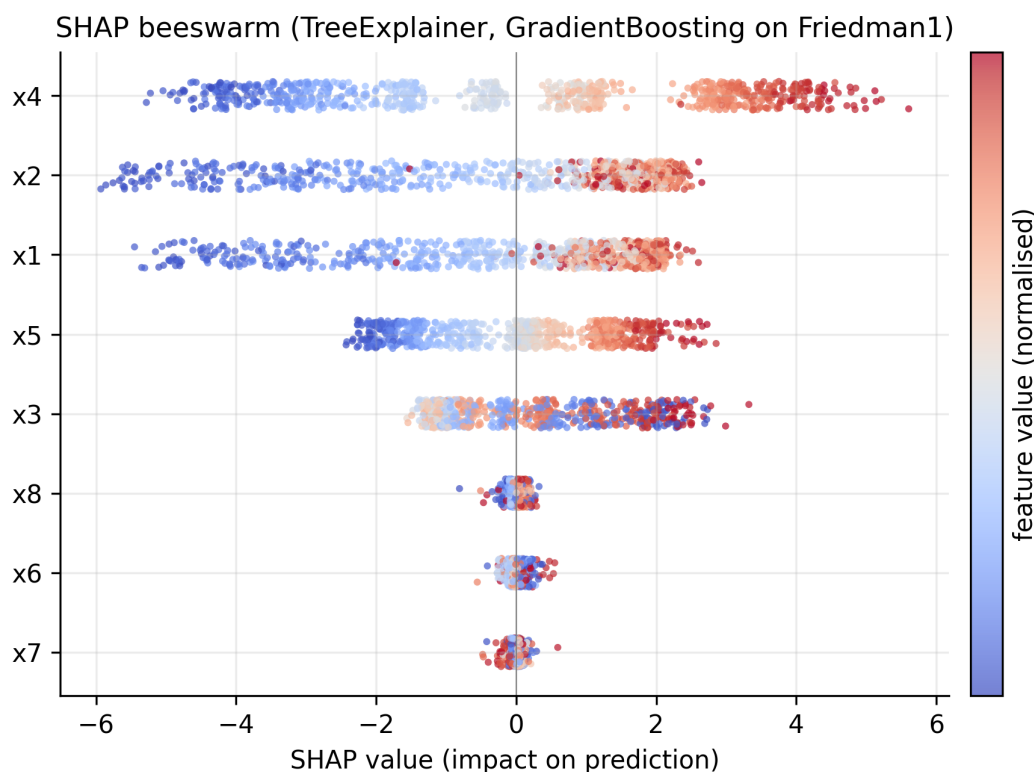


Figure 7: SHAP beeswarm summary for a Gradient Boosting regressor on the Friedman 1 dataset. Each dot is one observation's SHAP value for a given feature; horizontal position is the impact on the prediction; color encodes the (normalised) feature value. Features are ordered by mean $|\text{SHAP}|$.

10.1.1 18.1 Setup

Lundberg & Lee (NeurIPS 2017) unified a decade of additive feature-attribution methods (LIME, DeepLIFT, Integrated Gradients, Tree-SHAP, Shapley sampling) under the framework of **Shapley values** from cooperative game theory (P9). Treat features as players in a game where the "payout" is the model's prediction; the Shapley value of each feature is its fair share of the prediction relative to a baseline.

10.1.2 18.2 Formal definition

For a model f , an input $\mathbf{x} = (x_1, \dots, x_p)$, and a value function $v : 2^{\{1, \dots, p\}} \rightarrow \mathbb{R}$ with $v(S)$ representing the model's prediction when only features in S are "known":

$$\phi_i(\mathbf{x}) = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|! (p - |S| - 1)!}{p!} [v(S \cup \{i\}) - v(S)].$$

The Shapley values satisfy four axioms (efficiency, symmetry, dummy, additivity — see P9) and decompose the prediction exactly:

$$f(\mathbf{x}) = \mathbb{E}[f(\mathbf{X})] + \sum_{i=1}^p \phi_i(\mathbf{x}).$$

10.1.3 18.3 The value function: interventional vs observational

The choice of value function $v(S)$ — what it means for "features in S are known" — is the most subtle and consequential decision in SHAP.

- **Interventional** (a.k.a. "marginal") SHAP:

$$v(S) = \mathbb{E}_{X_{\bar{S}}} [f(\mathbf{x}_S, X_{\bar{S}})]$$

where $\bar{S} = N \setminus S$. Features outside S are drawn from their **marginal** distribution, breaking any correlation with features in S .

- **Observational** (a.k.a. "conditional") SHAP:

$$v(S) = \mathbb{E}[f(\mathbf{X}) \mid X_S = \mathbf{x}_S]$$

Features outside S are drawn from their **conditional** distribution given the observed values.

The two differ whenever features are correlated. Janzing, Minorics & Blöbaum (AISTATS 2020) argued the **interventional** formulation is the right one for *causal/explanatory* questions; the **conditional** form reflects predictive associations under the data distribution but can attribute importance to features the model never actually uses (if a feature is correlated with one the model does use, it inherits credit).

Practical implication: TreeSHAP's two modes (`feature_perturbation="tree_path_dependent"` $\approx$ observational; `feature_perturbation="interventional"` $\approx$ interventional) can give meaningfully different rankings. Pick deliberately.

10.1.4 18.4 Estimators

- **KernelSHAP** (model-agnostic): weighted linear regression on sampled coalitions. The weights $w(|S|) = \frac{p-1}{\binom{p}{|S|}|S|(p-|S|)}$ are derived to recover Shapley values exactly. Slow ($O(2^p)$ exact, $O(K \cdot \text{infer})$ sampled).
- **TreeSHAP** (Lundberg et al. 2018): polynomial-time **exact** SHAP for tree ensembles *in path-dependent mode*, with complexity $O(TLD^2)$ where T is trees, L leaves per tree, D depth. The *interventional* mode (`feature_perturbation="interventional"` with a background dataset of size R) is a Monte Carlo estimate that converges to interventional Shapley values as $R \rightarrow \infty$, at complexity $O(TLDR)$; it is not "exact" in the same sense as path-dependent. The practical workhorse for XGBoost / LightGBM / sklearn forests.
- **DeepSHAP** / **GradientSHAP**: approximations for neural networks; relate to DeepLIFT and Integrated Gradients respectively.
- **LinearSHAP**: closed form for linear models; for a linear model $f(\mathbf{x}) = \beta_0 + \sum_i \beta_i x_i$, $\phi_i(\mathbf{x}) = \beta_i(x_i - \mathbb{E}[X_i])$ under the interventional value function.

10.1.5 18.5 Global aggregation

Per-prediction $\phi_i(\mathbf{x})$ can be aggregated to global importance:

- **Mean absolute SHAP:** $\bar{\phi}_i = \frac{1}{n} \sum_j |\phi_i(\mathbf{x}_j)|$.
- **Mean squared SHAP:** $\frac{1}{n} \sum_j \phi_i(\mathbf{x}_j)^2$.
- **SHAP summary plot (beeswarm):** arguably the most informative single visualisation in modern ML interpretability — one row per feature, one dot per training example, x-position = SHAP value, colour = feature value. Reveals direction of effect, heterogeneity, interactions.

10.1.6 18.6 Code

```
# pip install shap xgboost
import shap
import xgboost as xgb
import numpy as np

rng = np.random.default_rng(0)
n, p = 1000, 5
X = rng.normal(size=(n, p))
y = 2 * X[:, 0] + X[:, 1] ** 2 - X[:, 2] * X[:, 3] + 0.5 * rng.normal(size=n)

model = xgb.XGBRegressor(n_estimators=200, max_depth=4).fit(X, y)

explainer = shap.TreeExplainer(model, feature_perturbation="interventional", data=X)
shap_values = explainer.shap_values(X)

# Per-prediction
print("First prediction breakdown:")
print(f"  baseline      = {explainer.expected_value:.3f}")
print(f"  sum of SHAPs   = {shap_values[0].sum():.3f}")
print(f"  prediction     = {model.predict(X[:1])[0]:.3f}")

# Global
global_importance = np.abs(shap_values).mean(axis=0)
for i, imp in enumerate(global_importance):
    print(f"  feature {i}: {imp:.4f}")
```

10.1.7 18.7 Pitfalls

- **Correlated features cause unstable attributions.** Shapley values split credit between correlated features in ways that are hard to predict; small data perturbations can move the split substantially. Group correlated features and report group-level Shapley values (Aas et al. 2021) for stability.
- **Interventional SHAP can query the model at unrealistic points** (e.g. age = 5, salary = \$200,000); the model may behave erratically there, and that behaviour shows up in the SHAP values.
- **SHAP explains the model, not the world.** A feature with high SHAP is one the model uses; it is not necessarily causally relevant.
- **Computational cost:** KernelSHAP is $O(2^p)$ exact, prohibitive beyond ~10 features; TreeSHAP is polynomial but non-trivial for deep deep ensembles (hundreds of trees, depth 10+).
- **Sampling variance** — KernelSHAP attributions are noisy estimates; reproducibility requires fixed seeds and enough samples (typically ≥ 1000).

10.1.8 18.8 When to use

Per-prediction explanations in regulated industries (credit, healthcare, insurance); global feature ranking when you have a tree ensemble (TreeSHAP is fast and exact); communicating model behaviour to stakeholders via summary plots; identifying buggy features (a feature that "should" be irrelevant showing up with high SHAP magnitude is a strong leakage signal).

10.2 19. Permutation Feature Importance (PFI)

10.2.1 19.1 Algorithm

1. Compute the baseline model performance $s_0 = \text{score}(f, \mathcal{D})$ on a held-out dataset $\mathcal{D}$.
2. For each feature $i \in \{1, \dots, p\}$:
 - Shuffle the column X_i across rows of $\mathcal{D}$, breaking its relationship with Y and with the other features.
 - Re-score the model on the permuted data: s_i .
3. PFI of feature i is the change in performance. In the *score* convention (higher = better, e.g. R^2 , accuracy — the default in `sklearn.inspection.permutation_importance`), $\text{PFI}_i = s_0 - s_i$; a positive value means the feature helped. In the *loss* convention (lower = better, e.g. MSE), $\text{PFI}_i = s_i - s_0$. Important features are positive in both conventions; noise features can show small negative values from permutation variance and are not a bug.
4. Repeat across multiple permutations; report mean and standard deviation.

10.2.2 19.2 Intuition

"How much worse does the model become if I destroy this feature's signal but keep its marginal distribution intact?"

10.2.3 19.3 Properties

- **Model-agnostic** — works for any predictor.
- Reflects **the model's use of the feature**, not the joint distribution of X_i, Y .
- Computed on **test/validation data** = generalisation-relevant importance.
- Computed on **training data** $\approx$ memorisation; a feature can show high training PFI even if it adds no out-of-sample value. Always use held-out data unless you have a specific reason not to.

10.2.4 19.4 The correlated-features trap

Suppose X_1, X_2 are perfectly correlated and the model uses X_1 exclusively. PFI of X_1 is large; PFI of X_2 is near zero. Retrain with a different random seed: the model may now use X_2 instead, and the PFI flips. **Individual PFI under feature correlation is fragile**; group PFI (permute correlated blocks together) is stable but less granular.

Conditional PFI (Strobl et al. 2008): permute X_i *within strata* defined by the other features. Removes the within-strata signal, preserves the joint distribution. Hard to estimate in high dimensions.

10.2.5 19.5 The extrapolation problem

Permuting X_i produces feature combinations the model has never seen. This can push the model to query unrealistic regions of input space; bad extrapolations cost score; PFI is inflated. Hooker et al. (Statistics and Computing 2021) document this carefully and propose:

- **LOCO (Leave-One-Covariate-Out)**: refit the model without X_i , compare scores. Slow but principled.
- **Conditional permutation**: as above.
- **Block-permutation** within correlated groups.

10.2.6 19.6 Code

```
import numpy as np
from sklearn.ensemble import RandomForestRegressor
from sklearn.inspection import permutation_importance
from sklearn.model_selection import train_test_split
```

```

rng = np.random.default_rng(0)
n, p = 1000, 6
X = rng.normal(size=(n, p))
y = 2 * X[:, 0] + X[:, 1] ** 2 + 0.5 * rng.normal(size=n)
# X[:, 5] is pure noise; X[:, 2:5] are noise but correlated with X[:, 0]
X[:, 2:5] = X[:, [0]] + 0.5 * rng.normal(size=(n, 3))

Xtr, Xte, ytr, yte = train_test_split(X, y, random_state=0)
model = RandomForestRegressor(n_estimators=200, random_state=0).fit(Xtr, ytr)

result = permutation_importance(model, Xte, yte, n_repeats=30, random_state=0)
for i in range(p):
    print(f"feature {i}: PFI = {result.importances_mean[i]:.4f} ± {result.importances_std[i]:.4f}")

```

10.2.7 19.7 Pitfalls

- **Use a held-out set.** Training-set PFI is uninformative for generalisation.
- **Average over many permutations** — single-shuffle PFI is noisy.
- **Don't use PFI as a causal measure.** It is not.
- **Correlated features attenuate each other's PFI** — interpret with care, prefer group importance.
- **PFI assumes the score function is sensitive** to the feature being shuffled. For classification with severely imbalanced classes, use `balanced_accuracy` or `roc_auc` rather than accuracy.

10.2.8 19.8 When to use

Quick model-agnostic global importance rankings; sanity checks on suspected leakage; default importance in `sklearn.inspection.permutation_importance`.

10.3 20. LIME (Local Interpretable Model-agnostic Explanations)

10.3.1 20.1 Setup

Ribeiro, Singh & Guestrin (KDD 2016). The conceptual precursor to SHAP and still widely used, particularly for image and text models.

10.3.2 20.2 Algorithm

For a specific input $\mathbf{x}_0$ to explain:

1. **Perturb $\mathbf{x}_0$** to generate samples $\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}$:
 - **Tabular:** sample around $\mathbf{x}_0$ from a fitted distribution (often Gaussian centred at training mean).
 - **Text:** randomly drop words from $\mathbf{x}_0$.
 - **Image:** turn off (set to mean colour) random superpixels from $\mathbf{x}_0$.
2. **Predict** with the black-box: $y^{(k)} = f(\mathbf{x}^{(k)})$.
3. **Weight** by proximity to $\mathbf{x}_0$: $w^{(k)} = \exp(-D(\mathbf{x}_0, \mathbf{x}^{(k)})^2 / \sigma^2)$.
4. **Fit a sparse linear surrogate g** on the interpretable representation $z^{(k)}$ (binary presence/absence of words or superpixels) with weights $w^{(k)}$, minimising:

$$\sum_{k=1}^m w^{(k)} (y^{(k)} - g(z^{(k)}))^2 + \Omega(g),$$

where Ω is a sparsity penalty. Ribeiro et al. (2016) use Ω to *select* the top K features via a LASSO path (the so-called K-LASSO step), then fit the final surrogate as a weighted *ridge* regression on those K features — not as a LASSO. The `lime` package's default `feature_selection="auto"` switches between LASSO-path and highest-weight selection depending on K .

5. **Return** the linear coefficients of g as the LIME explanation.

10.3.3 20.3 Intuition

LIME approximates the (potentially highly non-linear) decision boundary of f **locally** by a sparse linear model, on the theory that a sufficiently small neighbourhood of any prediction is approximately linear. The "interpretable representation" — superpixels, words — is a deliberate departure from the model's raw input space, designed for human readability.

10.3.4 20.4 Code

```
# pip install lime scikit-learn
from lime.lime_tabular import LimeTabularExplainer
import numpy as np
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import load_breast_cancer

data = load_breast_cancer()
X, y = data.data, data.target
model = RandomForestClassifier(n_estimators=200, random_state=0).fit(X, y)

explainer = LimeTabularExplainer(X, feature_names=data.feature_names,
                                 class_names=data.target_names, random_state=0)
exp = explainer.explain_instance(X[0], model.predict_proba, num_features=5)
for feat, weight in exp.as_list():
    print(f" {feat}: {weight:+.4f}")
```

10.3.5 20.5 Properties

- **Local:** explains one prediction at a time. Aggregating to global importance is possible (lime's `submodular_pick`) but not the primary use case.
- **Sparse:** LASSO returns a short list of "important" features for this prediction.
- **Model-agnostic:** only requires black-box query access.
- **Image and text** are LIME's signature modalities; for tabular data SHAP is generally preferred.

10.3.6 20.6 Pitfalls

- **Instability.** The neighbourhood is random; running LIME twice on the same prediction can give materially different explanations. Multiple runs and averaging help; the `slime` extension provides stability guarantees.
- **Sampling distribution matters.** For tabular data, the default Gaussian perturbation around $\mathbf{x}_0$ produces unrealistic combinations.
- **Locality is a hyperparameter.** The kernel bandwidth σ controls "how local"; explanations vary substantially with this choice. `LimeTabularExplainer` defaults to $\sigma = 0.75\sqrt{\text{num features}}$ and, more consequentially, to `discretize_continuous=True` — continuous features are quantile-binned before sampling, which materially changes the explanation. Both defaults should be set explicitly when reproducibility matters.
- **Not consistent with the Shapley axioms.** SHAP can be viewed as a principled generalisation of LIME; if axiomatic guarantees matter, prefer SHAP.
- **Image LIME superpixels** are themselves a model choice (SLIC, Felzenszwalb); the chosen segmentation can bias the explanation toward or against certain image regions.

10.3.7 20.7 When to use

Image and text explanations (superpixel and word-drop perturbations are intuitive); quick local interpretability prototype; settings where a single black-box prediction needs a human-readable justification; when SHAP is computationally prohibitive for the model.

10.4 21. Partial Dependence (PDP) and Individual Conditional Expectation (ICE)

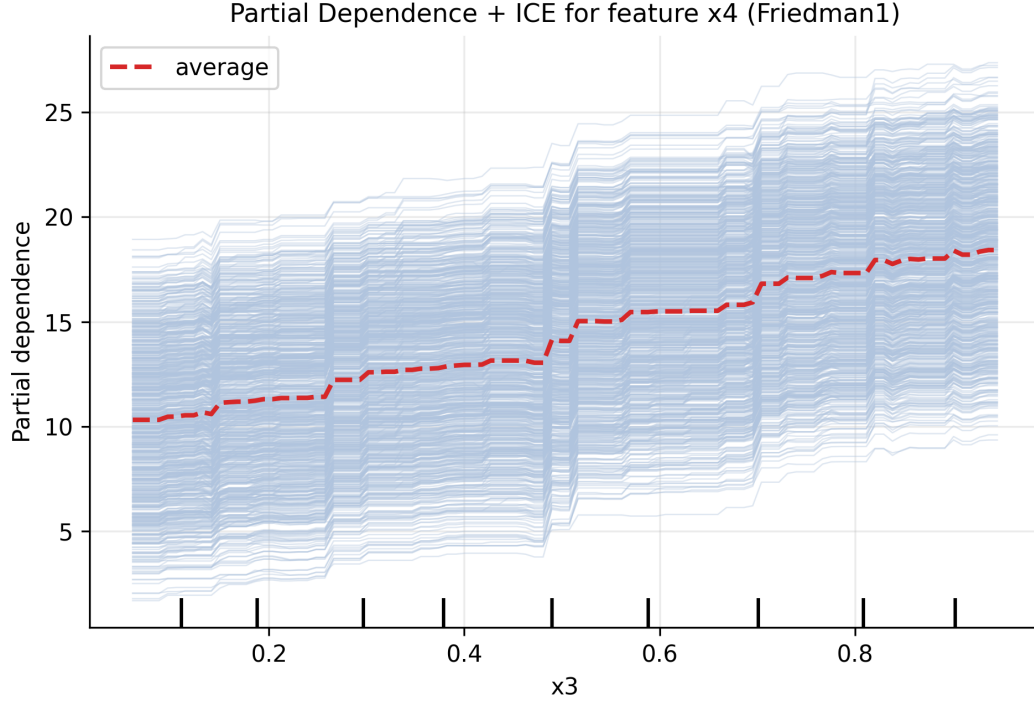


Figure 8: Partial Dependence (red) and Individual Conditional Expectation lines (light blue) for feature x_4 of a Gradient Boosting regressor on Friedman 1. The PDP is the average over all ICE curves; large ICE dispersion would signal feature interactions hidden by the PDP.

10.4.1 21.1 Partial dependence

Friedman (Annals of Statistics 2001). For a feature (or feature subset) X_S :

$$\text{PDP}_S(x_S) = \mathbb{E}_{X_{\bar{S}}} [f(x_S, X_{\bar{S}})] \approx \frac{1}{n} \sum_{i=1}^n f(x_S, x_{\bar{S},i}).$$

A PDP plots PDP_S as a function of x_S (one curve for one feature; a 3D surface or heatmap for two).

Intuition: the marginal effect of X_S on predictions, averaged over the empirical distribution of the other features.

10.4.2 21.2 Individual Conditional Expectation (ICE)

Goldstein et al. (JCGS 2015). PDP without the averaging: one curve per observation,

$$\text{ICE}_S^{(i)}(x_S) = f(x_S, x_{\bar{S},i}).$$

An ICE plot draws all n curves overlaid; the PDP is their pointwise mean. **ICE exposes interactions and heterogeneity that the PDP averages away** — if the curves fan out, the effect of X_S depends on the values of the other features.

Centred ICE (c-ICE): subtract each curve's value at the leftmost grid point, so all curves start at zero. Makes heterogeneity in *changes* visible without the distraction of different intercepts.

10.4.3 21.3 Accumulated Local Effects (ALE)

Apley & Zhu (JRSS-B 2020). PDP evaluates the model at $(x_S, x_{\bar{S},i})$ combinations that may be off the data manifold — the same extrapolation pathology that afflicts interventional SHAP and PFI. ALE plots use **conditional** expectations integrated over local windows of x_S :

$$\text{ALE}_S(x_S) = \int_{\min(x_S)}^{x_S} \mathbb{E} \left[\frac{\partial f(u, X_{\bar{S}})}{\partial u} \mid X_S = u \right] du - \text{const.}$$

Empirically: bin X_S , compute the average within-bin change in f (conditioning on X_S in the bin, not marginalising), accumulate, recentre.

Use ALE instead of PDP when features are correlated. ALE never queries the model at off-manifold points.

10.4.4 21.4 Code

```
import numpy as np
from sklearn.ensemble import RandomForestRegressor
from sklearn.inspection import PartialDependenceDisplay
import matplotlib.pyplot as plt

rng = np.random.default_rng(0)
n = 500
X = rng.uniform(-2, 2, size=(n, 3))
y = X[:, 0] ** 2 - X[:, 1] + X[:, 0] * X[:, 2] + 0.2 * rng.normal(size=n)

model = RandomForestRegressor(n_estimators=200, random_state=0).fit(X, y)

# PDP + ICE for feature 0
fig, ax = plt.subplots(figsize=(6, 4))
PartialDependenceDisplay.from_estimator(model, X, features=[0], kind="both", ax=ax)
plt.tight_layout()
# For ALE use the PyALE package: ale_plot(model, X, features=[0])
```

10.4.5 21.5 Properties

- **PDP averages out interactions** — a flat PDP does *not* mean a feature has no effect (it may have heterogeneous effects that cancel on average).
- **ICE reveals interactions** but is visually noisy for large n — subsample.
- **ALE avoids extrapolation** at the cost of slight bias when window sizes are too large.

10.4.6 21.6 Pitfalls

- **Extrapolation** for PDP: the marginalisation $\mathbb{E}_{X_{\bar{S}}}$ uses the unconditional distribution, generating feature combinations the model never trained on.
- **Averaging hides interactions** — always look at the ICE alongside.
- **Computational cost** of PDP for two features simultaneously is $O(n \cdot g^2)$ where g is grid resolution.
- **PDP is not a causal effect** — it shows how the model's prediction *would respond* to changing X_S holding the conditional distribution of other features fixed-at-marginal, which is not generally the same as the causal effect of intervening on X_S .

10.4.7 21.7 When to use

Whenever you want to *see* (not just rank) a feature's effect on a model. Especially useful for non-linear models where coefficients are unavailable. **ALE for correlated features.** PDP + ICE for an initial diagnostic. Two-feature PDP heatmaps to visualise pairwise interactions.

11 Part V — Tree-Specific and Embedded Metrics

Tree-based models (decision trees, random forests, gradient boosting) compute "importance" as a by-product of training. These are cheap, ubiquitous, and frequently misinterpreted — each carries well-documented biases that make them poor substitutes for the model-agnostic measures in Part IV. The same is true of the regression coefficients in penalised linear models, which double as importance scores by design.

11.1 22. Gini Importance / Mean Decrease in Impurity (MDI)

11.1.1 22.1 Definition

For a single decision tree, the importance of feature i is the total impurity reduction at all internal nodes that split on i , weighted by the fraction of training samples reaching each node:

$$\text{MDI}_i = \sum_{t \in T_i} \frac{n_t}{n} \Delta i(t),$$

where T_i is the set of internal nodes splitting on feature i , n_t is the number of training samples at node t , and the impurity decrease is

$$\Delta i(t) = i(t) - \frac{n_{t_L}}{n_t} i(t_L) - \frac{n_{t_R}}{n_t} i(t_R).$$

For classification $i(t)$ is the **Gini index** $1 - \sum_k p_{tk}^2$ or **entropy** $-\sum_k p_{tk} \log p_{tk}$; for regression it is the variance $\frac{1}{n_t} \sum_{x \in t} (y - \bar{y}_t)^2$.

For an ensemble (random forest, extra trees), average MDI across trees.

11.1.2 22.2 Intuition

"How much does this feature reduce impurity, summed over all the times the greedy algorithm chose to split on it, weighted by how many training samples were affected?"

11.1.3 22.3 Why MDI is biased — three reasons

1. **Cardinality bias.** Features with more possible split points (continuous, or categorical with many levels) have more chances to be selected greedily and tend to acquire larger MDI even when uninformative. Strobl et al. (BMC Bioinformatics 2007) is the canonical demonstration.
2. **Computed on training data.** MDI reflects how the trees were built, not whether splits generalise. A noise feature can acquire non-trivial MDI through overfitting in deep trees.
3. **Correlated features dilute each other.** Two features with identical signal split the MDI; if you remove one, the other's MDI roughly doubles.

11.1.4 22.4 Empirical demonstration of cardinality bias

```
import numpy as np
from sklearn.ensemble import RandomForestRegressor

rng = np.random.default_rng(0)
n = 2000
# Pure noise: a binary feature, a 10-category, and a continuous, all unrelated to y
X = np.column_stack([
    rng.integers(0, 2, n).astype(float),      # 2 levels
    rng.integers(0, 10, n).astype(float),     # 10 distinct ordered values (sklearn treats as ordered numeric,
    rng.normal(size=n),                       # continuous (many split points)
])
y = rng.normal(size=n) # pure noise

model = RandomForestRegressor(n_estimators=500, random_state=0).fit(X, y)
print("Pure noise features; ideal MDI would be ~equal (or zero):")
for i, imp in enumerate(model.feature_importances_):
```

```
print(f" feature {i}: MDI = {imp:.4f}")
# The continuous feature will dominate, despite all being equally uninformative.
```

11.1.5 22.5 Unbiased alternatives

- **Conditional inference forests** (Hothorn et al. 2006; `party::cforest` in R) use significance-test-based splitting that removes the cardinality bias.
- **Honest forests / unbiased MDI** (Li, Wang, Singh 2019): randomly subsample data for splitting vs leaf-value estimation, removing the in-sample-optimism bias.
- **MDA (§23) and PFI (§19) on held-out data** are usually preferred.

11.1.6 22.6 When to use

A cheap first look at what a forest is doing. **Always cross-check with held-out PFI before reporting.** Never use MDI alone in a setting that affects decisions.

11.2 23. Mean Decrease in Accuracy (MDA)

11.2.1 23.1 Algorithm

The original Breiman (2001) random forest importance:

1. For each tree t in the forest, evaluate prediction accuracy on its OOB sample O_t .
2. Permute feature i across O_t ; re-evaluate accuracy.
3. The per-tree importance is the accuracy drop. MDA of feature i is the average drop across trees.

11.2.2 23.2 Relationship to PFI

MDA is permutation feature importance evaluated within each tree's OOB sample. Statistically more efficient than computing a separate held-out PFI, because OOB samples are "free" — each tree saw a different bootstrap subset, and the rest serves as its private test set.

11.2.3 23.3 Properties

- Less biased than MDI (uses held-out data via OOB).
- Inherits all PFI pitfalls: correlated-feature instability, extrapolation, retraining sensitivity.
- Specific to bagging-based ensembles (random forests, extra trees); not natively defined in boosting.

11.2.4 23.4 Code

```
import numpy as np
from sklearn.ensemble import RandomForestRegressor
from sklearn.datasets import make_regression
from sklearn.inspection import permutation_importance

X, y = make_regression(n_samples=1000, n_features=10, n_informative=5,
                      noise=1.0, random_state=0)
model = RandomForestRegressor(n_estimators=500, oob_score=True, random_state=0).fit(X, y)

# scikit-learn does not expose Breiman-style OOB MDA directly; use
# held-out permutation importance (the closest practical proxy), or
# compute manually from model.estimators_ and their bootstrap indices.
from sklearn.model_selection import train_test_split

Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=0)
model = RandomForestRegressor(n_estimators=500, random_state=0).fit(Xtr, ytr)
```

```
# Permute on a held-out set, NOT on the training data (see section 19.3).
result = permutation_importance(model, Xte, yte, n_repeats=20, random_state=0)
for i in range(Xte.shape[1]):
    print(f"feature {i}: MDA-equiv = {result.importances_mean[i]:+.4f}")
```

11.2.5 23.5 When to use

Default importance for random forests when you don't have a separate held-out PFI run. **Prefer to MDI** in nearly all cases.

11.3 24. Gain, Split, and Cover Importance in Boosted Trees

Gradient-boosted ensembles (XGBoost, LightGBM, CatBoost) report multiple importance types:

11.3.1 24.1 Definitions

- **Gain** (a.k.a. *total gain*): sum of loss reduction attributable to splits on the feature, across all trees. Boosting analogue of MDI; typical default. For XGBoost, the per-split gain is the second-order Taylor approximation of the loss improvement,

$$\text{Gain} = \frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda} \right] - \gamma,$$

where G_*, H_* are sums of first- and second-order gradients in the partition; total gain is the sum of these over splits using the feature.

- **Split** (a.k.a. *weight* or *frequency*): number of times the feature was used as a split.
- **Cover** (a.k.a. *sample coverage*): average number of training samples flowing through splits that used the feature.
- **Total cover**: sum (not average) of cover.

In XGBoost: `importance_type` $\in$ {gain, weight, cover, total_gain, total_cover}. In LightGBM: `importance_type` $\in$ {gain, split}.

11.3.2 24.2 Differences from MDI

Boosting trees are typically shallow (the *weak learner* idea, depth $\in [3, 8]$), so the cardinality bias of MDI is attenuated. However:

- **Gain is still computed on training data** and inflates with noise overfitting under under-regularisation.
- **Split** is the most directly comparable to "frequency of use" and is preferred when features have very different resolutions of potential split points.
- **Cover** is most relevant when you care about *how much of the data* a feature influenced.

11.3.3 24.3 TreeSHAP as the modern replacement

Lundberg et al. (2018) showed that gain importance can produce misleading rankings under correlated features and proposed TreeSHAP as the principled alternative. For XGBoost / LightGBM, **prefer TreeSHAP** for any decision that matters; use gain / split / cover for quick triage.

CatBoost is somewhat different: its native "PredictionValuesChange" measures average change in predictions when the feature value changes; "LossFunctionChange" $\approx$ permutation importance, computed efficiently. Both are closer to PFI than to gain.

11.3.4 24.4 Code

```
# pip install xgboost
import xgboost as xgb
import numpy as np

rng = np.random.default_rng(0)
n, p = 1000, 6
X = rng.normal(size=(n, p))
y = 2 * X[:, 0] + X[:, 1] ** 2 + 0.3 * rng.normal(size=n)

model = xgb.XGBRegressor(n_estimators=200, max_depth=4).fit(X, y)

for itype in ["gain", "weight", "cover", "total_gain", "total_cover"]:
    scores = model.get_booster().get_score(importance_type=itype)
    print(f"\n{itype}:")
    for k, v in sorted(scores.items(), key=lambda kv: -kv[1]):
        print(f"    {k}: {v:.4f}")
```

11.3.5 24.5 When to use

- **Gain:** quick training-time importance for a fitted booster; sanity check.
- **Split:** when you care about how often a feature is consulted (e.g. for feature pruning decisions).
- **Cover:** rarely the right answer; useful for diagnosis.
- For anything decision-relevant: **TreeSHAP**.

11.4 25. LASSO and Ridge Coefficients as Importance

11.4.1 25.1 Definitions

LASSO (L_1 regularised regression; Tibshirani JRSS-B 1996):

$$\hat{\beta}_{\text{LASSO}} = \arg \min_{\beta} \frac{1}{2n} \|\mathbf{y} - \mathbf{X}\beta\|_2^2 + \lambda \|\beta\|_1.$$

Ridge (L_2 ; Hoerl & Kennard Technometrics 1970):

$$\hat{\beta}_{\text{Ridge}} = \arg \min_{\beta} \frac{1}{2n} \|\mathbf{y} - \mathbf{X}\beta\|_2^2 + \lambda \|\beta\|_2^2.$$

Elastic Net (Zou & Hastie JRSS-B 2005): combines both,

$$\hat{\beta}_{\text{EN}} = \arg \min_{\beta} \frac{1}{2n} \|\mathbf{y} - \mathbf{X}\beta\|_2^2 + \lambda_1 \|\beta\|_1 + \lambda_2 \|\beta\|_2^2.$$

11.4.2 25.2 The geometry of L_1 vs L_2

The L_1 unit ball has **corners on the axes**; the L_2 unit ball is round. As the OLS solution shrinks under penalty, the contact point with the L_1 ball tends to lie on a corner (a coordinate plane), making some coefficients exactly zero. The L_2 contact point is generically interior, so all coefficients shrink toward zero but none reach it. This geometric intuition is the cleanest explanation of why LASSO performs feature selection and Ridge does not.

11.4.3 25.3 Closed-form Ridge solution

$$\hat{\beta}_{\text{Ridge}} = (\mathbf{X}^\top \mathbf{X} + n\lambda I)^{-1} \mathbf{X}^\top \mathbf{y}.$$

Always invertible (adding $n\lambda I$ regularises the Gram matrix). LASSO has no closed form; coordinate descent (Friedman et al. 2007) is the dominant solver and what **glmnet** and **sklearn** use under the hood.

11.4.4 25.4 Critical preprocessing

- **Standardise features** (mean 0, unit variance). Otherwise the penalty applies unequally — large-scale features look spuriously unimportant.
- **Tune λ by cross-validation** (LassoCV, RidgeCV, ElasticNetCV). Importance rankings depend on λ .
- **Centre y** (or include an unpenalised intercept; the standard `sklearn` and `glmnet` interfaces do this automatically).

11.4.5 25.5 Stability under correlated features

LASSO is *known* to be unstable under multicollinearity — it tends to pick one feature from each correlated group and zero out the rest, with the choice sensitive to the random sample. Three responses:

- **Elastic Net**: the L_2 component groups correlated features (they survive together or perish together), recommended whenever you expect correlated predictors.
- **Stability Selection** (Meinshausen & Bühlmann JRSS-B 2010): rerun LASSO on bootstrap subsamples; report features selected in a high fraction of runs.
- **Adaptive LASSO** (Zou JASA 2006): reweight the penalty by an initial estimate ($w_i \propto 1/|\tilde{\beta}_i|^\gamma$), giving oracle consistency properties.

11.4.6 25.6 Post-selection inference

LASSO coefficients are **biased toward zero** by construction; the usual OLS t -tests on the selected features are *invalid* (they ignore the selection step). Two principled solutions:

- **LASSO + OLS refit**: use LASSO to select features, then refit by OLS on selected features for unbiased estimates. Simple, widely used, but inferentially imperfect.
- **Selective inference** (Lee et al. 2016; Tibshirani et al. 2016): build confidence intervals that condition on the selection event. Implemented in the R `selectiveInference` package; rare in Python.

11.4.7 25.7 Code

```
import numpy as np
from sklearn.linear_model import LassoCV, RidgeCV, ElasticNetCV
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

rng = np.random.default_rng(0)
n, p = 200, 20
X = rng.normal(size=(n, p))
true_beta = np.zeros(p); true_beta[:5] = [2, -1.5, 1, -0.5, 0.8] # only first 5 matter
y = X @ true_beta + 0.5 * rng.normal(size=n)

# Standardise + cross-validated LASSO
lasso = make_pipeline(StandardScaler(), LassoCV(cv=5, random_state=0)).fit(X, y)
beta = lasso[-1].coef_

print("LASSO coefficients (showing non-zero):")
for i, b in enumerate(beta):
    marker = " <- true signal" if i < 5 else ""
    if abs(b) > 1e-6:
        print(f"    beta[{i}] = {b:+.3f}{marker}")
print(f"\nNumber of selected features: {(np.abs(beta) > 1e-6).sum()}")
print(f"Chosen lambda: {lasso[-1].alpha_:.4f}")
```

11.4.8 25.8 Other uses

- **Logistic LASSO** for classification — same penalty, coefficients on the log-odds scale.
- **Group LASSO** (Yuan & Lin 2006) for selecting whole groups of features (e.g. all dummies for a categorical variable).

- **Sparse logistic regression** is the standard tool for high-dimensional binary screening (genomics, NLP).

11.4.9 25.9 Pitfalls

- **Coefficients are only meaningful after standardisation**; otherwise their magnitudes mix penalty effects with scale.
- **Sign is interpretable**; magnitude is comparable across features *only* after standardisation.
- **Highly biased toward zero by construction** — for unbiased estimates use the LASSO selection as a *first step* followed by OLS on the selected features.
- **No interactions** in vanilla LASSO; include explicit interaction terms or use tree-based methods.
- **High-correlation pairs** behave unpredictably without Elastic Net or Stability Selection.

11.4.10 25.10 When to use

Linear (or generalised linear) modelling with many features; high-dimensional problems where $p > n$; cases where sparsity is intrinsic to the scientific story; feature pre-screening before fitting a non-linear model.

12 Cross-Cutting Themes

12.1 Choosing a Measure: A Decision Tree

A condensed walk-through:

1. **What are you measuring — raw variables or a fitted model?**

- Raw variables → Parts I, II, III.
- Fitted model → Parts IV, V.

2. **For raw variables, what are the data types?**

- Continuous–continuous → Pearson (linear), Spearman (monotone), dCor / HSIC / MI (general).
- Continuous–ordinal → Spearman, Kendall; *polyserial* (not polychoric) if you believe a latent normal — polychoric requires *both* variables to be discretised latent-normals.
- Continuous–binary → Point-biserial (equivalent to Pearson on the 0/1 coding); rank-biserial / Mann–Whitney for non-linear monotone.
- Binary–binary → ϕ ; tetrachoric if you believe latent normal.
- Categorical–categorical → Cramér's V ; MI / NMI.
- Multivariate–multivariate → dCor, HSIC, KCCA; partial dCor for conditional analogues.
- Time series with direction → *Granger causality* for linear stationary VAR fits; *transfer entropy* as the nonlinear, non-parametric generalisation.
- Conditional dependence → Partial correlation (linear), CMI, conditional HSIC / HSCIC (Park & Muandet 2020), partial dCor (Székely & Rizzo 2014), conditional Chatterjee.

3. **How non-linear do you expect the relationship?**

- Linear → Pearson.
- Monotone → Spearman, Kendall.
- Functional non-monotone → Chatterjee ξ , dCor, HSIC, MI.
- Anything → MI, dCor, HSIC.

4. **Do you need a hypothesis test?**

- Pearson, Spearman, Kendall, ϕ , point-biserial, Cramér's V — closed-form tests.
- Chatterjee ξ — closed-form asymptotic normal.
- dCor, HSIC, MI, MIC, TE, CMI — permutation tests are the gold standard; gamma approximations for HSIC.

5. **For model feature importance, what kind of model and what question?**

- Tree ensemble + global ranking → TreeSHAP (preferred), or MDI / Gain (cheap).
- Any model + global ranking → PFI on held-out data.
- Any model + per-prediction explanation → SHAP.
- Per-prediction explanation of image / text → LIME.
- Visualising a feature's effect → PDP + ICE, or ALE for correlated features.
- Linear / GLM with sparsity → LASSO coefficients (after standardisation).

12.2 Significance, Permutation Tests, and Confidence Intervals

Every dependence measure in this guide can be turned into a hypothesis test of $H_0 : X \perp Y$. Two routes:

- **Asymptotic null distributions.** Available for Pearson, Spearman, Kendall, ϕ , point-biserial, Cramér's V , Chatterjee's ξ , and (with care) HSIC and dCor. Use when n is large and assumptions are met.

- **Permutation tests.** Compute D on the original data; randomly permute one variable many times and recompute D on each permutation; report the fraction of permutations exceeding the observed value as the p -value. Exact under exchangeability of the sample under H_0 (which holds for i.i.d. pairs under $X \perp Y$; fails for time series and repeated measures), up to Monte Carlo error; the universal fallback.

For **confidence intervals**, the **bootstrap** is universal: resample with replacement, recompute, report percentiles. For Pearson and Spearman, Fisher's z -transform gives narrower CIs under normality.

Two corner cases:

- The bootstrap can be **biased downward** for measures that are non-negative and bounded above (MI, dCor, HSIC); the **BCa** (bias-corrected and accelerated) bootstrap is preferred.
- For **time series**, ordinary permutation / bootstrap destroys temporal structure and is **invalid**. Use the **block bootstrap** with block length matched to the autocorrelation time scale.

12.3 Multiple Testing and the Garden of Forking Paths

Computing p -values for every pair of features in a 100-feature dataset gives 4{,}950 tests — at $\alpha = 0.05$, ~250 will fire even if every pair is independent. Two corrections:

- **Bonferroni:** α/m . Simple, conservative, controls **family-wise error rate (FWER)**.
- **Benjamini–Hochberg (BH):** sort p -values $p_{(1)} \leq \dots \leq p_{(m)}$; let K be the *largest* k with $p_{(k)} \leq k\alpha/m$, and reject all hypotheses with rank $\leq K$. Controls the **false discovery rate (FDR)** — the expected proportion of false positives among the rejections — when the test statistics are independent or positively dependent. More powerful than Bonferroni; the standard default for screening.
- **Benjamini–Yekutieli (BY)** (2001): replace the threshold by $p_{(k)} \leq k\alpha/(m \cdot H_m)$ with $H_m = \sum_{i=1}^m 1/i \approx \log m$. Controls FDR under *arbitrary* dependence between test statistics — the appropriate choice when feature-pair correlation tests are clearly not independent. More conservative than BH.

The deeper, harder problem is the **garden of forking paths** (Gelman & Loken 2013): even a single measure, computed across many plausible analytical choices (transformations, outlier rules, subsamples, model variants), implicitly searches a space of hypotheses without correction. Pre-registration, hold-out validation, and an honest accounting of how many models were tried are the only real defences.

12.4 Correlation, Causation, and Confounders

No measure in this document — not Pearson, not dCor, not SHAP, not Chatterjee's ξ — distinguishes correlation from causation on its own. They all summarise *statistical* dependence in the observed joint distribution. Common causes (confounders) produce association without causation; selection effects produce association where none exists in the underlying population; measurement error attenuates association even when causation is real.

The three main routes to causal claims:

1. **Randomised experiments.** The gold standard. Random assignment breaks confounding by construction.
2. **DAG-based observational reasoning** (Pearl, Imbens & Rubin, Hernán & Robins). Posit a causal DAG; use the back-door, front-door, or instrumental-variable criteria; estimate effects with the appropriate adjustment set.
3. **Natural experiments and quasi-experiments:** regression discontinuity, difference-in-differences, instrumental variables.

The dependence measures in this guide are inputs to step 2 — they identify candidate associations and CI relations — but they do not themselves produce causal claims. SHAP attributing high importance to a feature does **not** mean intervening on that feature changes the outcome. Conversely, a causally important feature can have small SHAP under a model that uses a proxy.

12.5 Pitfalls Common to All Metrics

A short, brutal checklist:

- **Sample size dominates.** Most measures are upward biased at small n — particularly MI, MIC, dCor, HSIC. Report estimators with sampling distributions, not as points.
 - **Outliers distort.** Pearson is the worst offender; rank-based and information-theoretic measures are more robust but not immune.
 - **Discretisation choices matter.** MI, MIC, transfer entropy depend on binning or kernel bandwidth.
 - **High dimensions are hard.** n/d ratios below 10 make non-parametric estimators very noisy.
 - **Stationarity for time series.** Without it, transfer entropy, autocorrelation, and most everything else are meaningless.
 - **Correlated features confuse importance.** SHAP, PFI, MDI, gain — all of them split credit between correlated features in ways that are hard to interpret.
 - **Training vs held-out data.** Importance measures computed on training data measure memorisation, not generalisation.
 - **A single number is rarely enough.** Always pair a coefficient with a plot. Anscombe's quartet exists for a reason.
-

13 Common Confusions and How to Resolve Them

The pairings below are the most-frequent confusions in practice. Each gets a one-paragraph resolution.

13.1 "Pearson vs covariance"

Covariance is unbounded and scale-dependent; Pearson is the scale-free version. $r = \text{Cov}(X, Y) / (\sigma_X \sigma_Y)$. Two datasets with covariance 5 and 50 may have identical Pearson if the marginal variances scale accordingly. **Report Pearson unless you specifically need the unit-bearing covariance** (e.g., for portfolio variance calculations where covariance contributes directly to $\sigma_p^2 = \sum_{ij} w_i w_j \sigma_{ij}$).

13.2 "Spearman vs Pearson — which to default to?"

If you don't know the relationship shape, **report both**. A large r with a similar ρ_S confirms linearity; a small r with a large ρ_S signals monotonic non-linearity; a small ρ_S with a large dCor signals non-monotonic dependence. Spearman is more robust by default; Pearson is the appropriate target when downstream methods (PCA, OLS, multivariate normal models) assume linearity.

13.3 "MI vs correlation — interpretive equivalence?"

For jointly normal (X, Y) with Pearson correlation ρ : $I(X; Y) = -\frac{1}{2} \log(1 - \rho^2)$. This is one of the few cases where MI and correlation map exactly. Outside normality, **MI captures dependences that correlation does not** (the $Y = X^2$ counterexample) and **correlation captures direction information that MI discards**. The two are complementary, not interchangeable.

13.4 "dCor vs HSIC"

Functionally near-equivalent: both are non-linear, both characterise independence, both are $O(n^2)$, both use permutation tests. dCor is parameter-free (distance is the only choice); HSIC requires kernel + bandwidth. **Default to dCor** for raw screening; switch to HSIC when you need kernel flexibility (e.g., to compare representations via CKA) or when the random-feature approximations make the large- n case tractable.

13.5 "SHAP vs PFI"

Both are model-agnostic global importance scores, both popular, often producing similar rankings. Differences:

- **SHAP**: per-prediction first, aggregate second; principled (Shapley axioms); reveals direction (positive/negative effect); slow without TreeSHAP.
- **PFI**: aggregate only; not axiomatic; the score-change is *signed* (positive when the feature helps) but does not encode direction of effect on individual predictions; fast.

Use SHAP when individual explanations matter or when direction is informative. Use PFI for a quick, model-agnostic global ranking. **Do not average their rankings naively** — they answer slightly different questions.

13.6 "MDI vs PFI vs SHAP for tree ensembles"

- **MDI** is cheapest, computed during training, but biased toward high-cardinality features.
- **PFI on held-out data** removes the bias and reflects generalisation, but suffers correlated-feature instability.
- **TreeSHAP** is the principled gold standard; exact, fast for trees, axiomatic, reveals direction.

For research or production decisions, prefer **TreeSHAP**. MDI is for triage.

13.7 "LIME vs SHAP"

LIME approximates the model locally with a sparse linear surrogate fitted on perturbed samples. SHAP computes Shapley values of features, providing exact additive decomposition. **SHAP can be derived as a special-case linear regression on coalitions with specific weights** (the "kernel SHAP" reformulation), placing LIME and SHAP in the same family — but LIME's kernel and sampling distribution generally fail to recover the Shapley axioms. Use SHAP when axiomatic guarantees matter.

13.8 "Granger causality vs transfer entropy"

For jointly Gaussian linear stationary processes, transfer entropy is monotone in the Granger F -statistic. **TE is the nonlinear, non-parametric generalisation.** Use Granger when you have a fitted linear VAR you trust; use TE when you suspect non-linear dynamics, but be aware that TE needs substantially more data ($n \gtrsim 10^4$) for reliable estimates.

13.9 "Partial correlation vs CMI"

Partial correlation $r_{XY \cdot Z}$ removes the **linear** effect of Z before computing Pearson on the residuals; in closed form,

$$r_{XY \cdot Z} = \frac{r_{XY} - r_{XZ}r_{YZ}}{\sqrt{(1 - r_{XZ}^2)(1 - r_{YZ}^2)}}.$$

CMI removes the **full statistical** effect of Z . They agree under joint normality and diverge under non-linearity. **Use partial correlation for linear-Gaussian intuition; CMI for general causal-discovery contexts.**

13.10 "Correlation between binary variables — should I use ϕ or the odds ratio?"

ϕ is symmetric, bounded in $[-1, 1]$, and is exactly Pearson on the 0/1 codings — but its maximum depends on the marginals. OR is invariant to row/column rescaling and to the sampling design (prospective vs retrospective), making it the standard in epidemiology. **Report both for 2×2 tables:** ϕ as effect-size summary, OR for design-invariant inference.

13.11 "Adjusted Rand Index vs Adjusted Mutual Information for clustering comparison"

ARI is pair-counting; AMI is information-theoretic. Both correct for chance. ARI tends to penalise differences in cluster shape more harshly; AMI penalises information loss more. In a horse race they usually agree; when they don't, the difference reveals where the partitions diverge. **Vinh, Epps & Bailey (2010) recommend AMI** when clusters are highly unbalanced; **ARI** when they are roughly balanced.

13.12 "Why does my training-set PFI show high importance for noise features?"

Because PFI on training data measures memorisation: a tree that memorised noise sees its training accuracy drop when the noise is shuffled. **Always evaluate PFI on held-out data.** The same logic explains why MDI of pure-noise high-cardinality features can be substantial — both are training-time signals.

13.13 "Why don't my SHAP values sum to the prediction?"

They should — exactly. If they don't, check:

1. **Baseline expectation:** the decomposition is $f(\mathbf{x}) = \mathbb{E}[f(\mathbf{X})] + \sum_i \phi_i$. Is `explainer.expected_value` defined on the same scale as `model.predict()`?

2. **Path-dependent vs interventional TreeSHAP.** The two modes use different baselines and different value functions; switching between them changes both $\mathbb{E}[f(\mathbf{X})]$ and the ϕ_i . Mixing one mode's baseline with another's ϕ_i breaks additivity.
 3. **Sampling estimator:** KernelSHAP with too few samples produces noisy attributions that don't sum exactly. Increase `nsamples`.
 4. **Multiclass models:** SHAP returns one value per class; the per-class decomposition holds, but you must pick a class.
 5. **Log-odds vs probability:** tree classifiers return probabilities, but TreeSHAP may operate on margins. Use `model_output="probability"` or convert manually.
-

14 Glossary

A one-line definition for every term used in the guide that might require a refresher. Bold terms cross-reference defining sections.

- **AMI** — *Adjusted Mutual Information*. NMI with chance-correction. §10.
- **Affine invariance** — invariance under $X \mapsto aX + b$.
- **ALE** — *Accumulated Local Effects*. PDP variant that avoids extrapolation under feature correlation. §21.
- **Asymptotic normality** — convergence of a properly scaled statistic to a normal distribution as $n \rightarrow \infty$.
- **BCa bootstrap** — *Bias-Corrected and accelerated bootstrap*. Improved bootstrap CI for skewed sampling distributions.
- **Bernoulli distribution** — random variable taking value 1 with probability p , 0 otherwise.
- **Bias (estimator)** — $\mathbb{E}[\hat{\theta}] - \theta$.
- **Bootstrap** — resampling with replacement to approximate the sampling distribution of an estimator. P6.
- **Brownian motion** — continuous-time stochastic process with independent Gaussian increments. §14.
- **CCA** — *Canonical Correlation Analysis*. Finds linear combinations of two random vectors with maximum correlation. §15.
- **Characteristic function** — $\varphi_X(t) = \mathbb{E}[e^{itX}]$. P3.
- **Characteristic kernel** — kernel whose mean embedding is injective in the underlying distribution. P4.
- **CKA** — *Centered Kernel Alignment*. Cosine version of HSIC. §16.
- **CMI** — *Conditional Mutual Information*. §12.
- **Concordant pair** — $(x_i, y_i), (x_j, y_j)$ such that $(x_i - x_j)(y_i - y_j) > 0$. §3.
- **Conditional independence** — $X \perp Y \mid Z$, meaning $p_{XY|Z} = p_{X|Z}p_{Y|Z}$.
- **Confounder** — a variable that causally influences both treatment and outcome, producing spurious association.
- **Copula** — a CDF on $[0, 1]^d$ with uniform marginals, encoding the dependence structure separated from marginals. P7.
- **Cramér's V** — categorical-categorical association measure based on χ^2 . §6.
- **Cross-covariance operator** — operator $\Sigma_{XY} : \mathcal{H}_Y \rightarrow \mathcal{H}_X$ such that $\text{Cov}(f(X), g(Y)) = \langle f, \Sigma_{XY}g \rangle$. §16.
- **dCor** — *distance correlation*. §13.
- **DAG** — *directed acyclic graph*. Used in causal inference. P8.
- **Doubly centred matrix** — matrix with both row and column means subtracted (plus the grand mean). §13.
- **Eigenvalue problem** — finding (λ, v) with $Av = \lambda v$.
- **Elastic Net** — $L_1 + L_2$ regularised regression. §25.
- **Entropy (Shannon)** — $H(X) = -\sum p(x) \log p(x)$. P2.
- **Equitability** — proposed property of MIC: equally noisy relationships should receive equal scores. §9.
- **Feature map (kernel)** — function $\phi : \mathcal{X} \rightarrow \mathcal{H}$ embedding inputs into an RKHS. P4.
- **Fisher's z -transform** — $z = \text{arctanh}(r)$, used to stabilise the variance of Pearson's r . §1.
- **Functional dependence** — Y is a measurable function of X . §17.
- **Gain importance** — sum of loss reductions attributable to splits on a feature in boosted trees. §24.
- **Gaussian process** — collection of random variables, any finite subset of which is jointly Gaussian.
- **Gini index** — impurity measure $1 - \sum_k p_k^2$. §22.
- **Granger causality** — X Granger-causes Y if past of X improves prediction of Y beyond past of Y . P8, §11.
- **HSIC** — *Hilbert-Schmidt Independence Criterion*. §16.
- **ICE** — *Individual Conditional Expectation*. PDP without averaging. §21.
- **Independence** — $X \perp Y$ means $p_{XY} = p_X p_Y$.

- **Interventional vs observational SHAP** — choice of value function in SHAP; one breaks feature correlations, the other respects them. §18.
- **Invariance** — preservation under transformation. Pearson under affine, Spearman under monotone, MI under bijective.
- **Jackknife** — leave-one-out resampling. P6.
- **KCCA** — *Kernel CCA*. §15.
- **Kendall's τ** — rank correlation based on concordant / discordant pairs. §3.
- **Kernel** — symmetric positive-definite function $k : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$. P4.
- **KL divergence** — $D_{\text{KL}}(p||q) = \sum p \log(p/q)$. P2.
- **KSG estimator** — Kraskov–Stögbauer–Grassberger kNN-based MI estimator. §8.
- **LASSO** — L_1 regularised regression. §25.
- **Latent variable** — unobserved variable inferred from observed indicators (e.g. tetrachoric model). §7.
- **LIME** — *Local Interpretable Model-agnostic Explanations*. §20.
- **Linear-Gaussian** — assumption that variables are linearly related with Gaussian noise.
- **MAD** — *Median Absolute Deviation*. Robust scale estimate.
- **Marginal distribution** — distribution of a single variable from a joint, obtained by integrating out the others.
- **MDA** — *Mean Decrease in Accuracy*. §23.
- **MDI** — *Mean Decrease in Impurity*. §22.
- **MIC** — *Maximal Information Coefficient*. §9.
- **MI** — *Mutual Information*. §8.
- **Monotone transformation** — strictly increasing (or strictly decreasing) function.
- **Multicollinearity** — high correlation among predictors in a regression.
- **NMI** — *Normalized Mutual Information*. §10.
- **Odds ratio** — $\text{OR} = (ad)/(bc)$ in a 2×2 table. §5.
- **OLS** — *Ordinary Least Squares*. Linear regression by minimising squared residuals.
- **Out-of-bag (OOB)** — in a bootstrapped tree, the samples *not* used to build that tree; provides a free held-out set. §23.
- **PDP** — *Partial Dependence Plot*. §21.
- **Permutation test** — hypothesis test by shuffling labels. P6.
- **PFI** — *Permutation Feature Importance*. §19.
- **ϕ coefficient** — Pearson on a 2×2 table. §5.
- **PMF / PDF / CDF** — probability mass / density / cumulative distribution function.
- **Point-biserial** — Pearson with one binary variable. §4.
- **Polychoric correlation** — latent-normal correlation estimated from ordinal data. §7.
- **Power (statistical)** — $1 - P(\text{type II error})$; probability of rejecting H_0 when it is false.
- **Probit transform** — $\Phi^{-1}(p)$.
- **Ranks** — order statistic positions of observations.
- **RBF kernel** — $k(x, y) = \exp(-\|x - y\|^2 / 2\sigma^2)$.
- **Reproducing kernel Hilbert space (RKHS)** — Hilbert space of functions in which evaluation is a continuous linear functional. P4.
- **Ridge** — L_2 regularised regression. §25.
- **SHAP** — *SHapley Additive exPlanations*. §18.
- **Shapley value** — unique fair-allocation rule from cooperative game theory. P9, §18.
- **Spearman's ρ** — Pearson applied to ranks. §2.
- **Stationarity** — distributional invariance under time shifts.
- **TE** — *Transfer Entropy*. §11.
- **Tetrachoric correlation** — polychoric specialised to 2×2 tables. §7.
- **TreeSHAP** — polynomial-time exact SHAP for tree ensembles. §18.
- **U-statistic** — unbiased estimator written as an average over k -tuples. P5.
- **Variational bound (MI)** — neural-network-trained lower bound on MI (MINE, InfoNCE, NWJ). §8.

15 Appendix A — Reference Implementations and Libraries

15.1 Python

Measure	Recommended library / call
Pearson, Spearman, Kendall	<code>scipy.stats.{pearsonr, spearmanr, kendalltau}</code> ; <code>pandas.DataFrame.corr</code>
Point-biserial, ϕ	<code>scipy.stats.pointbiserialr</code> ; Pearson on 0/1 codings
Cramér's V	<code>scipy.stats.contingency.association(method='cramer')</code> or manually via <code>chi2_contingency</code>
Tetrachoric / Polychoric	<code>factor_analyzer.utils.polychoric_correlation</code> ; <code>semopy</code> ; <code>rpy2</code> to <code>psych::polychoric</code>
Mutual information	<code>sklearn.feature_selection.mutual_info_{regression,classif}</code> (KSG kNN); <code>pyitlib</code> ;
MIC	<code>minepy</code>
NMI / AMI / ARI	<code>sklearn.metrics.{normalized_mutual_info_score, adjusted_mutual_info_score, ad</code>
Transfer entropy	<code>pyinform.transferentropy</code> ; <code>IDTx1</code> ; <code>RTransferEntropy</code> (R)
Conditional MI	<code>npeet</code> ; <code>causal-learn</code> (CI tests)
Distance correlation	<code>dcor</code> ; <code>hyppo</code>
HSIC, CKA	<code>hyppo</code> ; <code>dcor.hsic</code> ; <code>torch_cka</code> ; <code>pyHSICLasso</code>
KCCA	<code>pyrcca</code> ; <code>mvlearn</code>
Chatterjee's ξ	<code>xicor</code>
SHAP	<code>shap</code> (<code>TreeExplainer</code> , <code>KernelExplainer</code> , <code>DeepExplainer</code> , <code>GradientExplainer</code>)
Permutation importance	<code>sklearn.inspection.permutation_importance</code> ; <code>eli5</code>
LIME	<code>lime</code> (<code>lime_tabular</code> , <code>lime_text</code> , <code>lime_image</code>)
PDP / ICE	<code>sklearn.inspection.PartialDependenceDisplay</code> ; <code>pdpbox</code>
ALE	<code>PyALE</code> ; <code>alibi.explainers.ALE</code>
MDI	<code>RandomForest*.feature_importances_</code>
MDA	<code>sklearn.inspection.permutation_importance</code> (on OOB or held-out)
Gain / split / cover	<code>xgb_model.get_booster().get_score(importance_type=...)</code> ; <code>lgb_model.feature_importances_</code>
LASSO / Ridge / EN	<code>sklearn.linear_model.{Lasso, Ridge, ElasticNet}CV</code> ; <code>glmnet-python</code>
Stability Selection	<code>stability-selection</code> package; <code>mlxtend.feature_selection.SequentialFeatureSelector</code>
Group LASSO	<code>group-lasso</code> ; <code>celer</code>

15.2 R (selected canonical references)

`stats::cor` (Pearson, Spearman, Kendall); `psych::polychoric`; `energy::dcor` (the reference `dCor` implementation); `kpcalg`, `dHSIC` (HSIC); `XICOR` (Chatterjee); `minerva` (MIC); `infotheo` (MI); `RTransferEntropy`; `iml`, `DALEX`, `shapr`, `fastshap` (model interpretability); `glmnet` (LASSO / Ridge / EN); `ranger`, `randomForest`, `party::cforest` (forests with importance variants); `selectiveInference` (post-LASSO inference).

16 Appendix B — A Complete Numerical Walkthrough with Code

A single end-to-end example that exercises every major family of measures on the same synthetic dataset. Copy and run.

16.1 B.1 Generate a synthetic dataset

Six features, varying relationships to a target. Designed so each measure has at least one "win" — a relationship it detects that some other measure misses.

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(42)
n = 2000

# Features
x_lin   = rng.uniform(-1, 1, n)           # linear signal
x_mono  = rng.uniform(-1, 1, n)           # monotone non-linear signal
x_quad  = rng.uniform(-1, 1, n)           # quadratic (non-monotonic) signal
x_sin   = rng.uniform(0, 2*np.pi, n)     # sinusoidal signal
x_corr  = x_lin + 0.3 * rng.normal(size=n) # correlated with x_lin
x_noise = rng.normal(size=n)              # pure noise

# Target: depends on lin, mono, quad, sin; noise terms added
y = (
    1.5 * x_lin
    + np.tanh(3 * x_mono)
    + 1.0 * x_quad ** 2
    + 0.6 * np.sin(x_sin)
    + 0.3 * rng.normal(size=n)
)

X = np.column_stack([x_lin, x_mono, x_quad, x_sin, x_corr, x_noise])
feature_names = ["lin", "mono", "quad", "sin", "corr_lin", "noise"]
df = pd.DataFrame(X, columns=feature_names)
df["y"] = y
print(df.head())
```

16.2 B.2 Classical correlations

```
from scipy import stats

print(f"{'feature':10s} {'Pearson':>9s} {'Spearman':>9s} {'Kendall':>9s}")
for fname in feature_names:
    r, _ = stats.pearsonr(df[fname], df["y"])
    rho, _ = stats.spearmanr(df[fname], df["y"])
    tau, _ = stats.kendalltau(df[fname], df["y"])
    print(f"{'fname':10s} {r:>9.3f} {rho:>9.3f} {tau:>9.3f}")
```

Expected pattern: lin and mono show high values in all three; quad and sin show ~0 (Pearson blind to non-monotone); corr_lin close to lin; noise ~0.

16.3 B.3 Mutual information (KSG)

```
from sklearn.feature_selection import mutual_info_regression

mi = mutual_info_regression(X, y, n_neighbors=5, random_state=0)
print(f"\n{'feature':10s} {'MI (nats)':>10s}")
for fname, m in zip(feature_names, mi):
    print(f"{'fname':10s} {m:>10.3f}")
```

quad and sin will now have substantial MI despite zero correlation — the canonical demonstration of why MI exists.

16.4 B.4 Distance correlation and HSIC

```
import dcor
from hyppo.independence import Hsic

print(f"\n{'feature':10s} {'dCor':>9s} {'HSIC stat':>10s} {'HSIC p':>8s}")
for fname in feature_names:
    dc = dcor.distance_correlation(df[fname].values, y)
    hs_stat, hs_p = Hsic().test(
        df[fname].values.reshape(-1, 1), y.reshape(-1, 1),
        reps=999, workers=-1, random_state=0,
    )
    print(f"{fname:10s} {dc:>9.3f} {hs_stat:>10.4f} {hs_p:>8.3g}")
```

dCor and HSIC will detect quad and sin cleanly; noise should be small with a non-significant p .

16.5 B.5 Chatterjee's ξ

```
from xicor.xicor import Xi

print(f"\n{'feature':10s} {'xi(X,Y)':>9s} {'xi(Y,X)':>9s} {'p':>8s}")
for fname in feature_names:
    xi_xy = Xi(df[fname].values, y)
    xi_yx = Xi(y, df[fname].values)
    print(f"{fname:10s} {xi_xy.correlation:>9.3f} {xi_yx.correlation:>9.3f}
          {xi_xy.pval_asymptotic(ties=False):>8.3g}")
```

Note the asymmetry: $\xi(X, Y)$ and $\xi(Y, X)$ differ — informative when one is closer to 1 than the other.

16.6 B.6 Permutation feature importance

```
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import train_test_split
from sklearn.inspection import permutation_importance

Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=0)
model = RandomForestRegressor(n_estimators=400, random_state=0, n_jobs=-1).fit(Xtr, ytr)
print(f"\nR2 on test: {model.score(Xte, yte):.3f}")

result = permutation_importance(model, Xte, yte, n_repeats=30, random_state=0, n_jobs=-1)
print(f"\n{'feature':10s} {'PFI mean':>9s} {'PFI std':>9s}")
for fname, m, s in zip(feature_names, result.importances_mean, result.importances_std):
    print(f"{fname:10s} {m:>9.4f} {s:>9.4f}")
```

lin and corr_lin will share importance (correlated-feature instability); noise should be ~0.

16.7 B.7 SHAP (TreeSHAP)

```
import shap

explainer = shap.TreeExplainer(model, feature_perturbation="interventional", data=Xtr[:500])
# shap >= 0.43 prefers `explainer(Xte)` which returns an Explanation object;
# `shap_values` here returns a plain numpy array for clarity in this walkthrough.
shap_values = explainer.shap_values(Xte)

mean_abs_shap = np.abs(shap_values).mean(axis=0)
print(f"\n{'feature':10s} {'mean |SHAP|':>11s}")
for fname, m in zip(feature_names, mean_abs_shap):
```

```

print(f"{fname:10s} {m:>11.4f}")

# Verify additivity for the first prediction
i = 0
print(f"\nFor row {i}:")
print(f"  baseline + sum(SHAP) = {explainer.expected_value + shap_values[i].sum():.4f}")
print(f"  model.predict()      = {model.predict(Xte[i:i+1])[0]:.4f}")

```

16.8 B.8 LASSO with cross-validated λ

```

from sklearn.linear_model import LassoCV
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

# Augment with explicit non-linear features so LASSO has something to grab
X_aug = np.column_stack([X, X[:, 2] ** 2, np.sin(X[:, 3])])
aug_names = feature_names + ["quad2", "sin(x_sin)"]

lasso = make_pipeline(StandardScaler(), LassoCV(cv=5, random_state=0, n_jobs=-1)).fit(X_aug, y)
# NOTE: coefficients live in the standardised-feature space (each X column
# was rescaled to unit variance). Magnitudes are comparable across features
# (all on the same scale) but should not be read as "effect per raw unit of X".
beta = lasso[-1].coef_

print(f"\n{'feature':12s} {'LASSO  $\beta$  (std)':>14s}")
for fname, b in zip(aug_names, beta):
    print(f"{fname:12s} {b:>+14.3f}")
print(f"Chosen alpha (lambda): {lasso[-1].alpha_:.4f}")

```

Without the engineered features, LASSO finds only `lin`, `mono`, and `corr_lin`; with the engineered `quad2` and `sin(x_sin)`, it recovers them too — illustrating the inherent limitation of linear feature-importance under non-linearity.

16.9 B.9 Putting it all together

Run all of the above and you should observe:

Feature	Pearson	Spearman	MI	dCor	ξ	PFI	SHAP	LASSO (raw)
lin	high	high	high	high	high	high	high	high
mono	high	higher	high	high	high	high	high	high
quad	~0	~0	mid	mid	mid	mid	mid	~0
sin	~0	~0	mid	mid	mid	mid	mid	~0
corr_lin	high	high	high	high	high	low*	low*	mid
noise	~0	~0	~0	~0	~0	~0	~0	0

* PFI and SHAP for `corr_lin` are low because the model uses `lin` instead; they would swap if the seed changes.

The takeaway: no single measure tells the whole story. Run a battery, look at where they agree and where they don't, and let the disagreements teach you something about the data.

17 Appendix C — Annotated Bibliography

Each entry is brief — what the paper introduces or settles, and why you should read it.

17.1 Classical and rank correlation

- **Pearson, K. (1895)**, *Notes on regression and inheritance in the case of two parents*. Proc. R. Soc. Lond. — Founding paper for the product-moment correlation.
- **Spearman, C. (1904)**, *The proof and measurement of association between two things*. Am. J. Psych. — Introduces rank correlation.
- **Kendall, M. G. (1938)**, *A new measure of rank correlation*. Biometrika — Introduces τ .
- **Knight, W. R. (1966)**, *A computer method for calculating Kendall's tau*. JASA — The $O(n \log n)$ algorithm.
- **Cramér, H. (1946)**, *Mathematical Methods of Statistics*. Princeton — Introduces V .
- **Bergsma, W. (2013)**, *A bias-correction for Cramér's V and Tschuprow's T* . J. Korean Stat. Soc. — The recommended small-sample correction.
- **Olkin, I., Pratt, J. (1958)**, *Unbiased estimation of certain correlation coefficients*. Ann. Math. Stat. — Classical bias corrections.
- **Hotelling, H. (1936)**, *Relations between two sets of variates*. Biometrika — Introduces CCA.

17.2 Information theory and entropy-based

- **Shannon, C. E. (1948)**, *A mathematical theory of communication*. Bell Sys. Tech. J. — Foundational; defines mutual information.
- **Kullback, S., Leibler, R. A. (1951)**, *On information and sufficiency*. Ann. Math. Stat. — Introduces KL divergence.
- **Cover, T. M., Thomas, J. A. (2006)**, *Elements of Information Theory* (2nd ed.). Wiley — The standard textbook reference.
- **Miller, G. A. (1955)**, *Note on the bias of information estimates*. Information Theory in Psychology — Miller-Madow bias correction.
- **Kraskov, A., Stögbauer, H., Grassberger, P. (2004)**, *Estimating mutual information*. PRE 69 — The kNN (KSG) estimator used everywhere.
- **Belghazi, M. I. et al. (2018)**, *Mutual information neural estimation*. ICML — MINE.
- **Van den Oord, A., Li, Y., Vinyals, O. (2018)**, *Representation learning with contrastive predictive coding*. arXiv:1807.03748 — InfoNCE.
- **McAllester, D., Stratos, K. (2020)**, *Formal limitations on the measurement of mutual information*. AISTATS — Limits of variational MI estimators.
- **Reshef, D. N. et al. (2011)**, *Detecting novel associations in large data sets*. Science 334 — Introduces MIC.
- **Kinney, J. B., Atwal, G. S. (2014)**, *Equitability, mutual information, and the maximal information coefficient*. PNAS — The principled critique of MIC's equitability.
- **Reshef, Y. A. et al. (2016)**, *Measuring dependence powerfully and equitably*. JMLR — MICe and reformulated equitability.
- **Schreiber, T. (2000)**, *Measuring information transfer*. PRL 85 — Introduces transfer entropy.
- **Barnett, L., Barrett, A. B., Seth, A. K. (2009)**, *Granger causality and transfer entropy are equivalent for Gaussian variables*. PRL 103 — The TE-Granger bridge.
- **Vinh, N. X., Epps, J., Bailey, J. (2010)**, *Information theoretic measures for clusterings comparison*. JMLR — AMI and rigorous treatment of NMI variants.

17.3 Foundations of dependence

- **Rényi, A. (1959)**, *On measures of dependence*. Acta Math. Hung. — Seven axioms a "good" dependence measure should satisfy; the foundational reference no measure in this guide fully satisfies.
- **Hoeffding, W. (1948)**, *A non-parametric test of independence*. Ann. Math. Stat. — The D -statistic; historical antecedent of dCor and HSIC.

- Embrechts, P., McNeil, A., Straumann, D. (2002), *Correlation and dependence in risk management: properties and pitfalls*. In *Risk Management*. — The standard modern reference on copula-based dependence, marginal-invariance, and what goes wrong with linear correlation under heavy tails.

17.4 Geometric, kernel, and modern independence

- Székely, G., Rizzo, M., Bakirov, N. (2007), *Measuring and testing dependence by correlation of distances*. Ann. Stat. 35 — Introduces dCor.
- Székely, G., Rizzo, M. (2009), *Brownian distance covariance*. Ann. Appl. Stat. — The Brownian covariance equivalence.
- Székely, G., Rizzo, M. (2014), *Partial distance correlation with methods for dissimilarities*. Ann. Stat. — Bias-corrected dCov and partial dCor.
- Huo, X., Székely, G. (2016), *Fast computing for distance covariance*. Technometrics — The $O(n \log n)$ univariate algorithm.
- Gretton, A. et al. (2005), *Measuring statistical dependence with Hilbert–Schmidt norms*. ALT — Introduces HSIC.
- Gretton, A. et al. (2007), *A kernel statistical test of independence*. NeurIPS — Gamma approximation to the HSIC null.
- Sejdinovic, D. et al. (2013), *Equivalence of distance-based and RKHS-based statistics in hypothesis testing*. Ann. Stat. — Unifies dCor and HSIC under a kernel framework.
- Cortes, C., Mohri, M., Rostamizadeh, A. (2012), *Algorithms for learning kernels based on centered alignment*. JMLR — Origin of centred kernel alignment.
- Kornblith, S. et al. (2019), *Similarity of neural network representations revisited*. ICML — Popularises CKA for neural network analysis.
- Yamada, M. et al. (2014), *High-dimensional feature selection by feature-wise kernelized lasso*. Neural Computation — HSIC Lasso.
- Park, J., Muandet, K. (2020), *A measure-theoretic approach to kernel conditional mean embeddings*. NeurIPS — Conditional HSIC / HSCIC.
- Bach, F. R., Jordan, M. I. (2002), *Kernel independent component analysis*. JMLR — KCCA and kernel ICA.
- Andrew, G. et al. (2013), *Deep canonical correlation analysis*. ICML — Deep CCA.
- Chatterjee, S. (2021), *A new coefficient of correlation*. JASA 116 — Introduces ξ .
- Azadkia, M., Chatterjee, S. (2021), *A simple measure of conditional dependence*. Ann. Stat. — The conditional Chatterjee extension.
- Lin, Z., Han, F. (2023), *On boosting the power of Chatterjee's rank correlation*. Biometrika — Improved finite-sample variant.

17.5 Tetrachoric / polychoric

- Pearson, K. (1900), *Mathematical contributions to the theory of evolution. VII. On the correlation of characters not quantitatively measurable*. Phil. Trans. Roy. Soc. — Original tetrachoric.
- Olsson, U. (1979), *Maximum likelihood estimation of the polychoric correlation coefficient*. Psychometrika — The standard MLE.
- Foldnes, N., Grønneberg, S. (2021), *The sensitivity of polychoric correlation to non-normality*. Brit. J. Math. Stat. Psych. — Robustness diagnostics.

17.6 Model-agnostic feature importance

- Ribeiro, M. T., Singh, S., Guestrin, C. (2016), *"Why should I trust you?" Explaining the predictions of any classifier*. KDD — LIME.
- Lundberg, S., Lee, S.-I. (2017), *A unified approach to interpreting model predictions*. NeurIPS — Introduces SHAP.
- Lundberg, S. et al. (2018), *Consistent individualized feature attribution for tree ensembles*. arXiv:1802.03888 — TreeSHAP.

- Janzing, D., Minorics, L., Blöbaum, P. (2020), *Feature relevance quantification in explainable AI: a causal problem*. AISTATS — The interventional-vs-conditional SHAP debate.
- Aas, K., Jullum, M., Løland, A. (2021), *Explaining individual predictions when features are dependent*. Artif. Intell. — SHAP under correlated features.
- Strobl, C. et al. (2008), *Conditional variable importance for random forests*. BMC Bioinformatics — Conditional PFI.
- Hooker, G., Mentch, L., Zhou, S. (2021), *Unrestricted permutation forces extrapolation*. Stat. Comput. — The PFI extrapolation critique and LOCO.
- Friedman, J. H. (2001), *Greedy function approximation: a gradient boosting machine*. Ann. Stat. — Boosting + PDP.
- Goldstein, A. et al. (2015), *Peeking inside the black box*. JCGS — ICE plots.
- Apley, D., Zhu, J. (2020), *Visualizing the effects of predictor variables in black box supervised learning models*. JRSS-B — ALE plots.

17.7 Tree and linear feature importance

- Breiman, L. (2001), *Random forests*. ML 45 — Introduces MDA and the modern random forest.
- Strobl, C. et al. (2007), *Bias in random forest variable importance measures*. BMC Bioinformatics — The canonical MDI critique.
- Hothorn, T., Hornik, K., Zeileis, A. (2006), *Unbiased recursive partitioning*. JCGS — Conditional inference trees.
- Lei, J., G'Sell, M., Rinaldo, A., Tibshirani, R. J., Wasserman, L. (2018), *Distribution-free predictive inference for regression*. JASA — LOCO with conformal-based importance intervals.
- Covert, I., Lundberg, S., Lee, S.-I. (2020), *Understanding global feature contributions with additive importance measures*. NeurIPS — SAGE, the global Shapley-based feature-importance framework.
- Apley, D., Zhu, J. (2020), *Visualizing the effects of predictor variables in black box supervised learning models*. JRSS-B — ALE and M-plots; the conditional and conditional-corrected analogues of PDP.
- Friedman, J., Popescu, B. (2008), *Predictive learning via rule ensembles*. Ann. Appl. Stat. — Friedman's H -statistic for interaction strength.
- Li, X., Wang, Y., Singh, R. (2019), *A debiased MDI feature importance measure for random forests*. NeurIPS — Modern unbiased MDI.
- Tibshirani, R. (1996), *Regression shrinkage and selection via the lasso*. JRSS-B 58 — LASSO.
- Hoerl, A., Kennard, R. (1970), *Ridge regression*. Technometrics — Ridge.
- Zou, H., Hastie, T. (2005), *Regularization and variable selection via the elastic net*. JRSS-B — Elastic Net.
- Meinshausen, N., Bühlmann, P. (2010), *Stability selection*. JRSS-B — Stable LASSO.
- Zou, H. (2006), *The adaptive lasso and its oracle properties*. JASA — Adaptive LASSO.
- Friedman, J., Hastie, T., Tibshirani, R. (2007), *Pathwise coordinate optimization*. Ann. Appl. Stat. — Coordinate descent for LASSO.
- Lee, J. et al. (2016), *Exact post-selection inference, with application to the lasso*. Ann. Stat. — Selective inference.

17.8 Further modern non-parametric measures and tests

- Heller, R., Heller, Y., Gorfine, M. (2013), *A consistent multivariate test of association based on ranks of distances*. Biometrika — The HHG independence test; a powerful rank-of-distance alternative to dCor.
- López-Paz, D., Hennig, P., Schölkopf, B. (2013), *The randomized dependence coefficient*. NeurIPS — RDC; a scalable non-linear coefficient based on random non-linear projections + canonical correlation.
- Bergsma, W., Dassios, A. (2014), *A consistent test of independence based on a sign covariance related to Kendall's tau*. Bernoulli — The τ^* statistic.

- **Azadkia, M., Chatterjee, S. (2021)**, *A simple measure of conditional dependence*. Ann. Stat. — Conditional Chatterjee coefficient and the FOCI feature-selection algorithm.
- **Zhang, K., Peters, J., Janzing, D., Schölkopf, B. (2011)**, *Kernel-based conditional independence test*. UAI — KCIT.
- **Shah, R., Peters, J. (2020)**, *The hardness of conditional independence testing and the generalised covariance measure*. Ann. Stat. — GCM; a benchmark conditional-independence test with quantitative inference guarantees.

17.9 Causation, multiplicity, foundations

- **Pearl, J. (2009)**, *Causality* (2nd ed.). CUP — The reference for causal reasoning beyond correlation.
- **Hernán, M., Robins, J. (2020)**, *Causal Inference: What If*. CRC Press — A practitioner-oriented complement to Pearl.
- **Benjamini, Y., Hochberg, Y. (1995)**, *Controlling the false discovery rate*. JRSS-B — The BH procedure.
- **Benjamini, Y., Yekutieli, D. (2001)**, *The control of the false discovery rate in multiple testing under dependency*. Ann. Stat. — BY procedure (FDR control under arbitrary dependence); the appropriate default when feature-pair tests are not independent.
- **Gelman, A., Loken, E. (2013)**, *The garden of forking paths*. Manuscript — On the implicit multiple-comparisons problem.
- **Sklar, A. (1959)**, *Fonctions de répartition à n dimensions et leurs marges*. Publ. Inst. Stat. Univ. Paris — Sklar's theorem on copulas.

End of guide.